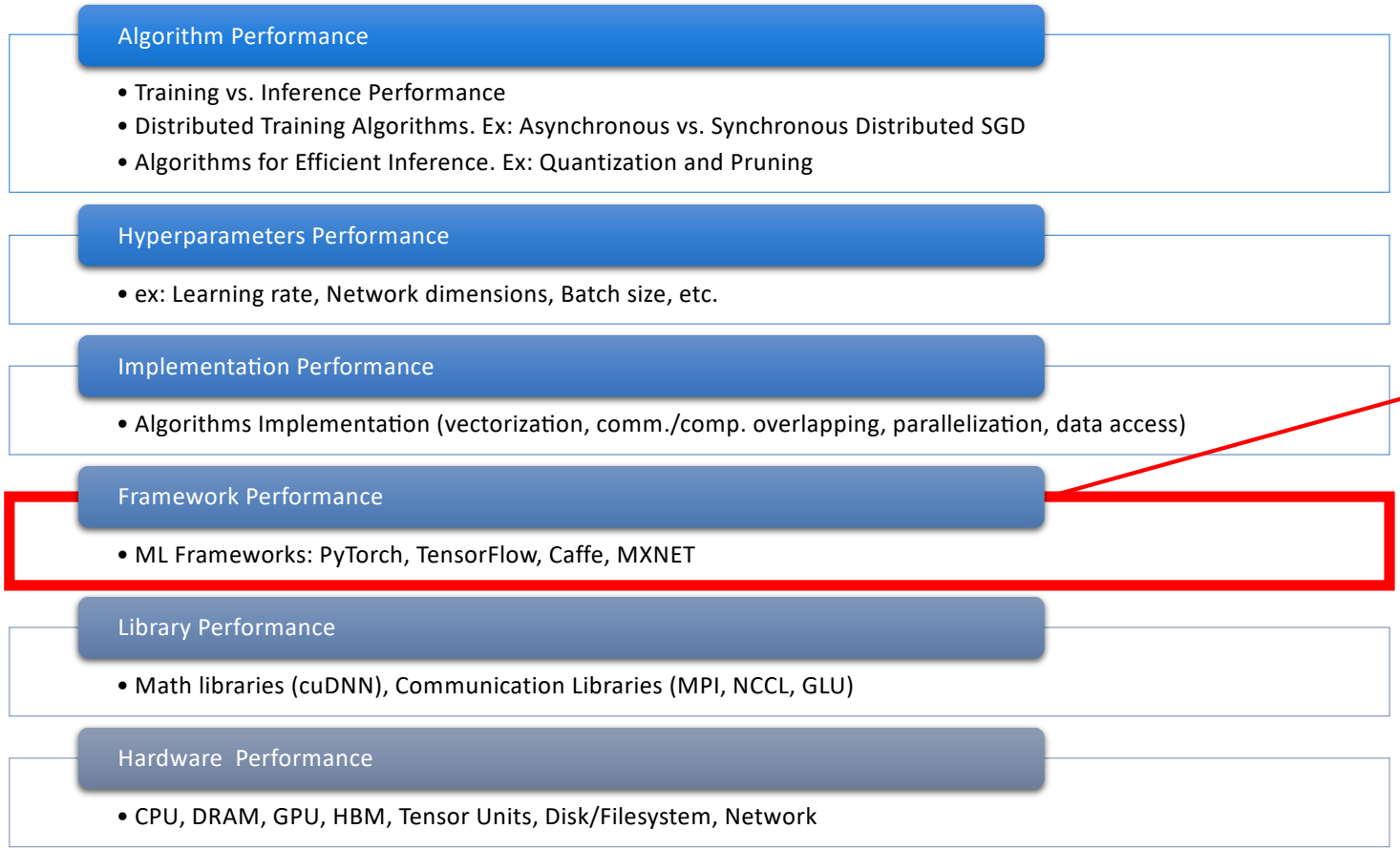
Introduction to High-Performance Machine Learning

Lecture 3 16/09/26

Dr. Kaoutar El Maghraoui

Python and PyTorch performance

ML Performance Factors



Focus for today's lecture

Outline

- Python performance
- PyTorch performance
  - Key Concepts
  - Computation Graph Approach
  - Torch.compile

Python Performance

Python

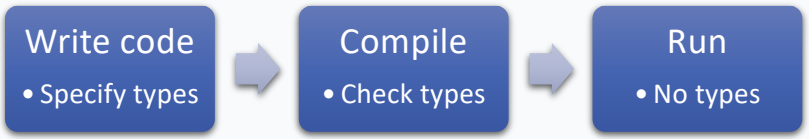
- Created in 1991 by Guido Van Rossum
- **Productivity-oriented** language
- Focuses on **Code readability:**
  - Fewer lines of code
  - Enormous library support
  - Many libraries in Python have underlying C++ code
- Performance relevant features:
  - **Dynamic typing**
  - **Memory management**

Static vs Dynamic Typing

Static Typing (C/C++)

- Type known at compile time
- Direct memory access
- No type checks at runtime
- Fast execution

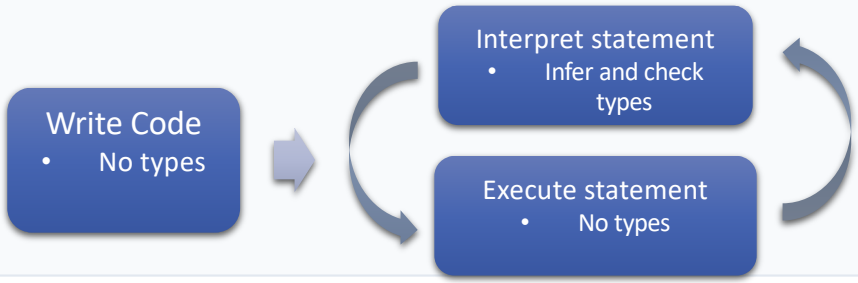
```
int x = 5;  // Type fixed
x = "hello";  // ❌ Error!
```



Dynamic Typing (Python)

- Type determined at runtime
- Everything is a PyObject*
- Type checks on every operation
- Flexibility costs performance
- Duck typing 

```
x = 5       # int
x = 'hello'  # ✅ Fine!
```



Pros & Cons

Static	Dynamic
• The compiler can catch common errors and mistakes early on (+) • You can fix things before deployment (+) • More code to write (-) • Need to know all the types beforehand (-)	• More expressive and flexible (+) • Write code faster (+) • Code more error-prone and brittle (-)

Performance Implications of Dynamic Typing

⚠️ Overhead Sources

- Type lookups on every operation
- Reference counting for memory
- Boxing/unboxing primitives
- Dictionary lookups for attributes

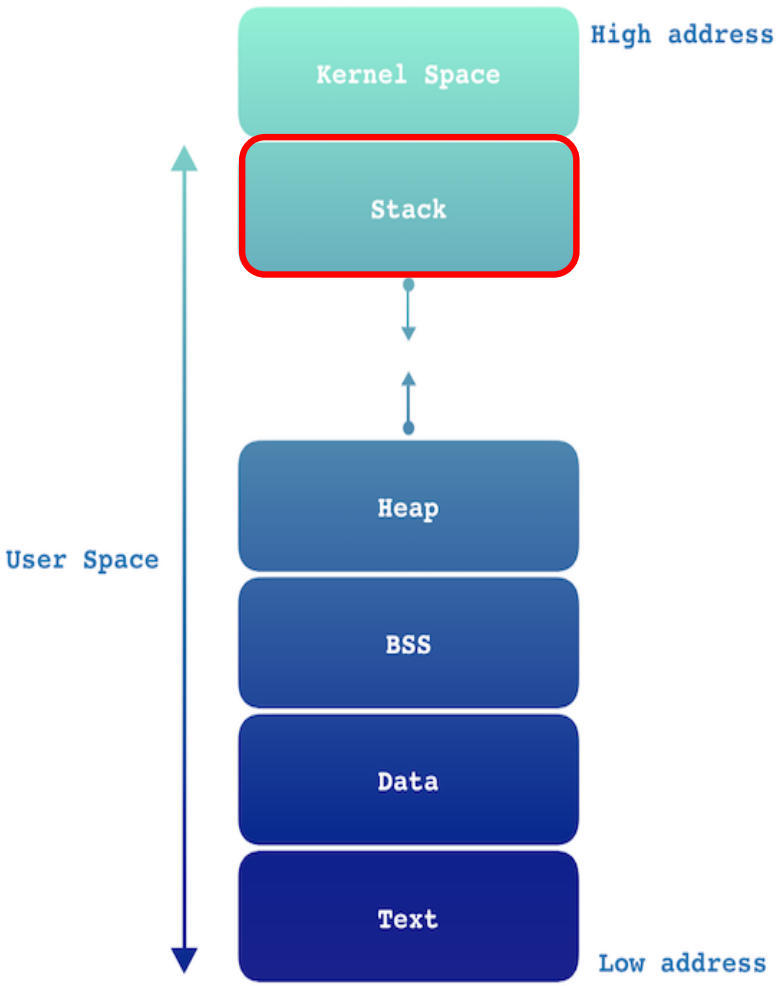
✅ Benefits

- Rapid prototyping & development
- Flexible APIs & duck typing
- Interactive debugging
- Easier to learn & maintain

PyTorch bridges this gap: Python flexibility + compiled C++/CUDA performance

Memory Layout in a Program

- **User Space vs Kernel Space**
  - User code runs in *user space*; the OS runs in *kernel space*
  - Crossing this boundary (syscalls) is expensive and tightly controlled
- **Text / Data / BSS (Low Addresses)**
  - **Text**: compiled program instructions (read-only)
  - **Data / BSS**: global and static variables
  - Mostly fixed at program load time
- **Heap**
  - Dynamically allocated memory (malloc, Python objects, tensors)
  - Flexible but expensive: allocation, indirection, fragmentation
  - **Python heavily relies on the heap**
- **Stack**
  - Function calls, local variables, return addresses
  - Fast allocation/deallocation
  - Limited size, managed automatically per thread
- **Key Performance Takeaway**
  - Stack = fast, predictable
  - Heap = flexible, but higher overhead
  - **Python performance is impacted because most objects live on the heap**



Memory Management: Stack vs Heap

Stack Memory

- Fixed-size, automatically managed
- Function local variables
- Fast allocation/deallocation
- LIFO (Last In, First Out)
- Limited size (~8MB typical)

```
def f():
    x = 5  # Stack
```

Heap Memory

- Dynamic size, manual management
- Objects with longer lifetime
- Slower allocation (malloc/free)
- Can grow as needed
- Requires garbage collection

```
data = [1,2,3]  # Heap
model = Model()  # Heap
```

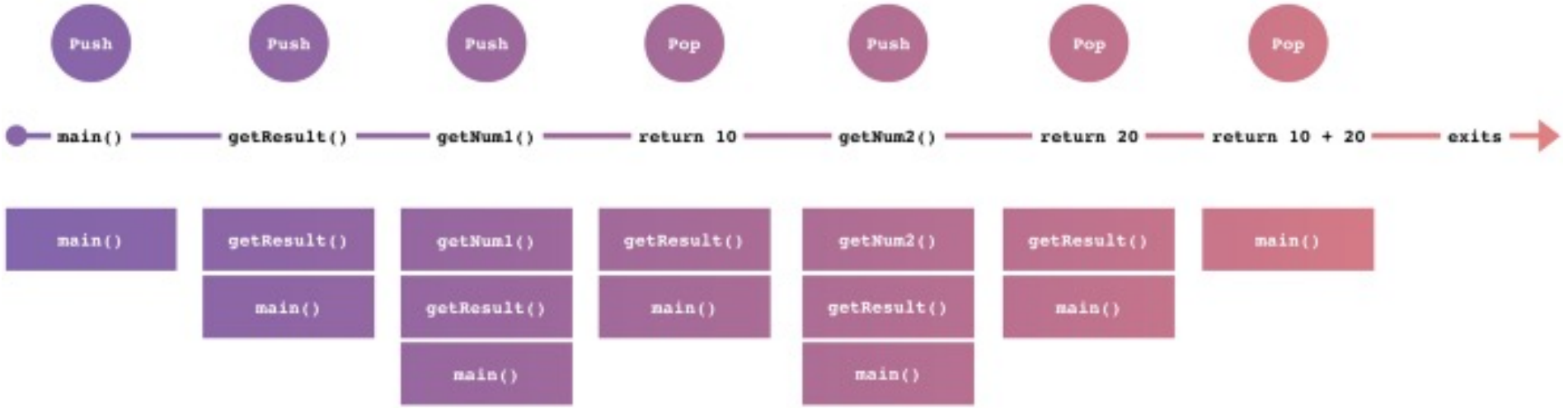
Example

The data pushed for a function call is named a **stack frame**, and it contains the following data.

- The arguments (parameter values) passed to the routine
- The return address back to the routine's caller
- Space for the local variables of the routine

- When the function is called, the stack frame is pushed to the top of stack. Then the process is executed, and the function goes out of scope, the stack frame pops from the top.

```
int main() {
  int result = getResult();
}

int getResult() {
  int num1 = getNum1();
  int num2 = getNum2();
  return num1 + num2;
}

int getNum1() {
  return 10;
}

int getNum2() {
  return 20;
}
```



Python's Automatic Memory Management

Reference Counting

Every object tracks how many references point to it. When refcount hits 0, memory is freed immediately.

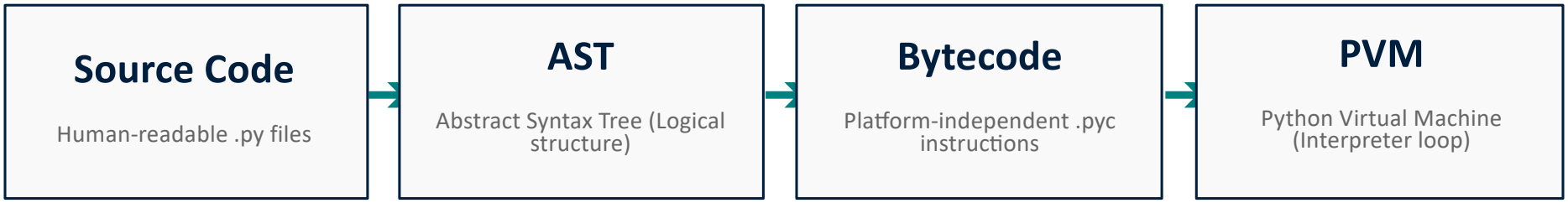
Garbage Collection

Breaks circular references that refcounting can't handle. Runs periodically to reclaim memory.

Performance Impact

Overhead on every operation (refcount updates). GC pauses can affect latency. Trade-off: productivity vs raw speed.

Python Execution: From Source to Bytecode



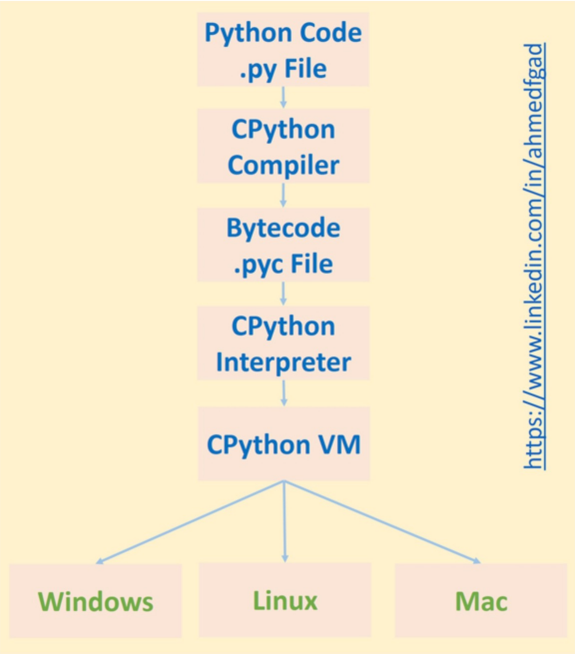
The Compilation Phase

Python code is first parsed into a **Concrete Syntax Tree (CST)**, then lowered into an **AST**, and finally compiled into **Bytecode**. This happens once per execution or when the source changes.

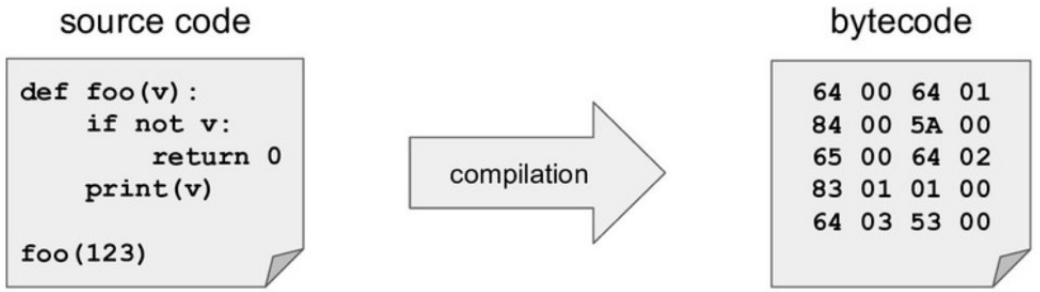
The Interpretation Phase

The **PVM** is a stack-based machine that executes bytecode instructions one by one. This "eval loop" is where the majority of Python's runtime overhead occurs.

CPython Flow



CPython source code to bytecode

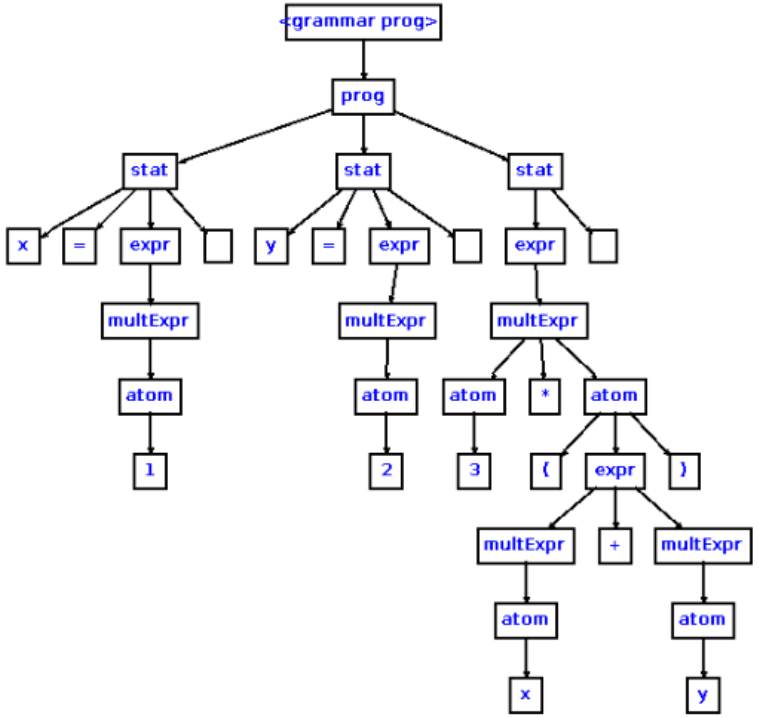


Source → Parse Tree → AST → CFG → Bytecode → Optimizations.

1. **Lexing & Parsing:** Source code (**.py** file) is tokenized and parsed into a **parse tree**.
2. **AST Generation:** The parse tree is converted into an **Abstract Syntax Tree (AST)**, representing program structure.
3. **Semantic Analysis / Symbol Table:** Names are bound to scopes (local, global, nonlocal, free variables).
4. **Control Flow Graph (CFG):** The AST is lowered into basic blocks / control flow, preparing for bytecode emission.
5. **Bytecode Emission:** Bytecode instructions (e.g. **LOAD_CONST**, **CALL_FUNCTION**) are generated and packaged into a **PyCodeObject**.
6. **Peephole Optimization:** A local optimization pass simplifies bytecode (e.g. constant folding, jump chaining).
7. **Interpreter Execution:** The **Python Virtual Machine** (**ceval.c**) executes the bytecode instruction by instruction.

Example: CST

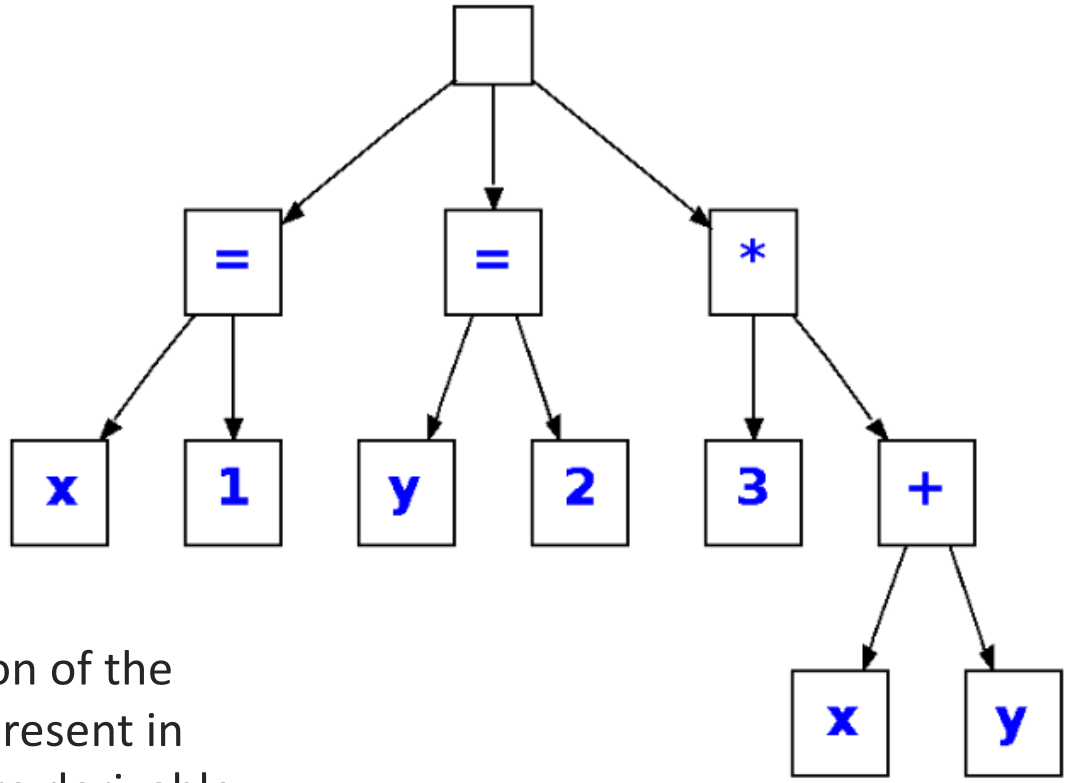
- x=1
- y=2
- 3*(x+y)



The parse tree is a concrete representation of the input. The parse tree retains all the information of the input. The empty boxes represent whitespace, i.e., end of line.

Example: AST

- x=1
- y=2
- 3*(x+y)



The AST is an abstract representation of the input. Notice that parents are not present in the AST because the associations are derivable from the tree structure.

Peephole Optimizations

Definition:

- Small, local optimizations applied to Python bytecode after compilation, scanning short instruction sequences (the "peephole") and replacing them with simpler, faster forms.

Goal:

- Improve efficiency without major overhead — lightweight optimization.

Examples:

- **Constant Folding / Expressions:** 2 + 2 → 4, 4 * 25 → 100.
- **Dead Code Removal:** Remove unused imports or unreachable code.
- **Short Sequence Optimization:** Combine multiple LOAD_CONST into a single BUILD_TUPLE.
- **Variable Lookup Optimization:** Replace slower global lookups with faster local ones where possible.

⚡ *Note:* These optimizations are simple and local — Python does **not** perform aggressive global optimizations like a JIT compiler would

Python Bytecode

- Bytecode looks like a simplified assembly code for a **stack machine**
- Caches in .pyc file for faster execution the second time the same file is executed
- Run on a VM that executes the machine code corresponding to each byte code
- Core of CPython is an eval loop that implements a simple stack-based virtual machine.
- The bytecode generated for any user function (or code in general) can be inspected using the dis module in human readable form

```
>>> import torch
>>> import dis
>>> def f():
...     x = torch.randn(2,2)
...     return x.mm(x)
...
>>> dis.dis(f)
  2           0 LOAD_GLOBAL              0 (torch)
              2 LOAD_ATTR                1 (randn)
              4 LOAD_CONST               1 (2)
              6 LOAD_CONST               1 (2)
              8 CALL_FUNCTION            2
             10 STORE_FAST               0 (x)

  3          12 LOAD_FAST                0 (x)
             14 LOAD_ATTR                2 (mm)
             16 LOAD_FAST                0 (x)
             18 CALL_FUNCTION            1
             20 RETURN_VALUE
>>>
```

dis: bytecode disassembler

https://docs.python.org/library/dis.html

```
>>> def hello():
...     return "Kaixo!"
...
>>> import dis
>>> dis.dis(hello)
  2           0 LOAD_CONST               1 ('Kaixo!')
              3 RETURN_VALUE
```

```
1  >>> from dis import dis
2  >>> def square(x):
3  ...     return x*x
4  ...
5
6  >>> dis(square)
7  2           0 LOAD_FAST                0 (x)
8              2 LOAD_FAST                0 (x)
9              4 BINARY_MULTIPLY
10             6 RETURN_VALUE
```

- The ./Include/opcodes.h file contains a listing of the Python Virtual Machine's bytecode instructions

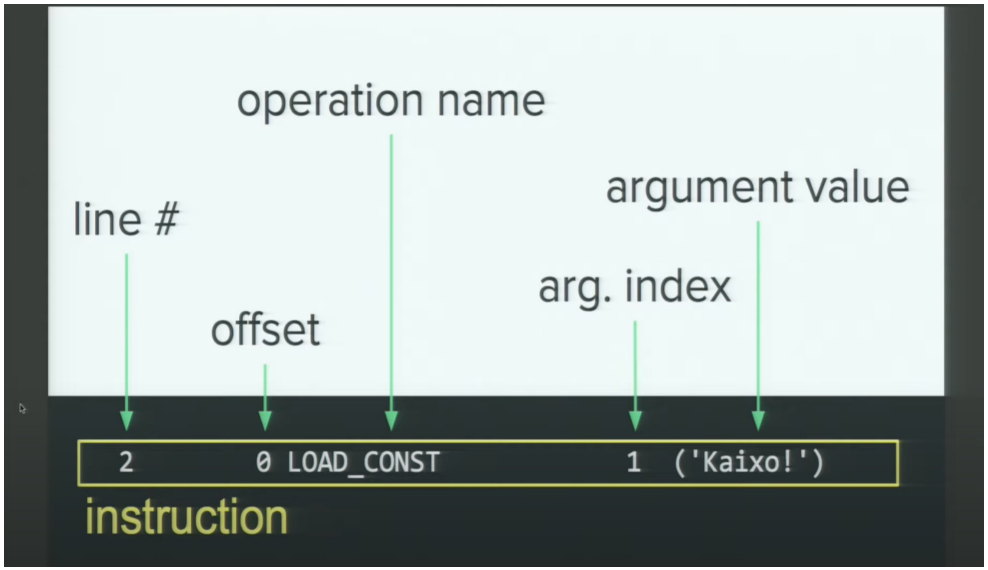
Bytecode Instructions

sample operations

https://docs.python.org/library/dis.html#python-bytecode-instructions

`LOAD_CONST(c)`	pushes *c* onto top of stack (TOS)
`BINARY_ADD`	pops & adds top 2 items, result becomes TOS
`CALL_FUNCTION(a)`	calls function with arguments from stack *a* indicates # of positional & keyword args

```
>>> dis.opmap['BINARY_ADD']    # => 23
>>> dis.opname[23]             # => 'BINARY_ADD'
```



https://www.youtube.com/watch?v=GNPKBICTF2w

Python interpreter

- Always running in a basic main thread
- Can do context-switching among its threads
- Based on a **Stack Machine with push and pop**
- Interpreter loop:
  1. Read next instruction in bytecode
  2. Evaluate the 16 bits bytecode: ***oparg*** and ***opcode***
  3. Switch/case: Call the corresponding C function (macro) that executes the instruction

CPython interpreter - Stack Machine Example

- Evaluate expression:

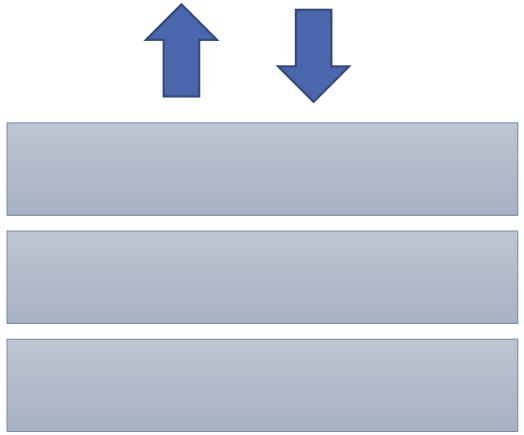
```
d = a + b * c
```

- Values array: [a, b, c, d]
- Compiled Bytecode:

```
LOAD_FAST 0
LOAD_FAST 1
LOAD_FAST 2
BINARY_MULTIPLY
BINARY_ADD
STORE_FAST 3
```

Index in the values array

- Stack state:



CPython interpreter - Stack Machine Example

- Evaluate expression:

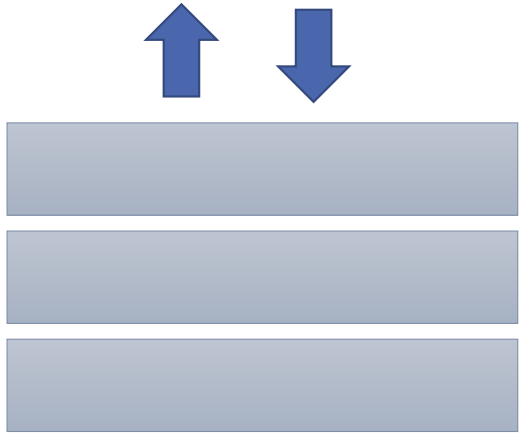
```
d = a + b * c
```

- Values array: [a, b, c, d]
- Compiled Bytecode:

```
LOAD_FAST 0
LOAD_FAST 1
LOAD_FAST 2
BINARY_MULTIPLY
BINARY_ADD
STORE_FAST 3
```

Index in the values array

- Stack state:



CPython interpreter - Stack Machine Example

- Evaluate expression:

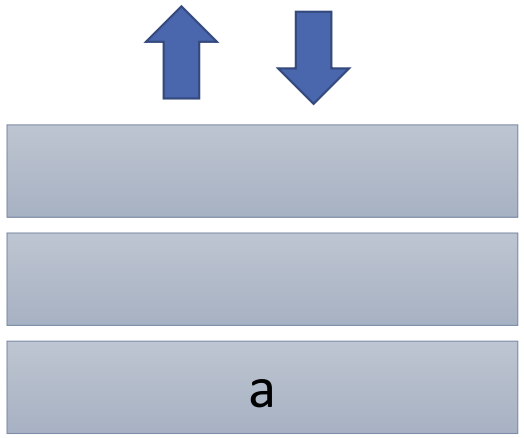
```
d = a + b * c
```

- Values array: [a, b, c, d]
- Compiled Bytecode:

IP

```
LOAD_FAST 0
LOAD_FAST 1
LOAD_FAST 2
BINARY_MULTIPLY
BINARY_ADD
STORE_FAST 3
```

- Stack state:



CPython interpreter - Stack Machine Example

- Evaluate expression:

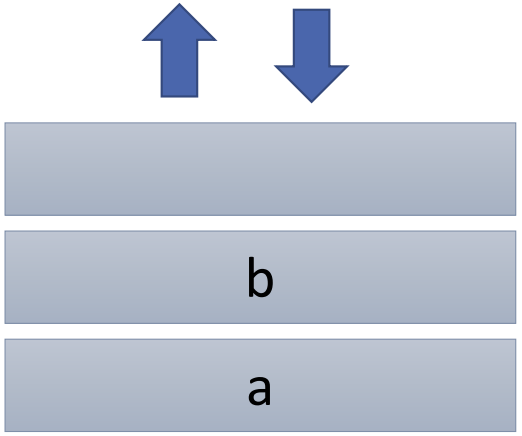
```
d = a + b * c
```

- Values array : [a, b, c, d]
- Compiled Bytecode:

IP

```
LOAD_FAST 0
LOAD_FAST 1
LOAD_FAST 2
BINARY_MULTIPLY
BINARY_ADD
STORE_FAST 3
```

- Stack state:



CPython interpreter - Stack Machine Example

- Evaluate expression:

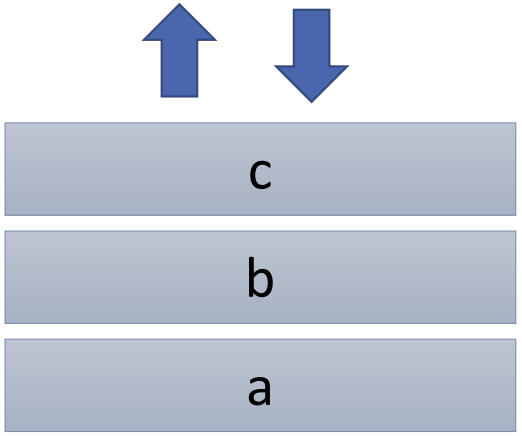
```
d = a + b * c
```

- Values array: [a, b, c, d]
- Compiled Bytecode:

IP

```
LOAD_FAST 0
LOAD_FAST 1
LOAD_FAST 2
BINARY_MULTIPLY
BINARY_ADD
STORE_FAST 3
```

- Stack state:



CPython interpreter - Stack Machine Example

- Evaluate expression:

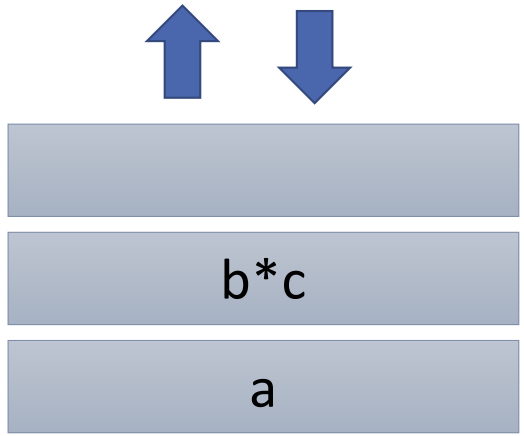
```
d = a + b * c
```

- Values array: [a, b, c, d]
- Compiled Bytecode:

```
LOAD_FAST 0
LOAD_FAST 1
LOAD_FAST 2
BINARY_MULTIPLY
BINARY_ADD
STORE_FAST 3
```

IP

- Stack state:



CPython interpreter - Stack Machine Example

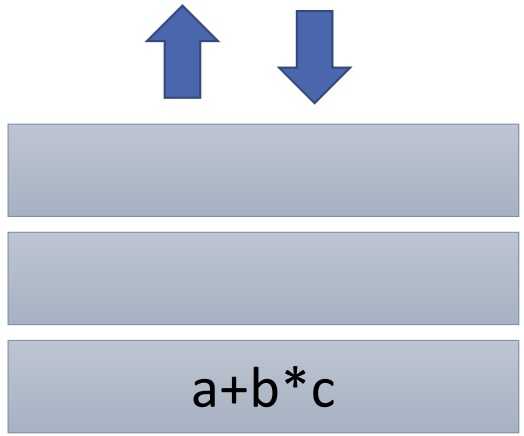
- Evaluate expression:

```
d = a + b * c
```

- Values array: [a, b, c, d]
- Compiled Bytecode:

```
IP  LOAD_FAST 0
    LOAD_FAST 1
    LOAD_FAST 2
    BINARY_MULTIPLY
    BINARY_ADD
    STORE_FAST 3
```

- Stack state:



CPython interpreter - Stack Machine Example

- Evaluate expression:

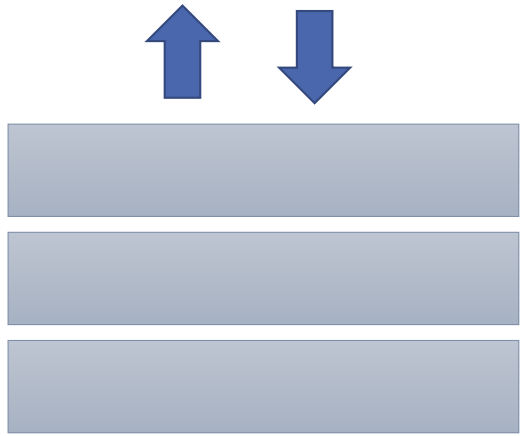
```
d = a + b * c
```

- Values array: [a, b, c, **d**]
- Compiled Bytecode:

```
LOAD_FAST 0
LOAD_FAST 1
LOAD_FAST 2
BINARY_MULTIPLY
BINARY_ADD
STORE_FAST 3
```

IP

- Stack state:



Key Insight: Python Operation Expansion

Every Python Operation = Many Interpreter Steps

A simple x + y in Python expands to:

1. LOAD_FAST to push x onto stack
2. LOAD_FAST to push y onto stack
3. Type checking (are both numbers?)
4. Dispatch to appropriate __add__ method
5. BINARY_ADD to perform addition
6. Reference count updates
7. Boxing result as Python object

Solution: Push compute to compiled kernels

The Performance Challenge of Python

Productivity vs. Performance

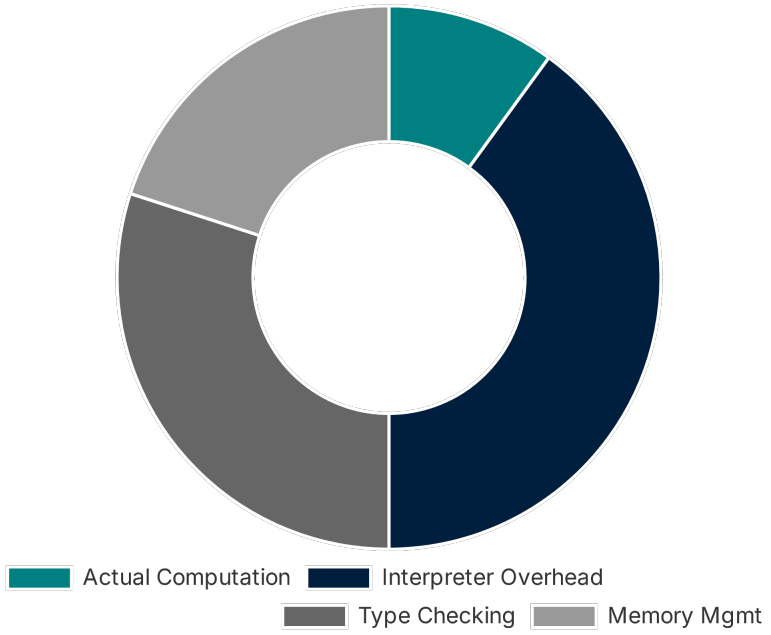
Python prioritizes developer speed and readability, but this abstraction comes at a significant execution cost.

Dynamic Typing Overhead

The interpreter must check types at runtime for every operation, preventing many compile-time optimizations.

Memory Management

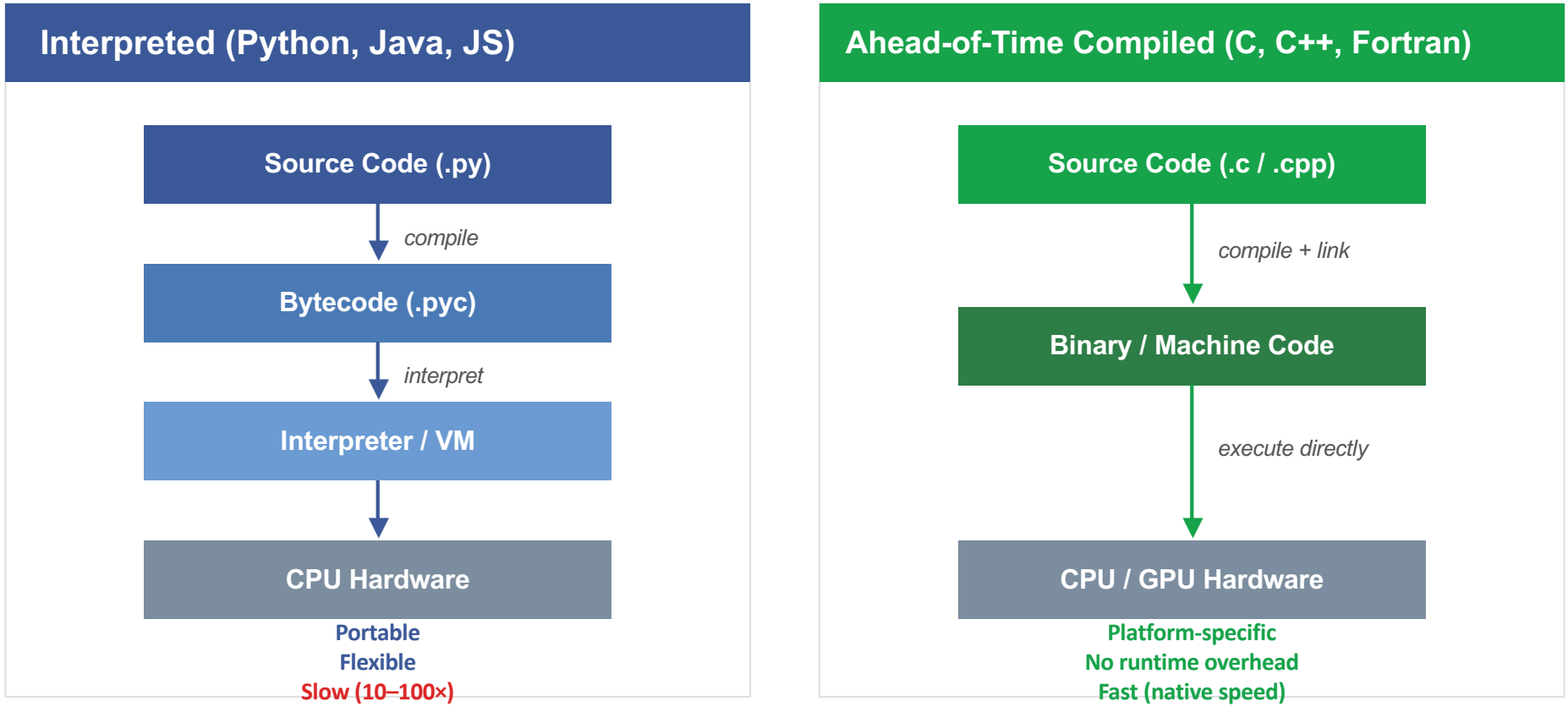
Automatic garbage collection and object overhead (everything is an object) lead to cache inefficiency.



Conceptual: Execution Time Breakdown

From Language to Binary Execution

Two paths from source code to hardware



For ML, we want Python's ease + C/C++'s speed. How do we bridge them?

The Performance Gap

Why Python alone can't do AI at scale?

Python's Overhead

- Every value is a heap-allocated PyObject (28 bytes per float)
- Dynamic type checks on every operation
- Reference counting on every assignment
- GIL prevents true multi-threading
- Interpreter dispatch per bytecode instruction

What AI Workloads Need

- Contiguous memory (cache-friendly, SIMD-ready)
- No per-element type checking
- Direct hardware access (GPU, SIMD, TPU)
- Parallel execution across cores
- One instruction → millions of FLOPs

Concrete example: 1000×1000 matrix multiply

Pure Python nested loops: ~45 seconds

C with BLAS (cblas_dgemm): ~0.05 seconds

That's ~900× faster. We need C's speed with Python's API.

NumPy: What's Under the Hood?

Python API Layer (~35% of source)

- numpy.array(), np.dot(), np.linalg
- High-level array creation & manipulation
- Broadcasting rules & fancy indexing
- Python-friendly API over C internals
- Type dispatching to compiled routines

```
import numpy as np
A = np.random.rand(1000,1000)
result = np.dot(A, A)  # Python
```

calls into

C Core + Fortran BLAS/LAPACK (~65%)

- $O(10^5)$ lines of C (ndarray core, ufuncs)
- PyArray_* C API / CPython bridge
- BLAS/LAPACK — external Fortran libraries
- Optimized sorting, searching, indexing
- SIMD vectorization for element-wise ops

```
// Actual execution path:
cblas_dgemm(...)   // BLAS routine
// 100-1000x faster than Python
```

"NumPy looks like Python, but almost everything performance-critical runs in optimized C and Fortran."

TensorFlow: Architecture Deep Dive

From Python ease-of-use to hardware-level acceleration — only a small fraction of TensorFlow's runtime is Python

tf.keras / Python API `Python`	User-facing: model.fit(), layers, optimizers. Pure Python wrappers over C++ ops.
TF Python Runtime `Python + C++`	Eager execution, tf.function tracing, GradientTape. Python orchestrates C++ engine.
TF Core (C++ Runtime) `C++`	Graph optimization, op scheduling, internal session mgmt (largely hidden in TF 2.x).
XLA Compiler `C++ (HLO IR)`	Graph-level optimization via jit_compile=True. Required for TPUs, increasingly used on GPU/CPU.
Kernels & Hardware `C++ / CUDA / Drivers`	MatMul, Conv2D kernels (C++/CUDA). GPU via cuDNN/cuBLAS, TPU via XLA, CPU via Eigen/MKL.

PyTorch: Python API vs C++/CUDA Core

Python Frontend (~30% of codebase)

- User-facing API: torch.tensor, nn.Module
- Autograd engine interface (Python side)
- Model definition & training loops
- Dynamic computation graph construction
- Data loading & preprocessing (DataLoader)

```
model = nn.Linear(784, 10)
output = model(input)  # Python
loss.backward()        # Python
```

calls into

C++/CUDA Backend (~70% of codebase)

- ATen: core tensor library (C++)
- Autograd engine (C++ for speed)
- cuDNN / cuBLAS for GPU kernels
- torch.compile / TorchInductor (C++)
- Memory allocator (caching, CUDA)

```
// ATen C++ (actual execution)
at::Tensor mm(Tensor self, Tensor mat2)
cublasSgemm(...)  // GPU kernel
```

How does Python Call into C/C+?

CPython Extension Modules

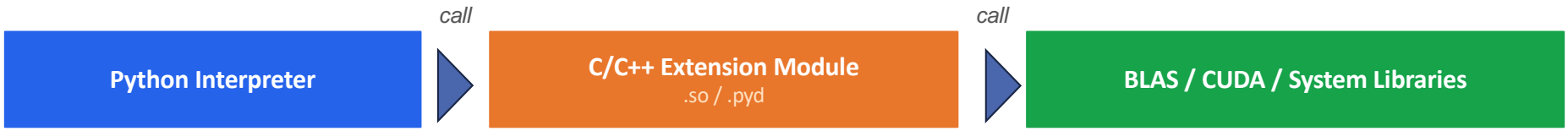
The classic mechanism for calling C/C++ from Python — but one of several approaches used in modern ML

How They Work

- C/C++ code compiled into shared libraries
- (.so on Linux, .pyd on Windows)
- Linked directly to the CPython interpreter
- Can manipulate Python objects at the C level
- Uses the PyObject / PyArray_* C API
- Authoring tools: raw C API, Cython, pybind11

Why Use Them

- Bypass Python's interpreter overhead entirely
- Store data in C/C++ structures (not Python objects)
- Call compiled functions and system-level APIs
- NumPy's ndarray is the classic example:
- Python object wrapping a C struct with pointers
- to contiguous memory, dtype, shape, strides



Example: NumPy uses the CPython C API directly. PyTorch and TensorFlow use pybind11 to generate bindings.

Beyond Extension Modules

CPython extensions are one pattern — modern ML frameworks use several mechanisms to cross the Python→C++ boundary

Mechanism	How It Works	Used By
CPython C API	C code links to libpython, manipulates PyObject* directly. Compiled to .so/.pyd.	NumPy (core), SciPy, older extensions
pybind11	Header-only C++ library that auto-generates Python bindings. Modern replacement for raw C API and SWIG.	PyTorch (ATen), TensorFlow 2.x
Graph Serialization	Python builds a computation graph (data structure), hands it to a C++ runtime for execution. No direct function call.	TF tf.function, ONNX Runtime
Bytecode Capture	Intercepts Python bytecode at runtime, extracts compute graph without tracing. Preserves Python semantics.	PyTorch TorchDynamo (torch.compile)
Dynamic Codegen	Generates and compiles C++/CUDA/Triton kernels at runtime from the captured graph. Loaded via dlopen.	TorchInductor, XLA, TVM

Key insight: modern ML frameworks often combine multiple mechanisms — e.g., PyTorch uses pybind11 for ATen, bytecode capture via TorchDynamo, and dynamic codegen via TorchInductor, all in a single torch.compile() call.

How Does Python Call into C/C++?

Three approaches, from loosely coupled to tightly integrated

1. Subprocess

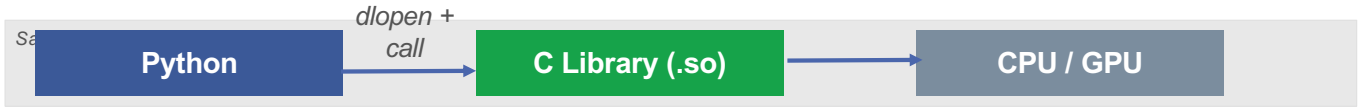
Separate process

✗ Slow (serialization + IPC overhead)
✗ No shared memory between Python and C
✓ Crash isolation (C crash doesn't kill Python)

2. ctypes / FFI

(Foreign Function Interface. Ctype is a python built-in FFI)

Direct function calls



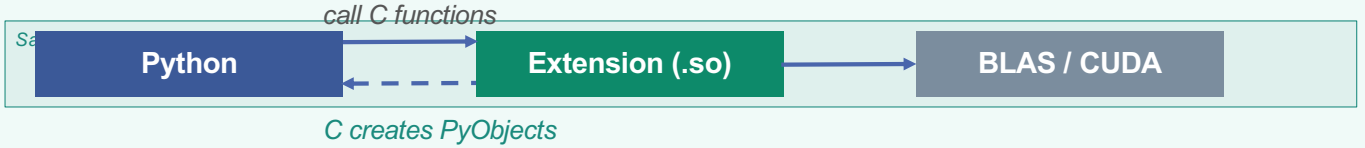
✓ No serialization (in-process calls)
✗ Marshaling overhead per call (type conversion)
✗ Can't create new Python types backed by C data

Marshaling:

- Converting Python objects into bytes (pickle, JSON, protobuf, etc.)
- Copying data across process boundaries
- Reconstructing C data structures on the other side

3. Extension Module

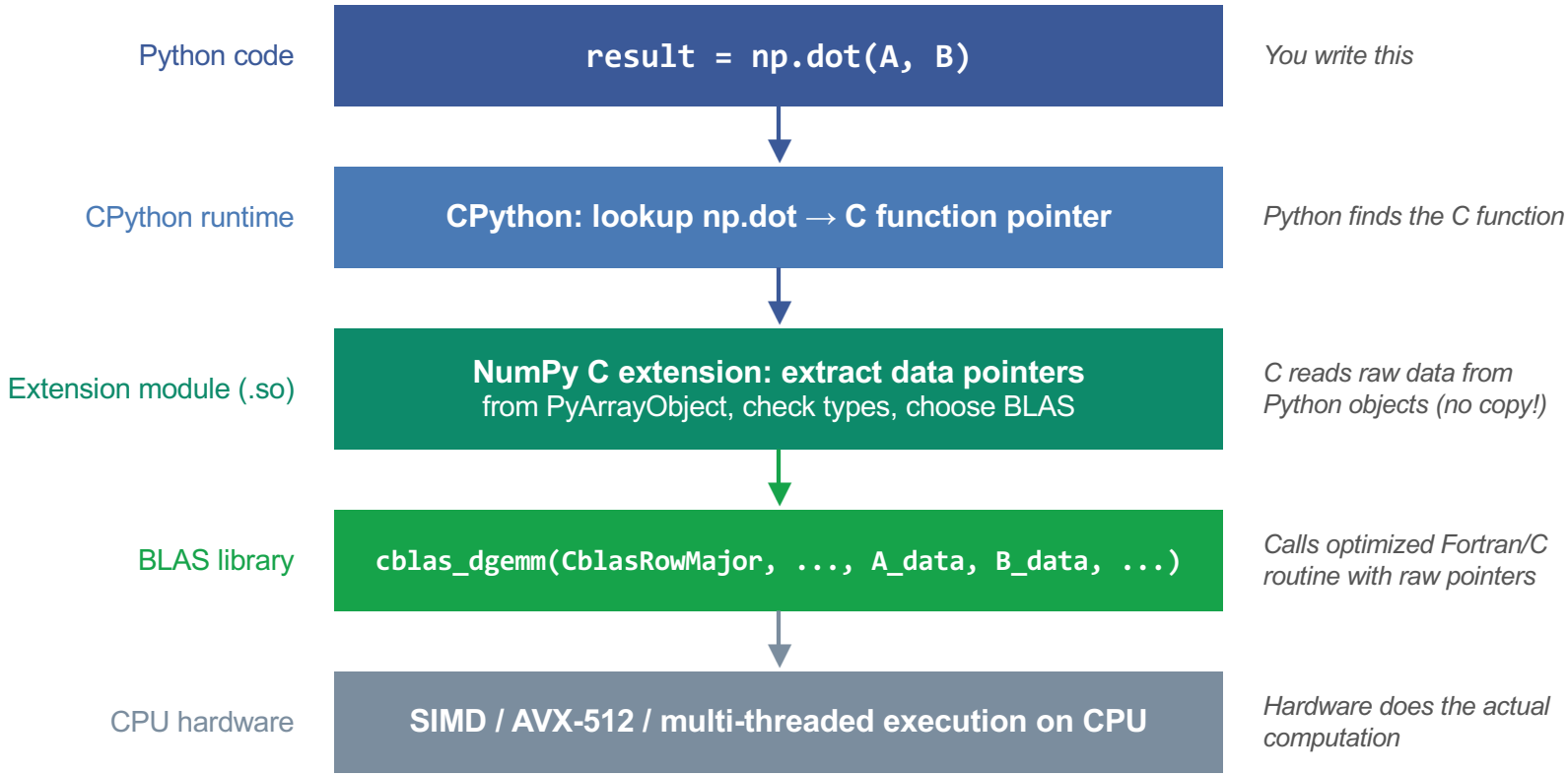
Deep integration



✓ No serialization, no marshaling
✓ C code can create Python objects with C data inside (e.g. ndarray)
✓ Two-way: Python calls C, C manipulates Python objects

Inside an Extension Module Call

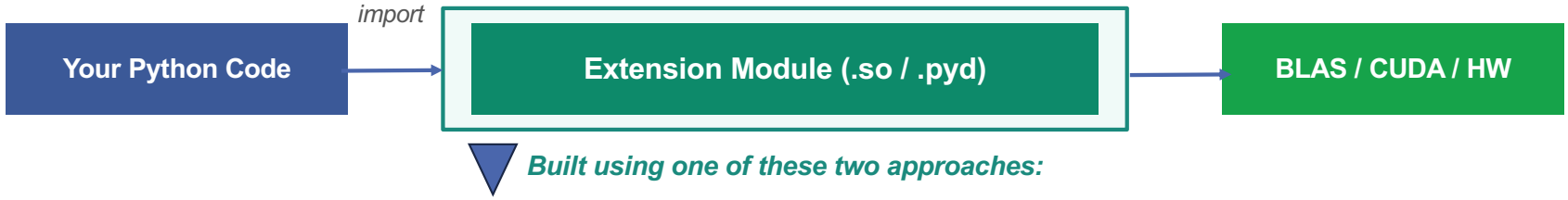
What happens when you call np.dot(A, B)



Key: the extension module is the bridge — it unwraps Python objects to get C data, then calls hardware-optimized libraries with raw pointers. No copies, no serialization.

Writing Extension Modules: Two Approaches

Both produce the same thing — a .so/.pyd file — but the developer experience is very different



CPython C API (since 1991)

- Write raw C code against Python's C API
- Manually manage reference counting (Py_INCREF/DECREF)
- Define module init function (PyInit_modulename)
- Verbose: ~50–100 lines of boilerplate per function
- Used by: NumPy, SciPy, CPython stdlib

```c
// C API: wrapping a simple add function
static PyObject* my_add(PyObject* self,
                        PyObject* args) {
  int a, b;
  PyArg_ParseTuple(args, "ii", &a, &b);
  return PyLong_FromLong(a + b);
}
```

evolution

pybind11 (since 2015)

- Header-only C++ library — no dependencies
- Auto-generates all CPython boilerplate
- Automatic reference counting & type conversion
- C++-native: templates, classes, STL containers
- Used by: PyTorch (ATen), TensorFlow 2.x

```cpp
// pybind11: same function, much less code
#include <pybind11/pybind11.h>
int add(int a, int b) { return a + b; }

PYBIND11_MODULE(example, m) {
  m.def("add", &add);
}
```

What is pybind11?

A lightweight header-only C++ library that exposes C++ code to Python — and vice versa

What It Does

- Takes your C++ functions and classes
- Auto-generates CPython wrapper code
- Compiles to a .so that Python can import
- Handles ref counting, type conversion, exceptions automatically

Key Features

- Header-only: just #include, no linking
- Supports: functions, classes, lambdas, STL, numpy arrays, enums
- Automatic Python↔C++ type conversion
- Replaced SWIG in most modern projects

Why ML Frameworks Use It

- C++ is the language of AI runtimes (ATen, TF Core, XLA)
- Need to expose 1000s of C++ ops to Python
- Raw C API would be 100K+ lines of boilerplate
- pybind11: one macro per function/class

Real-world pattern: exposing a C++ tensor operation to Python

C++ side:

```
// my_ops.cpp
#include <pybind11/pybind11.h>
#include <torch/extension.h>

torch::Tensor my_relu(torch::Tensor x) {
  return torch::clamp_min(x, 0);
}
PYBIND11_MODULE(my_ops, m) {
  m.def("my_relu", &my_relu);
}
```

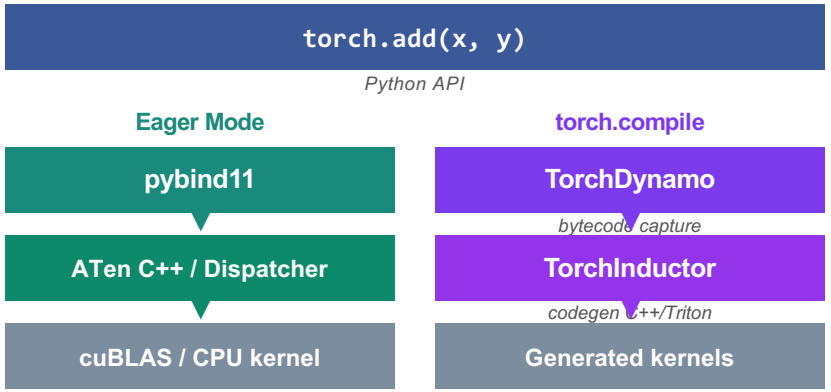
Python side:

```
# Use it like any Python function
import torch
from torch.utils.cpp_extension import load

my_ops = load(name="my_ops",
              sources=["my_ops.cpp"])

x = torch.randn(1000)
y = my_ops.my_relu(x)  # calls C++
```

pybind11 as a Python–C++ Binding Layer

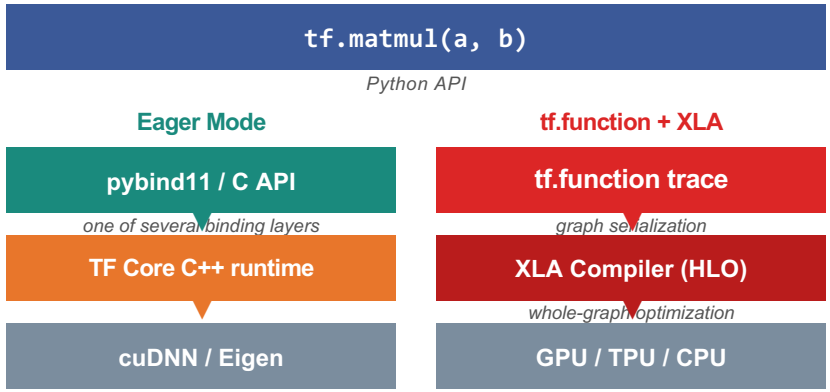
pybind11 handles API bindings — but performance-critical execution goes through compilers, not bindings

PyTorch (Python-first)



- pybind11 is central: every eager op goes through it
- torch.compile bypasses Python dispatch entirely
- ATen has ~2,000 ops, all auto-generated bindings
- Compiler path: no Python on the hot path

TensorFlow (Graph-first)



- pybind11 is one of several binding layers (not central)
- Graph mode bypasses pybind11 entirely
- XLA is the dominant path for optimized execution
- TPUs require XLA; GPUs increasingly use it

Both use pybind11 for Python bindings — but real performance comes from their compilers (TorchInductor / XLA), not the bindings.

Example: CPython Extensions

Step 1: Write the C Code (mymodule.c)

```c

#define PY_SSIZE_T_CLEAN
#include <Python.h>

/* Function to add two numbers */
static PyObject* mymodule_add(PyObject* self, PyObject* args) {
    double a, b;
    if (!PyArg_ParseTuple(args, "dd", &a, &b)) {
        return NULL;  // Error handling
    }
    return Py_BuildValue("d", a + b);
}

/* Method definitions */
static PyMethodDef MyMethods[] = {
    {"add", mymodule_add, METH_VARARGS, "Add two numbers"},
    {NULL, NULL, 0, NULL}  // Sentinel to indicate the end
};

/* Module definition */
static struct PyModuleDef mymodule = {
    PyModuleDef_HEAD_INIT,
    "mymodule",  // Module name
    "A simple C extension module",  // Module docstring
    -1,
    MyMethods
};

/* Module initialization function */
PyMODINIT_FUNC PyInit_mymodule(void) {
    return PyModule_Create(&mymodule);
}

```

Step 2: Compile the C Extension

To compile the module, create a **setup.py** file.

```python

from setuptools import setup, Extension

setup(
    name="mymodule",
    version="1.0",
    ext_modules=[Extension("mymodule", ["mymodule.c"])],
)

```

Step 3: Build and Install the Module

```
(base) kaoutar-elmaghraoui@kaoutars-mbp-4 performance % python setup.py build
running build
running build_ext
building 'mymodule' extension
clang -fno-strict-overflow -Wsign-compare -Wunreachable-code -DNDEBUG -O2 -Wall -fPIC -O2 -isystem /opt/anaconda3/include -arch arm64 -fPI
naconda3/include -arch arm64 -I/opt/anaconda3/include/python3.12 -c mymodule.c -o build/temp.macosx-11.1-arm64-cpython-312/mymodule.o
creating build/lib.macosx-11.1-arm64-cpython-312
clang -bundle -undefined dynamic_lookup -Wl,-rpath,/opt/anaconda3/lib -L/opt/anaconda3/lib -Wl,-rpath,/opt/anaconda3/lib -L/opt/anaconda3/
-11.1-arm64-cpython-312/mymodule.o -o build/lib.macosx-11.1-arm64-cpython-312/mymodule.cpython-312-darwin.so
ld: warning: duplicate -rpath '/opt/anaconda3/lib' ignored
(base) kaoutar-elmaghraoui@kaoutars-mbp-4 performance % python setup.py install
running install
```

Step 4: Use the C Extension in Python

Once installed, you can import the module and use it like a normal Python function:

```python

import mymodule

result = mymodule.add(3.5, 2.5)
print(f"Result: {result}")  # Output: 6.0

```

Example: Cpython Extensions

Step 1: Write the C Code (mymodule.c)

```c

#define PY_SSIZE_T_CLEAN
#include <Python.h>

/* Function to add two numbers */
static PyObject* mymodule_add(PyObject* self, PyObject* args) {
    double a, b;
    if (!PyArg_ParseTuple(args, "dd", &a, &b)) {
        return NULL;  // Error handling
    }
    return Py_BuildValue("d", a + b);
}

/* Method definitions */
static PyMethodDef MyMethods[] = {
    {"add", mymodule_add, METH_VARARGS, "Add two numbers"},
    {NULL, NULL, 0, NULL}  // Sentinel to indicate the end
};

/* Module definition */
static struct PyModuleDef mymodule = {
    PyModuleDef_HEAD_INIT,
    "mymodule",  // Module name
    "A simple C extension module",  // Module docstring
    -1,
    MyMethods
};

/* Module initialization function */
PyMODINIT_FUNC PyInit_mymodule(void) {
    return PyModule_Create(&mymodule);
}

```

This function is **exposed to Python** as mymodule.add().

Example: Cpython Extensions

Step 1: Write the C Code (mymodule.c)

```c

#define PY_SSIZE_T_CLEAN
#include <Python.h>

/* Function to add two numbers */
static PyObject* mymodule_add(PyObject* self, PyObject* args) {
    double a, b;
    if (!PyArg_ParseTuple(args, "dd", &a, &b)) {
        return NULL;  // Error handling
    }
    return Py_BuildValue("d", a + b);
}

/* Method definitions */
static PyMethodDef MyMethods[] = {
    {"add", mymodule_add, METH_VARARGS, "Add two numbers"},
    {NULL, NULL, 0, NULL}  // Sentinel to indicate the end
};

/* Module definition */
static struct PyModuleDef mymodule = {
    PyModuleDef_HEAD_INIT,
    "mymodule",  // Module name
    "A simple C extension module",  // Module docstring
    -1,
    MyMethods
};

/* Module initialization function */
PyMODINIT_FUNC PyInit_mymodule(void) {
    return PyModule_Create(&mymodule);
}

```

Registers functions that can be called from Python.

Field	Description
`"add"`	The name of the function in Python (`mymodule.add()`).
`mymodule_add`	The C function implementing `add()` .
`METH_VARARGS`	Indicates it takes a **tuple** (`args`) from Python.
`"Add two numbers"`	A docstring (shows up in `help(mymodule)`).

Example: Cpython Extensions

Step 1: Write the C Code (mymodule.c)

```c

#define PY_SSIZE_T_CLEAN
#include <Python.h>

/* Function to add two numbers */
static PyObject* mymodule_add(PyObject* self, PyObject* args) {
    double a, b;
    if (!PyArg_ParseTuple(args, "dd", &a, &b)) {
        return NULL;  // Error handling
    }
    return Py_BuildValue("d", a + b);
}

/* Method definitions */
static PyMethodDef MyMethods[] = {
    {"add", mymodule_add, METH_VARARGS, "Add two numbers"},
    {NULL, NULL, 0, NULL}  // Sentinel to indicate the end
};

/* Module definition */
static struct PyModuleDef mymodule = {
    PyModuleDef_HEAD_INIT,
    "mymodule",  // Module name
    "A simple C extension module",  // Module docstring
    -1,
    MyMethods
};

/* Module initialization function */
PyMODINIT_FUNC PyInit_mymodule(void) {
    return PyModule_Create(&mymodule);
}

```

Defines the module structure.

Field	Purpose
`PyModuleDef_HEAD_INIT`	Macro for initializing the module.
`"mymodule"`	The module name (Python `import mymodule`).
`"A simple C extension module"`	Module docstring.
`-1`	Indicates no per-interpreter state.
`MyMethods`	The method table (`add()` function is included here).

Example: Cpython Extensions

Step 1: Write the C Code (mymodule.c)

```c

#define PY_SSIZE_T_CLEAN
#include <Python.h>

/* Function to add two numbers */
static PyObject* mymodule_add(PyObject* self, PyObject* args) {
    double a, b;
    if (!PyArg_ParseTuple(args, "dd", &a, &b)) {
        return NULL;  // Error handling
    }
    return Py_BuildValue("d", a + b);
}

/* Method definitions */
static PyMethodDef MyMethods[] = {
    {"add", mymodule_add, METH_VARARGS, "Add two numbers"},
    {NULL, NULL, 0, NULL}  // Sentinel to indicate the end
};

/* Module definition */
static struct PyModuleDef mymodule = {
    PyModuleDef_HEAD_INIT,
    "mymodule",  // Module name
    "A simple C extension module",  // Module docstring
    -1,
    MyMethods
};

/* Module initialization function */
PyMODINIT_FUNC PyInit_mymodule(void) {
    return PyModule_Create(&mymodule);
}

```

- This function **creates the module** and returns it to Python when import mymodule is executed.
  1. Calls PyModule_Create(&mymodule), which **registers the module**.
  2. Returns a pointer to the module.

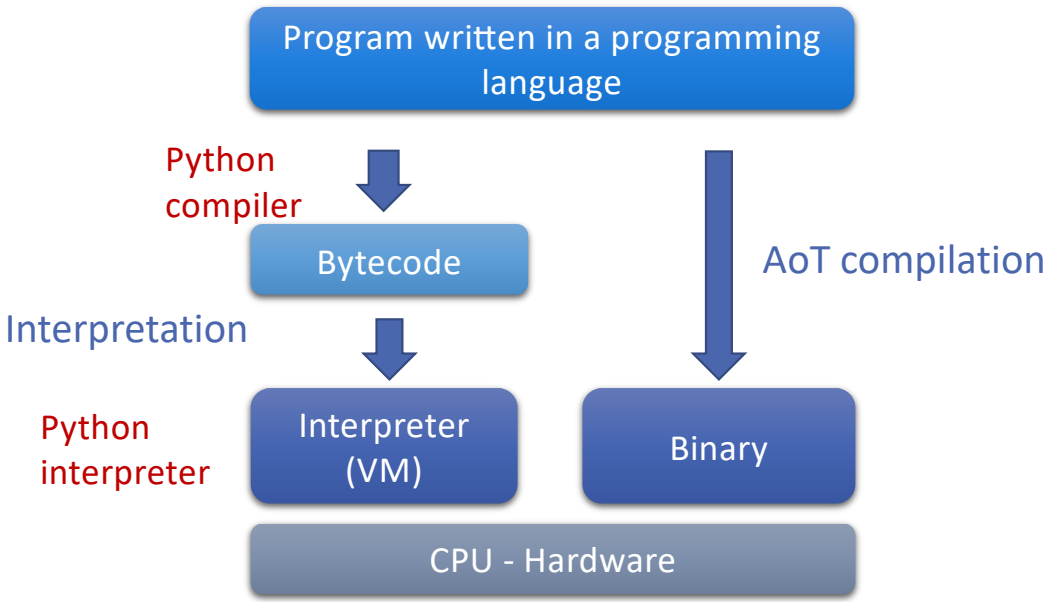
Python Performance – Just In Time Compilation

Some Definitions

- Assembly code is processor architecture specific
  - Assembly language is a low-level programming language that must be converted into machine code using software called an assembler
- Machine code is either a set of instructions in machine language or binary format which can be directly executed by the CPU
- JIT compilers aim to shorten the path from Python code to machine code at runtime, improving speed

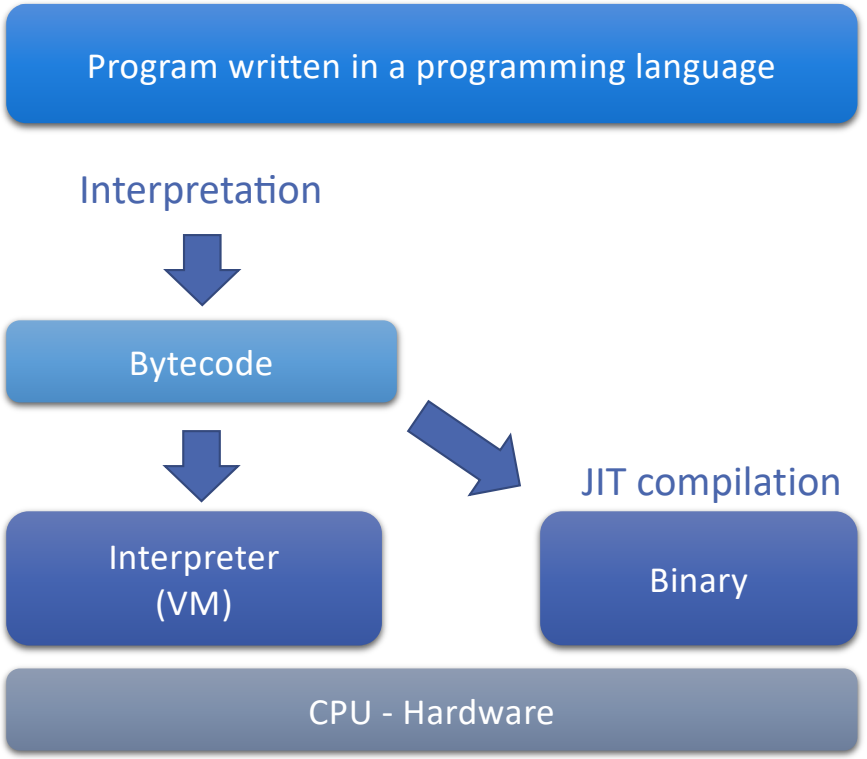
From language to binary execution

- **Interpretation:**
  1. Compile to bytecode
     - Bytecodes are platform-independent
     - Non-runnable code, needs a virtual machine to run
     - Python: collection of opcode and oparg
  2. Interpret in a virtual machine
     - Ex. Python, Java, Javascript
     - Virtual machine takes care of the differences between bytecodes for different platforms
- **Ahead of Time** compilation:
  1. Compile to binary code
  2. Execute in hardware
     - Ex. C, C++, Fortran
- Is there a third approach?



Just in time compilation

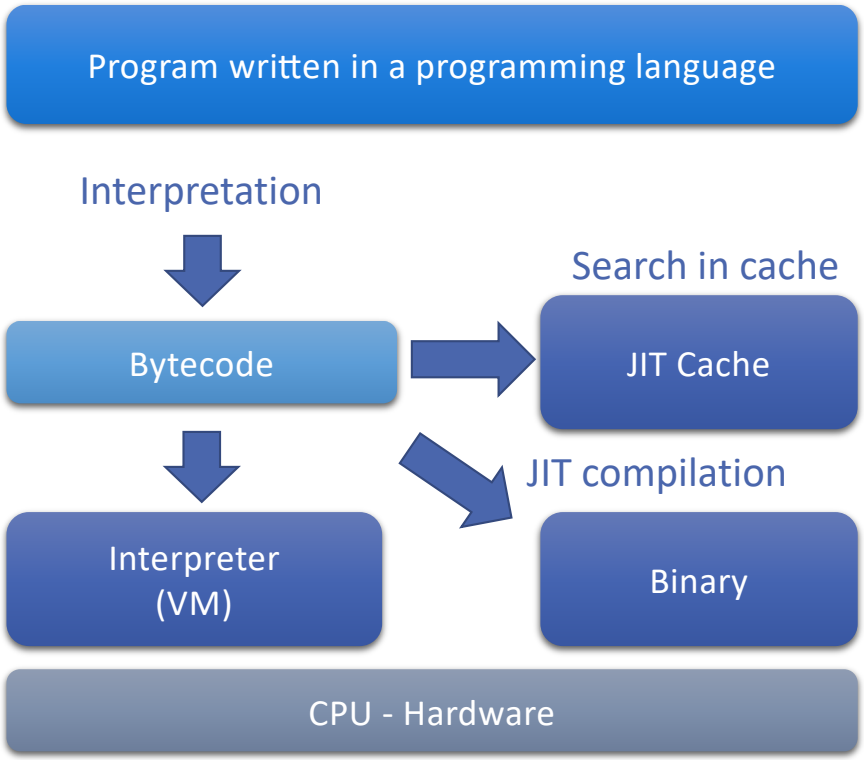
- JIT compilation
  1. Compile to bytecode
  2. Two options:
     1. Default: Execute in VM
     2. **JIT:** Compile to binary on the fly and execute on hardware
- When is it convenient to do JIT?
  - For functions that are going to be executed many times (ex. Chrome browser V8 engine Javascript JIT)
  - JIT allows observing the program's behavior while running and optimizing it on the fly (dynamic analysis and then conversion to machine code)



JIT Binary Caching

- We can keep a cache of already JIT compiled functions

1. Compile to bytecode
2. Search in JIT cache:
    1. If found use the binary
    2. If not found compile to binary
3. Execute



Python JIT Implementation

- Historically not supported by CPython, the standard Python implementation. Experimental support added in Python 3.13.
  - Starting with Python 3.13, CPython has added **an experimental "copy-and-patch" JIT** option. It's **not enabled by default**, but it marks a shift in the default interpreter toward supporting JIT in the core
  - *--enable-experimental-jit*, adds a JIT pipeline.
  - CPython 3.11+ already includes a **specializing interpreter**, which is *not* a JIT but provides significant speedups.

- Supported by other projects such as:
  - PyPy (tracing JIT)
  - Numba (LLVM JIT for numeric kernels)

- Cons:
  - Overhead of the compilation process
  - The degree of performance improvements varies depending on the code and usage patterns

What is PyPY?

- PyPY is:
  - An implementation of Python in Python
  - A very flexible compiler framework (with some features that are especially useful for implementing interpreters)
- PyPY grew out of a desire to modify and extend the implementation of Python
  - Increase performance (JIT compilation and better garbage collectors)
  - Ease of porting to new platforms like the JVM or to low-memory situations
  - Add expressiveness (stackless-style routine, logic programming)
- Python also has other tools and libraries like **Numba** and **Cython** that can help optimize Python code by **compiling selected parts** of it into machine code

How does JIT work in the context of PyPy

- **Parsing and Analysis:** PyPy parses Python code, creates an AST, and performs initial analysis.
- **JIT Compilation:** PyPy's JIT compiler dynamically generates machine code based on runtime execution patterns.
- **Optimizations:** The JIT compiler optimizes machine code by inlining functions, optimizing loops, and eliminating branches.
- **Execution:** CPU executes generated machine code, offering a significant performance boost.
- **Profiling and Deoptimization:** JIT compiler monitors program behavior, deoptimizing when necessary for efficiency.

- You can turn on or off JIT when using PyPy
  - **pypy –jit off**

Python JIT Compilation - Numba

- "Numba runs on CPython but compiles functions outside the interpreter using LLVM.
- **Numba:**
  - Generates optimized code using the LLVM (low level virtual machine) compiler
  - Example: use the @jit decorator to compile at 1st execution
  - Can compile at:
    - Import time
    - Runtime
    - Statically
    - http://numba.pydata.org/numba-doc/0.37.0/user/jit.html
- What is the difference between Numba @jit and a CPython extension?

```
from numba import jit
from numpy import arange

# jit decorator tells Numba to compile this function.
# The argument types will be inferred by Numba when
# function is called.
@jit
def sum2d(arr):
    M, N = arr.shape
    result = 0.0
    for i in range(M):
        for j in range(N):
            result += arr[i,j]
    return result

a = arange(9).reshape(3,3)
print(sum2d(a))
```

https://numba.readthedocs.io/en/stable/user/5minguide.html

Numba vs. Cpython Extensions

Dimension	Numba @jit	CPython Extension
Written in	Python	C / C++
Compilation	Runtime (JIT)	Ahead-of-time
Uses LLVM	Yes	No (usually)
Python support	Subset only	Full
Can define Python types	❌	✅
Memory control	Limited	Full
Zero-copy arrays	Partial	Full
GIL control	Limited	Full
Best for	Numeric kernels	Frameworks & runtimes
Used by PyTorch / TF	❌	✅

Why ML Frameworks Don't JIT Python Itself

- Instead, they JIT:
  - Graphs
  - Tensor programs
  - Kernels
- Examples:
  - PyTorch
  - TorchDynamo → AOTAutograd → TorchInductor
  - TensorFlow / JAX
  - GraphDef / jaxpr → XLA
- Numba
  - JITs numeric kernels, not full Python programs

- Python remains:
  - The orchestration layer
  - The control plane
  - Not the performance-critical path

PyTorch Key Concepts

PyTorch

A Python-based scientific computing package targeted at two sets of audiences:

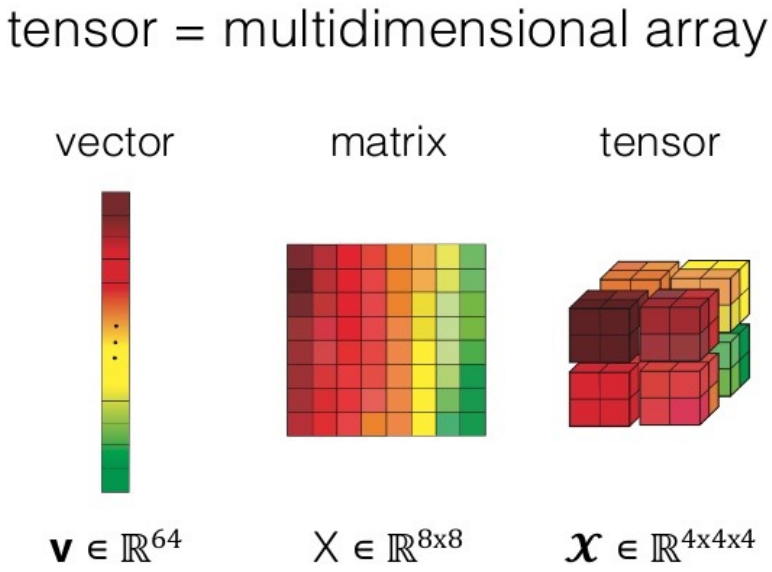
- *A replacement for numpy to use the power of GPUs*
- *A deep-learning research platform that provides maximum flexibility and speed*

[This lesson uses material from http://pytorch.org/tutorials/ throughout.]

- To install: https://github.com/pytorch/pytorch#installation

Tensors

- Tensors are matrix-like data structures which are essential components in deep learning libraries and efficient computation.
- GPUs are especially effective at calculating operations between tensors



From: https://www.slideshare.net/BertonEarnshaw/a-brief-survey-of-tensors

- Tensor operations:
  - ones, zeros, add, dot, etc.
- PyTorch tensors can live on
  - CPU
  - GPU (speedup!)
  - Or other accelerators

PyTorch Tensors

- Import Torch:

```
from __future__ import print_function
import torch
```

- Construct a 2x3 matrix, uninitialized:

```
x = torch.Tensor(2, 3)
print(x)
 0.00e+00  0.00e+00  1.15e-24
 1.58e-29  1.67e-37  2.97e-41
[torch.FloatTensor of size 2x3]
```

- Use tensor in CUDA

```
device = torch.device("cuda")
y = torch.ones_like(x, device=device) # directly create a tensor on GPU
x = x.to(device) # or just use .to("cuda")
```

- Initialize zeros or ones tensors

```
x = torch.zeros(2,3)
x = torch.ones(2,3)
```

- Convert a numpy array

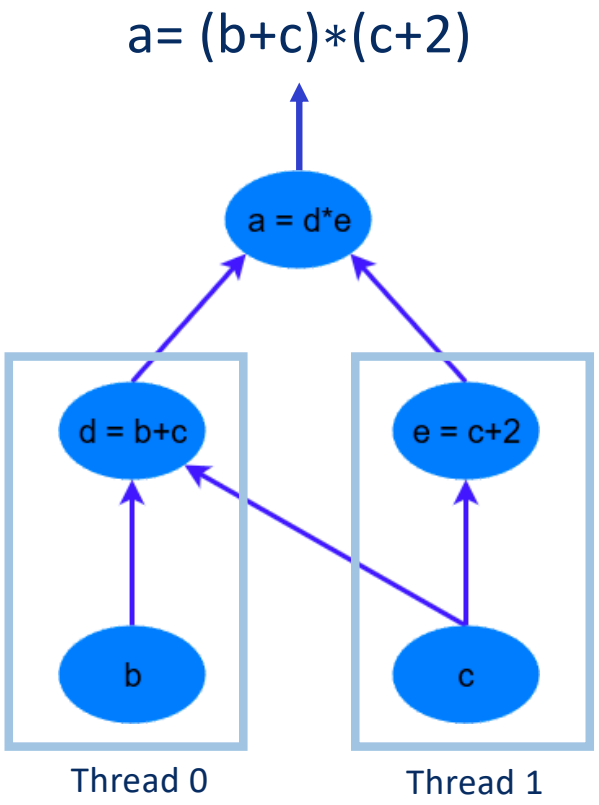
```
b = torch.from_numpy(a)
```

- the slice functionality is available like in numpy

```
x = torch.rand(2,3) # Initialize a tensor randomly
print x[:,1] #second column
0.6297 0.1196
[torch.FloatTensor of size 2]
print x[0,:] #first row
0.9749 0.6297 0.3045
[torch.FloatTensor of size 3]
```

Computation Graphs

- A computational graph represents a function in a directed acyclic graph of its component functions
- **Performance optimizations:** Computation Graph exposes parallelism!
- In PyTorch the graph construction is **dynamic:** the graph is built at run-time
  - Easier debugging
  - Better for some algorithms (RNNs)
- In TensorFlow graph construction is **static:** meaning the graph is "compiled" and then run
  - Compiler adds latency but can also apply optimizations

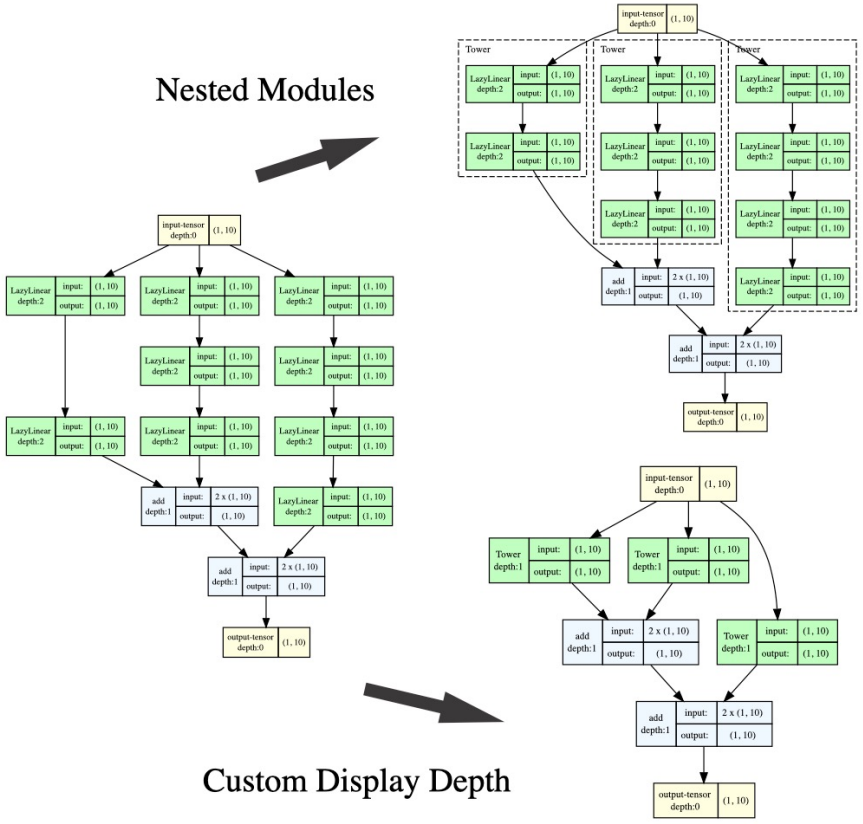


Visualizing PyTorch Computational Graphs

- Example: Torchview provides visualization of PyTorch models in the form of visual graphs. Visualization includes tensors, modules, torch.functions, and info such as input/output shapes.

- Link: https://github.com/mert-kurttutan/torchview

- Example Notebooks
  - **Introduction Notebook**
  - **Computer Vision Models Notebook**
  - **NLP Models Notebook**



Autograd in PyTorch

- **Autograd** builds the Computation Graph **Dynamically**
- The **Tensor** class is the main component of this autograd system in PyTorch (from PyTorch 0.4 version, the *Variable* class is deprecated)
- If you set its attribute *.requires_grad* as *True*, it starts to track all operations on it
- The gradient for this tensor will be accumulated into *.grad* attribute
- Tensors allow automatic gradient computation when the *.backward()* function is called
- Based on the graph, *<variable>.backward()* computes the **gradient** and writes it in ***grad***
  - Example: **b.backward()** computes $\frac{d(y)}{dx}$

```
x = torch.randn(5, 5) # requires_grad=False by default
y = torch.randn(5, 5) # requires_grad=False by default
z = torch.randn((5, 5), requires_grad=True)
a = x + y
a.requires_grad
            False
b = a + z
b.requires_grad
            True
b.backward
```

Chain Rule - Recap

- If we have a function f(x) that is composed of another function g(x), written as:

$$f(x) = h(g(x))$$

- Then the derivative of f(x) with respect to x is:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \cdot \frac{dg}{dx}$$

- The derivative of f with respect to x is the derivative of f **with respect to** g multiplied by the derivative of g **with respect to** x.

PyTorch Tensors, Functions and Gradients

- Create a tensor

```
x = torch.tensor(torch.ones(2, 2) * 2,
requires_grad=True)
```

- Do a simple math equation:

```
z = 2 * (x * x) + 5 * x
```

- To get the gradient of this operation with respect to *x* i.e. *dz/dx* we can analytically calculate this.

- If all elements of *x* are 2, then we should expect the gradient *dz/dx* to be a (2, 2) shaped tensor with 13-values.
- However, first we have to run the .backwards() operation to compute these gradients.
- To compute gradients, we need to compute them with respect to something.
- In this case, we can supply a (2,2) tensor of 1-values to be what we compute the gradients against – so the calculation simply becomes *d/dx*:

```
z.backward(torch.ones(2, 2)*2)
print(x.grad)
        tensor([[ 13.,  13.],
                [ 13.,  13.]])
```

PyTorch Neural Network

- *torch.nn.module* is used to define a neural network
- Example: NN with 3 fully connected layers
  - Using RELU activation for 1st and 2nd layer
  - Input to 1st FC layer: 256 features
  - Input to 2nd FC layer: 120 values
  - Input to 3rd FC layer: 84 values
- Define only forward prop. : backward prop is automatically derived from it

```
import torch
import torch.nn as nn
import torch.nn.functional as F

#Inherit from class nn.Module
class Net(nn.Module):

   def __init__(self):
      super(Net, self).__init__()
      # y = Wx + b
      self.fc1 = nn.Linear(256, 120)
      self.fc2 = nn.Linear(120, 84)
      self.fc3 = nn.Linear(84, 10)

   def forward(self, x):
      x = F.relu(self.fc1(x))
      x = F.relu(self.fc2(x))
      x = self.fc3(x)
      return x
```

PyTorch Loss Function

- Loss function:
  - How "far" from the **target** is the **output** of forward propagation (prediction)
- NN provides various loss functions with syntax:
  - *loss = <loss-function>(output, target)*
  - *loss, output* and *target* are Tensors

```
#net is the network previously defined
#input is the input data of the network
net = Net()
input = torch.randn(256) # a dummy input
output = net(input)
#criterion is a Mean-Squared Error loss function
criterion = nn.MSELoss()

target = torch.arange(1, 11)  # a dummy target
#target comes from the labelled dataset
loss = criterion(output, target)

print(loss)
```

PyTorch Backpropagation and weights update

- First reset gradients of the network
- Compute backward prop.
  - It uses *autograd* inside
- Update the weights with SGD:

  *weight = weight - learning_rate * gradient*
- Different optimization algorithms are in *torch.optim*

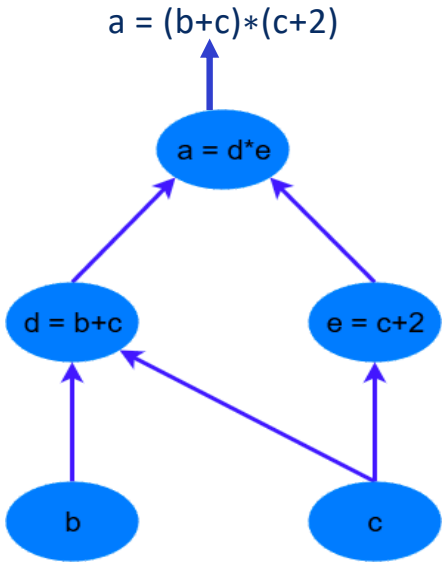
```
# Zeroes the gradient buffer of all parameters
net.zero_grad()

#Backpropagation step
loss.backward()

#Stocastic Gradient Descent weights update
learning_rate = 0.01
for f in net.parameters():
    f.data.sub_(f.grad.data * learning_rate)
```

Computation graph evaluation approaches

- Computation Graph
  - Forward propagation (use it as it is)
  - Backward propagation (compute gradients)

- Two evaluation approaches:
  - **Declarative**: declare all at once, compile, compute
    - TensorFlow, Caffe, Theano
  - **Imperative**: declare and compute each element at runtime
    - PyTorch, Chainer

- Deep Learning Programming Paradigm

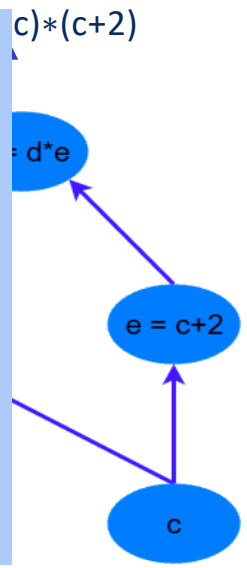


Computation graph evaluation approaches

- Compu
  - Forv
  - Back
- Two ev
  - **Decl**
    - T
  - **Imp**
    - PyTorch, Chainer
- Deep Learning Programming Paradigm

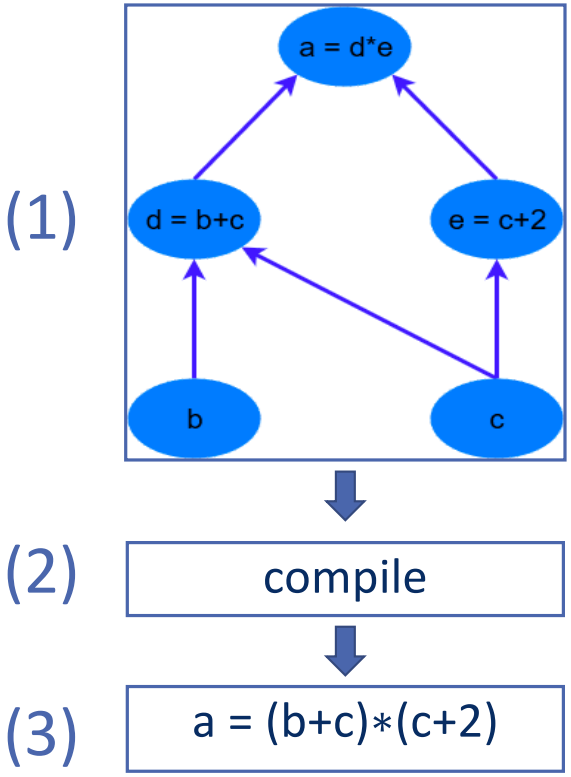
Static graphs = optimized but rigid.

Dynamic graphs = flexible and Pythonic, but less globally optimized.



Declarative/Symbolic approach

- Declarative approach:
  1. **Declare** the full computation graph in a high-level language
     - (ex. Python operators)
  2. **Compile** it and **optimize** based on full knowledge of the computation
     - Memory management opt, Operations fusion, etc.
     - Compiled computation graph can be run on environments without Python Interpreter like edge devices, mobile, backend devices
  3. **Compute** it on the **computing engine**
     - Separate compute engine can be highly optimized for performance



Declarative framework example: TensorFlow

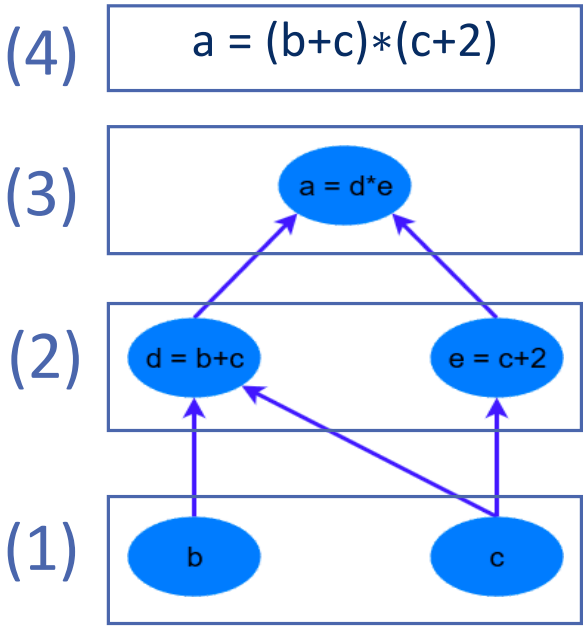
1. Declare:
   - Constants
   - Variables
   - Operators
2. Create session
   - Engine start/end
- Execute engine

```
from __future__ import print_function
import tensorflow as tf

# Basic constant operations (a and b represent the output)
a = tf.constant(2)
b = tf.constant(3)

with tf.Session() as sess:
    print("a=2, b=3")
    print("Addition with constants: %i" % sess.run(a+b))
    print("Multiplication with constants: %i" % sess.run(a*b))

# Basic Operations with variable as graph input
# The value returned by the constructor represents the output
# of the Variable op. (define as input when running session)
# tf Graph input
a = tf.placeholder(tf.int16)
b = tf.placeholder(tf.int16)

# Define some operations
add = tf.add(a, b)
mul = tf.multiply(a, b)

# Launch the default graph.
with tf.Session() as sess:
    # Run every operation with variable input
    print("Addition with variables: %i" % sess.run(add, feed_dict={a: 2, b: 3}))
    print("Multiplication with variables: %i" % sess.run(mul, feed_dict={a: 2, b: 3}))
```

Imperative approach

1. **Start** declaring computation graph
2. Execute each single component of the graph: **do not wait** for full graph declaration
3. If more components are added keep computing

- Graph is built on the fly while the program is executed



Eager mode of execution in DL

- **Eager execution** (or eager evaluation): computational graph (the set of steps needed to perform forward or backwards propagation through the network) is build at runtime by the framework, e.g., Pytorch
  - Code that is executing the mathematical operations involved is ultimately a C++ or CUDA kernel
  - Result of each individual operation is immediately transferred to (and accessible from) the Python process as Python process manages the computation graph
  - Debugging is much easier `import pdb; pdb.set_trace()`
  - Good developer experience
- Tensorflow used **graph mode execution** by default in version 1, switched to using eager execution by default in TensorFlow 2

Graph mode of execution in DL

- **Graph execution = define, compile, then run**
  - Enables global optimizations: memory reuse, operator fusion, parallel scheduling
  - Faster than eager due to reduced Python overhead
- **Trade-offs**:
  - Harder to debug (intermediate states not directly visible)
  - Requires specialized tooling (C++/CUDA debuggers)
- **Latest Framework Support**:
  - **TensorFlow 2**: eager by default, graph mode with @tf.function
  - **PyTorch 2**: eager by default, but torch.compile (Dynamo + Inductor) dynamically builds and optimizes graphs
  - **JAX**: all code compiled to graph (via XLA)
- **Why it matters**:
  - **Development**: Eager preferred for research/prototyping
  - **Production**: Graph mode preferred for deployment (performance, portability, no Python overhead)
  - **Modern trend**: Hybrid execution (start eager → compile to graph automatically)

Summary: Declarative vs. Imperative Approaches

Declarative (Symbolic)	Imperative (Eager Execution)
• Define the **entire graph first**, then compile and run • Global view of computation → allows **optimizations** (memory mgmt, op fusion) • Requires a **session/execution engine** • Less intuitive for debugging (need to inspect after graph execution) • Examples: **TensorFlow (v1)**, **Theano**, **Caffe**	• Build graph **step by step** as program runs • Immediate execution → easier to debug (print intermediate results) • Graph is **dynamic** → adapts easily to loops, conditionals (great for RNNs) • More flexible, but fewer global optimizations • Examples: **PyTorch**, **Chainer**

Declarative vs. Imperative approach comparison

	Declarative	**Imperative**
Productivity		
Debugging		
Static analysis/optimization		

Declarative vs. Imperative approach comparison

	Declarative	Imperative
Productivity	-	+
Debugging	-	+
Static analysis/optimization	+	-

Co-evolution of execution modes in DL frameworks

- TensorFlow, which started as a graph framework, now supports eager.
- PyTorch, which started as an eager framework, now supports graph

Recap of PyTorch Fundamental Concepts

Tensors

- Like a NumPy array but can run on hardware accelerators such as GPU

Autograd

- A Pakage for building computational graphs out of Tensors, and automatically computing gradients

Module

- A neural network layer that may store the state of the learnable parameters

PyTorch Autograd Example

Example adopted from Justin Johnson

PyTorch Autograd Example

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example

Creating tensors and setting **requires_grad=True** enables the Autograd feature

Operations on tensors with **requires_grad=True** cause PyTorch to build a computational graph

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example

Loss Gradients will not be computed with respect to data

Gradients will be computed with respect to weights

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example

In the forward pass, we don't need to track intermediate values. PyTorch tracks them for us in the graph

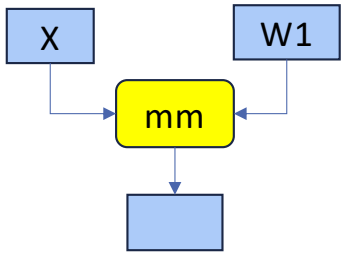
```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example

All the gradients will be computed with respect to all inputs that have **requires_grad=True (w1 and w2 in this example)**

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

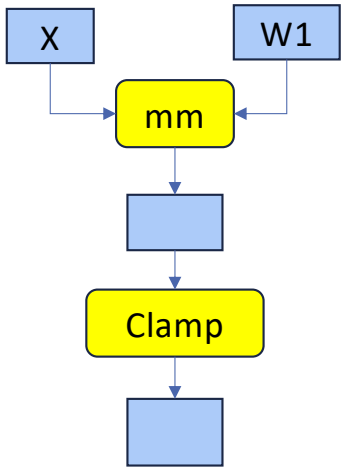
PyTorch Autograd Example




```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

Every operation on a tensor with **requires_grad=True** will add to the computational graph. The resulting tensor will also have **requires_grad=True**

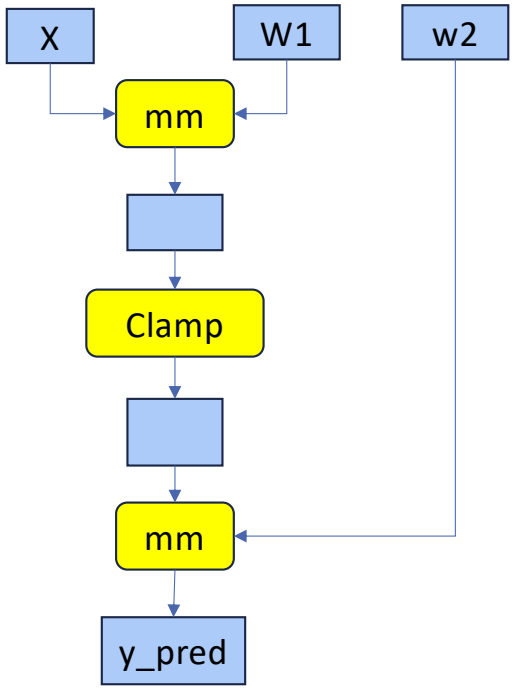
PyTorch Autograd Example



Every operation on a tensor with **requires_grad=True** will add to the computational graph. The resulting tensor will also have **requires_grad=True**

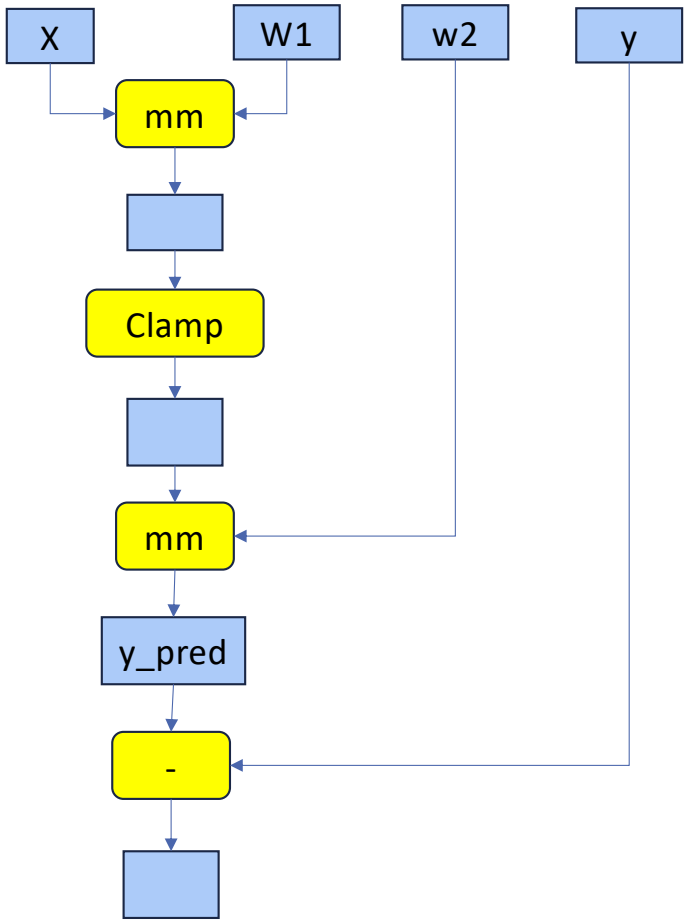
```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example



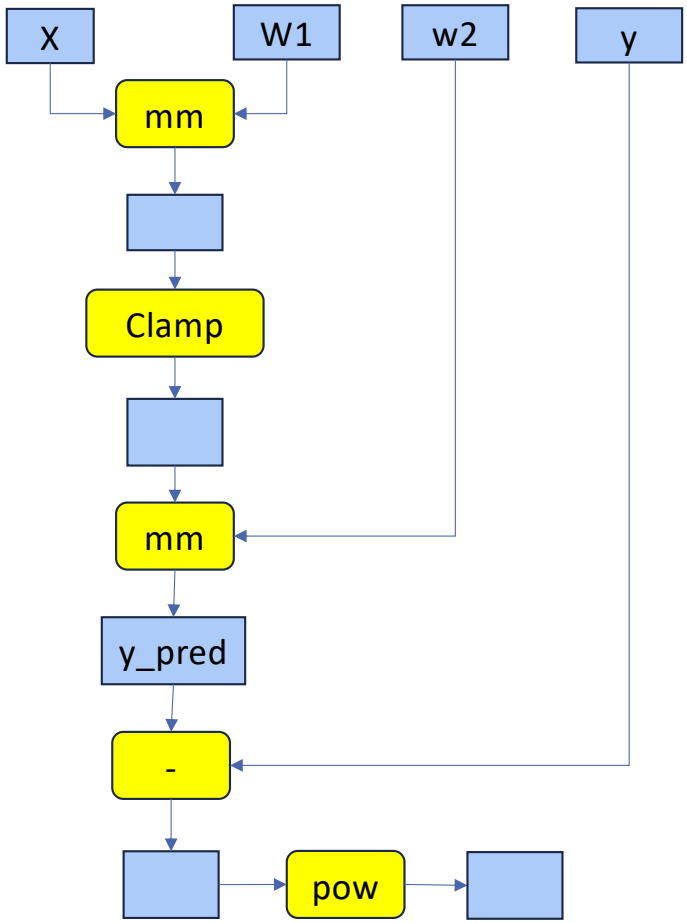

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example



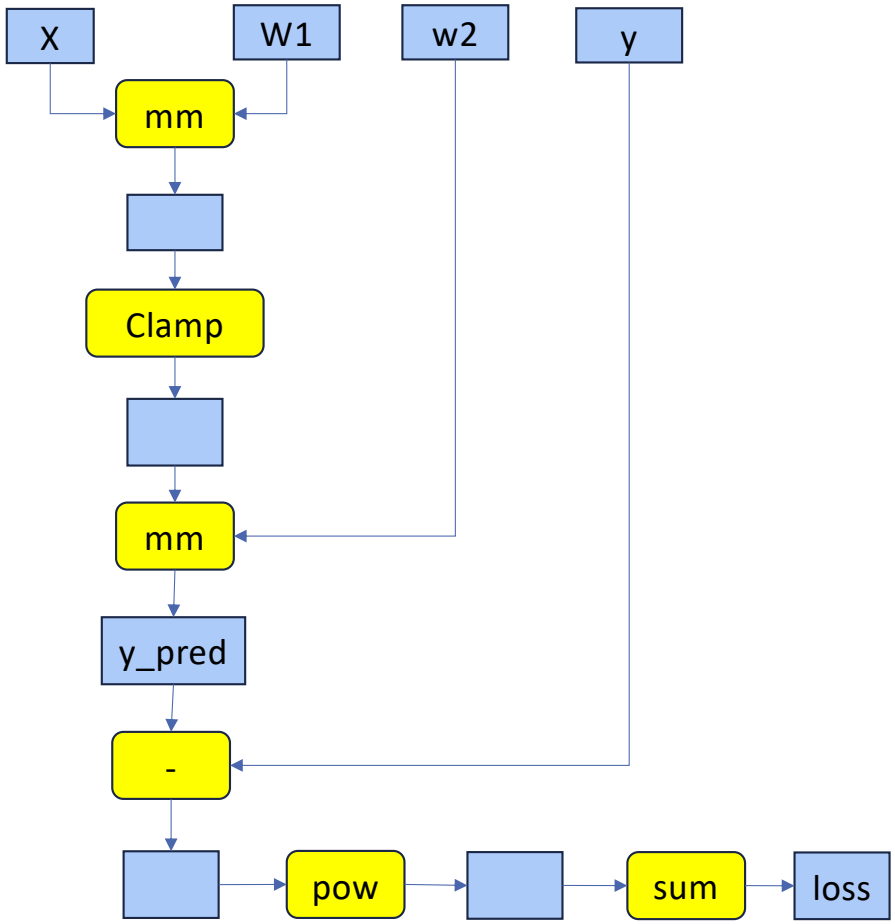

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example



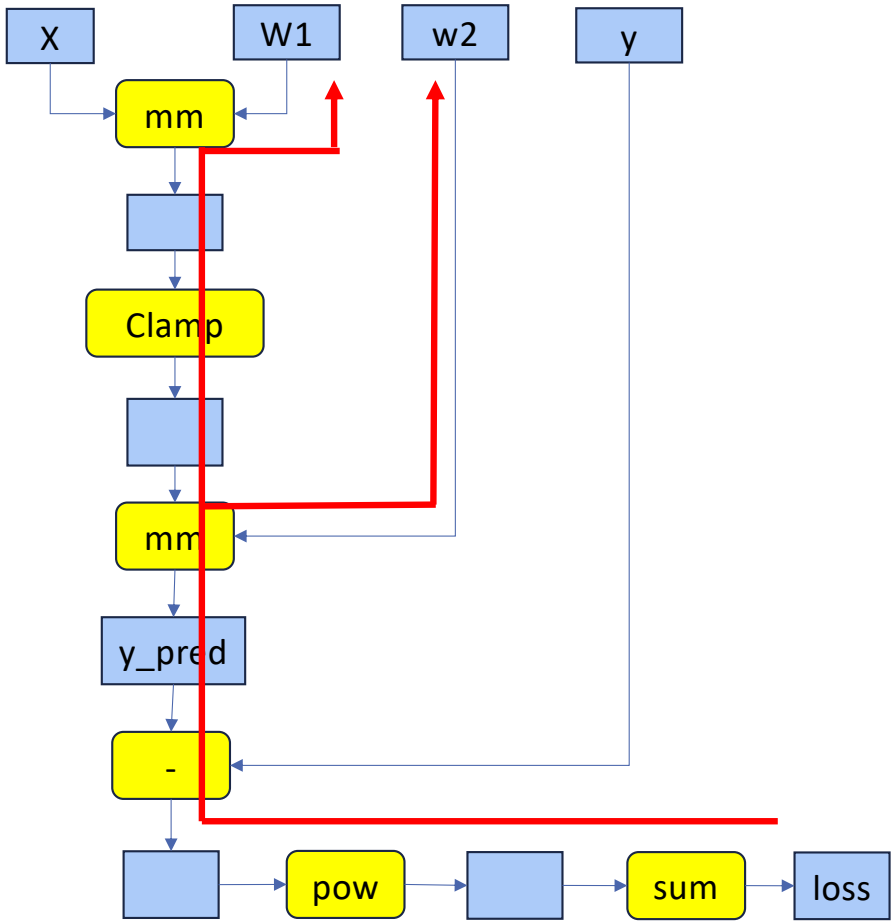

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example




```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example




```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example

At the end of the backward step, gradients are accumulated into w1.grad and w2.grad, and the graph is destroyed.

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example

At the end of the backward step, gradients are accumulated into w1.grad and w2.grad, and the graph is destroyed.

Weights are then updated with the gradients (make gradients step on weights)

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example

At the end of the backward step, gradients are accumulated into w1.grad and w2.grad, and the graph is destroyed.

Forgetting to set the gradients to zero is a common mistake.

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch Autograd Example

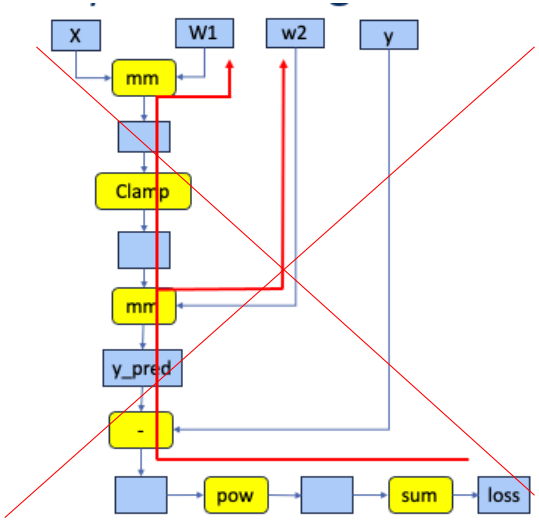
X W1 w2 y

At the end of the backward step, gradients are accumulated into w1.grad and w2.grad, and the graph is destroyed.

This informs PyTorch not to build a computational graph for these tensor operations.

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()

    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        w1.grad.zero_()
        w2.grad.zero_()
```

PyTorch: Dynamic Computation Graph



After each backprop, the dynamic computational graph is thrown away.

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()
```

PyTorch: Dynamic Computation Graph

Dynamic graphs allow using regular Python control flow during the forward pass.

We initialize two different weight matrices for the second layer. Which one to

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2a = torch.randn(H, D_out, requires_grad=True)
w2b = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
prev_loss = 5.0
for t in range(500):
  w2 = w2a if prev_loss < 5.0 else w2b
  y_pred = x.mm(w1).clamp(min=0).mm(w2)
  loss = (y_pred - y).pow(2).sum()

  loss.backward()
  prev_loss = loss.item()
```

PyTorch: Dynamic Computation Graph

Dynamic graphs allow using regular Python control flow during the forward pass.

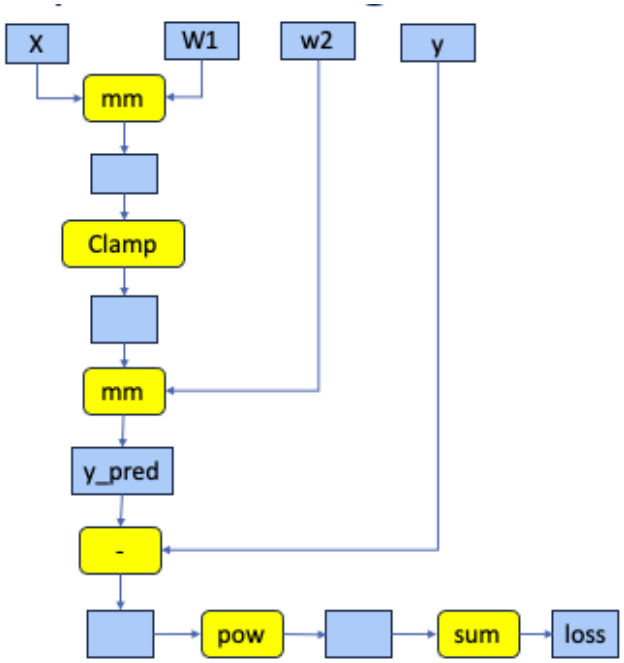
We decide which one to use at each layer based on loss at the previous iteration.

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2a = torch.randn(H, D_out, requires_grad=True)
w2b = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
prev_loss = 5.0
for t in range(500):
  w2 = w2a if prev_loss < 5.0 else w2b
  y_pred = x.mm(w1).clamp(min=0).mm(w2)
  loss = (y_pred - y).pow(2).sum()

  loss.backward()
  prev_loss = loss.item()
```

Alternative Approach: Static Computation Graphs

Step 1: Build the computational graph that describes the computation. This includes finding paths for backpropagation.

Step 2: Reuse the same graph on every iteration




```
graph = build_graph()

for x_batch, y_batch in loader:
    run_graph(graph, x=x_batch, y=y_batch)
```

TensorFlow 1.x style pseudocode

PyTorch Torch.compile

PyTorch: Bridging Python and Performance

PyTorch solves the Python performance problem:

🎨 **Frontend: Pythonic API**	Easy to debug, flexible control flow
⚡ **Backend: Compiled Kernels**	Heavy lifting in optimized C++/CUDA code
🔄 **Autograd: Auto Differentiation**	Builds computation graph dynamically

Why We Need Compilation in PyTorch

Eager Mode Limitations

- **Interpreter Overhead:** Python's eval loop adds latency to every operation.
- **Lack of Global View:** Cannot optimize across multiple operations (e.g., fusion).
- **Memory Inefficiency:** Intermediate tensors are often stored unnecessarily.

Production Requirements

- **Portability:** Deploying models in C++ servers or mobile without Python.
- **Hardware Tuning:** Generating kernels optimized for specific GPU architectures.
- **Throughput:** Maximizing utilization of modern, high-speed accelerators.

The Goal: Optimize performance without sacrificing ease of use.

Legacy PyTorch Optimization: TorchScript

Tracing (torch.jit.trace)

> Records operations on example inputs.

> **Pros:** Fast, minimal code changes.

> **Cons:** Fails on dynamic control flow (if/loops); can be silently incorrect.

Scripting (torch.jit.script)

> Analyzes Python source code to create a graph.

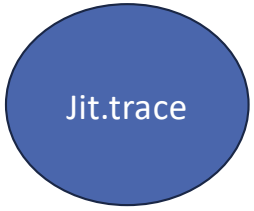
> **Pros:** Handles dynamic control flow; explicit error messages.

> **Cons:** Requires code refactoring; limited Python subset support.

Legacy Status

TorchScript is now considered legacy in PyTorch 2.0+. It has been superseded by **torch.compile**, which offers better performance and a "Zero Code Change" experience.

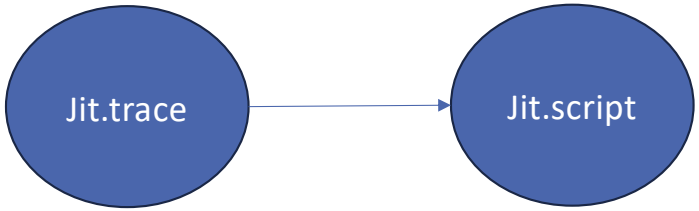
PyTorch Compiler Journey



Trace: Capturing graphs vis tracing functions

- Run a model with certain inputs
- Records or traces all the executed operations into a graph
- Only correct if the function is independent of the data it operates on

PyTorch Compiler Journey



TorchScript

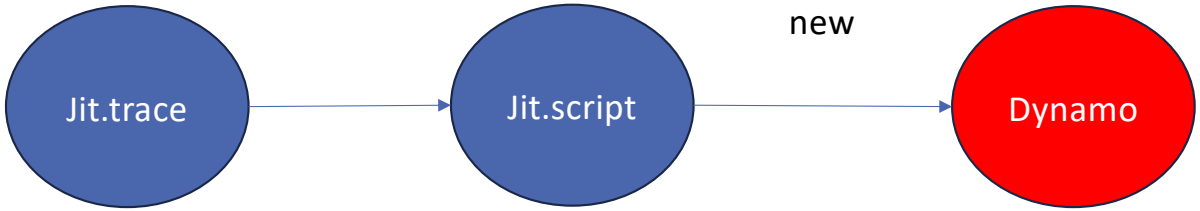
Capturing graphs through source code inspection

- TorchScript is a statically typed subset of Python
- **jit.script** inspects the PyTorch source code and compiles it into a **TorchScript graph**.
- Requires the source code to be refactored and free of 3rd party libraries.

Proposed Solution in PyTorch 2.0

```
cmodel = torch.compile(model)
```

PyTorch Compiler Journey



The Modern Solution: torch.compile

torch.compile(model)

The flagship feature of **PyTorch 2.0**. A unified JIT compiler stack that provides significant speedups with **Zero Code Changes**.

TorchDynamo

Graph Acquisition. Intercepts Python bytecode and extracts computation graphs safely using guards.

AOTAutograd

Graph Lowering. Captures forward and backward graphs ahead-of-time for joint optimization.

TorchInductor

Graph Compilation. The default backend that generates optimized Triton (GPU) or C++ (CPU) kernels.

Fundamentals of a deep learning compiler

- Optimizing high-level code into optimized low-level code

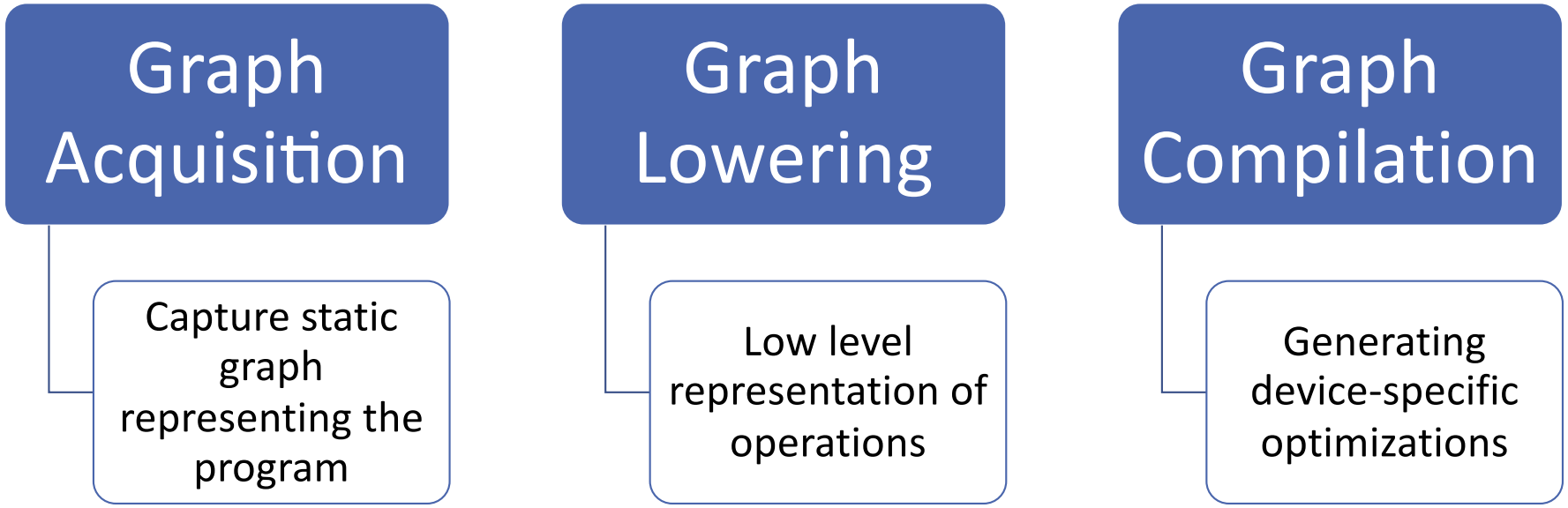
Graph Acquisition

Graph Lowering

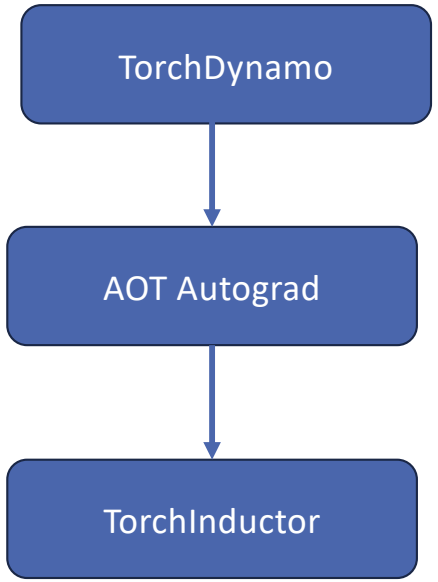
Graph Compilation

Fundamentals of a deep learning compiler

- Optimizing high-level code into optimized low-level code



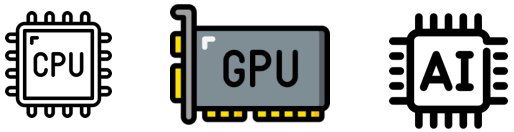
The whole Compiler Stack



Capture graphs of forward ops

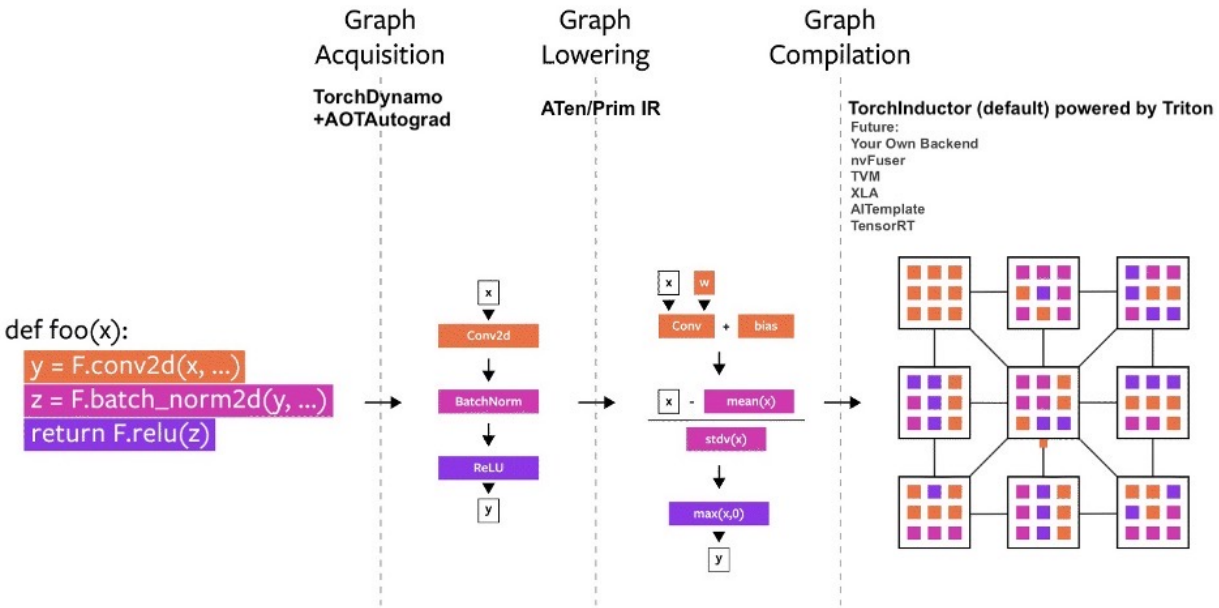
Capture graphs of backward ops

Lower to device and optimize



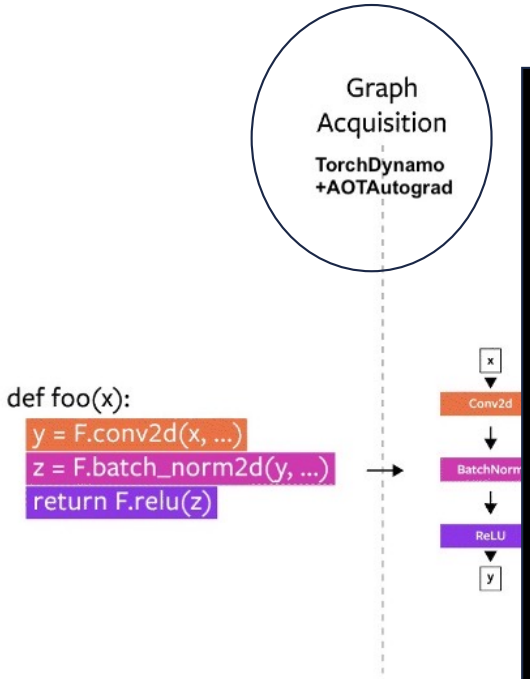
Any new hardware accelerator

The PyTorch Compilation Process



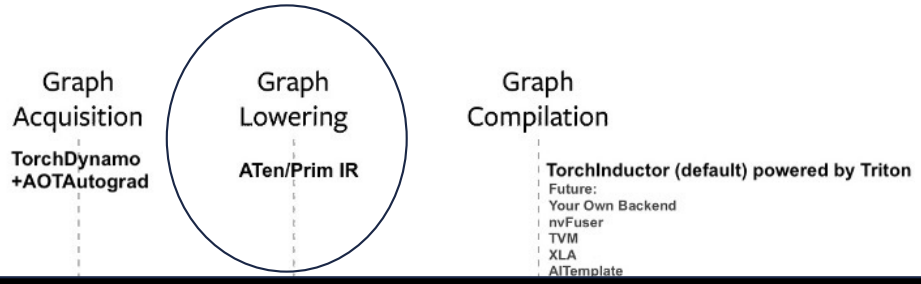
Source: https://pytorch.org/get-started/pytorch-2.0

The PyTorch Compilation Process



1. Graph Acquisition (TorchDynamo + AOTAutograd)
   - The leftmost section shows a Python function foo(x), which contains a sequence of PyTorch operations (conv2d, batch_norm2d, and relu).
   - TorchDynamo is responsible for capturing these operations dynamically at runtime by tracing and converting the eager execution into a graph representation.
   - AOTAutograd (Ahead-of-Time Autograd) is then applied, which extracts the necessary computations for automatic differentiation.
   - TorchDynamo runs during model execution to capture the graph, while AOTAutograd operates ahead-of-time, generating the optimized backward pass before the actual computation starts.

The PyTorch Compilation Process



2. Graph Lowering (ATen/Prim IR)

- Once the computation graph is captured, it is converted into an intermediate representation (IR).
- This IR consists of ATen (PyTorch's core tensor library) and Prim (Primitive) operators, which provide a lower-level abstraction.
- High-level PyTorch operations are decomposed into more granular tensor operations, such as:
  - Conv2d is split into conv + bias
  - BatchNorm is decomposed into its mathematical components (mean, std calculations)
  - Activation functions like ReLU remain as individual operations.

rch-2.0

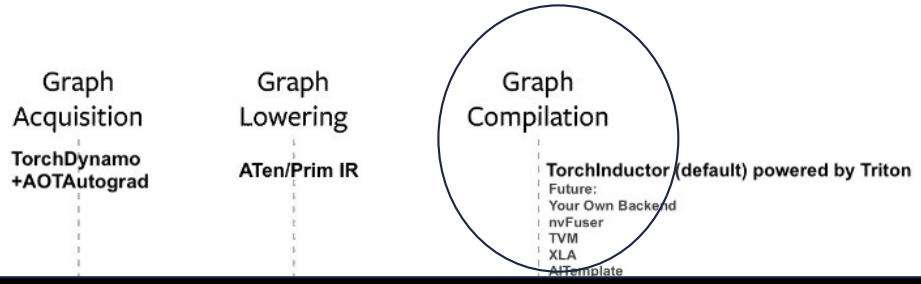
Example: Op decompositions: high-level ops → small, canonical ATen set (1/2)

- **Linear layer (F.linear / nn.Linear)**
  - High level: F.linear(x, w, b)
  - Decomposes to:
    - aten.addmm(x_bias, x, w_t) (with w_t = aten.t(w) and x_bias = b broadcast)
    - If no bias: just aten.mm(x, w_t) or aten.addmm with a zero bias.
- **Softmax / LogSoftmax**
  - F.softmax(x, dim) → numerically stable form:
    - m = aten.amax(x, dim=dim, keepdim=True)
    - e = aten.exp(aten.sub(x, m))
    - d = aten.sum.dim_IntList(e, [dim], keepdim=True)
    - aten.div.Tensor(e, d)
  - F.log_softmax(x, dim) →
    - m = aten.amax(...)
    - logsumexp = m + aten.log(aten.sum(exp(x-m)))
    - aten.sub.Tensor(x, logsumexp)

Example: Op decompositions: high-level ops → small, canonical ATen set (2/2)

- **Cross-Entropy**
  - F.cross_entropy(logits, target) →
    - aten.log_softmax(logits, dim=-1) + aten.nll_loss_forward(...)
    - Backward: aten.nll_loss_backward(...)
- **LayerNorm**
  - F.layer_norm(x, normalized_shape, w, b, eps) →
    - aten.native_layer_norm(x, w, b, normalized_shape, eps)
    - Backward: aten.native_layer_norm_backward(...)
- **Conv2d**
  - F.conv2d(x, w, b, stride, padding, dilation, groups) →
    - aten.convolution(x, w, b, stride, padding, dilation, False, {0,0}, groups)
    - (A single canonical aten.convolution op used by backends.)

The PyTorch Compilation Process



3. Graph Compilation (TorchInductor)

- In this stage, the graph is compiled into optimized kernel code.
- TorchInductor (default compiler in PyTorch 2.0) is used, which is powered by Triton, a deep learning compiler optimized for GPUs.
- Future alternative backends such as TVM, XLA, TensorRT, and custom backends are also supported.
- The final graph is mapped onto a parallelized execution model, where optimized tensor operations are distributed across multiple processing units (e.g., GPU cores or CPU vector units)

rch-2.0

Key Design Points

- TorchDynamo allows PyTorch to remain dynamic while still enabling compilation.
- AOTAutograd improves gradient computation performance.
- TorchInductor uses deep kernel optimization to generate highly efficient code.
- The compilation process significantly improves inference and training speeds without requiring changes to existing PyTorch code.

PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation

Jason Ansel, Meta
Edward Yang, Meta
Horace He, Meta
Natalia Gimelshein, OpenAI
Animesh Jain, Meta
Michael Voznesensky, Meta
Bin Bao, Meta
Peter Bell, Quansight
David Berard, Meta
Evgeni Burovski, Quansight
Geeta Chauhan, Meta
Anjali Chourdia, Meta
Will Constable, Meta
Alban Desmaison, Meta
Zachary DeVito, Meta
Elias Ellison, Meta
Will Feng, Meta
Jiong Gong, Intel
Michael Gschwind, Meta
Brian Hirsh, Meta
Sherlock Huang, Meta
Kshiteej Kalambarkar, Quansight
Laurent Kirsch, Meta
Michael Lazos, Meta
Mario Lezcano, Quansight
Yanbo Liang, Meta
Jason Liang, Meta
Yinghai Lu, Meta
CK Luk, Meta
Bert Maher, Meta
Yunjie Pan, University of Michigan
Christian Puhrsch, Meta
Matthias Reso, Meta
Mark Saroufim, Meta
Marcos Yukio Siraichi, Quansight
Helen Suk, Meta
Michael Suo, Meta
Phil Tillet, OpenAI
Eikan Wang, Intel
Xiaodong Wang, Meta
William Wen, Meta
Shunting Zhang, Meta
Xu Zhao, Meta
Keren Zhou, OpenAI, George Mason University
Richard Zou, Meta
Ajit Mathews, Meta
Gregory Chanan, Meta
Peng Wu, Meta
Soumith Chintala, Meta

ASPLOS '24, April 27-May 1, 2024, La Jolla, CA, USA

ACM ISBN 979-8-4007-0385-0/24/04
https://doi.org/10.1145/3620665.3640366

Abstract

This paper introduces two extensions to the popular PyTorch machine learning framework, TorchDynamo and TorchInductor, which implement the `torch.compile` feature released in PyTorch 2. TorchDynamo is a Python-level just-in-time (JIT) compiler that enables graph compilation in PyTorch programs without sacrificing the flexibility of Python. It achieves this by dynamically modifying Python bytecode

https://pytorch.org/assets/pytorch2-2.pdf

Dynamo

Tries to combine the best of both worlds

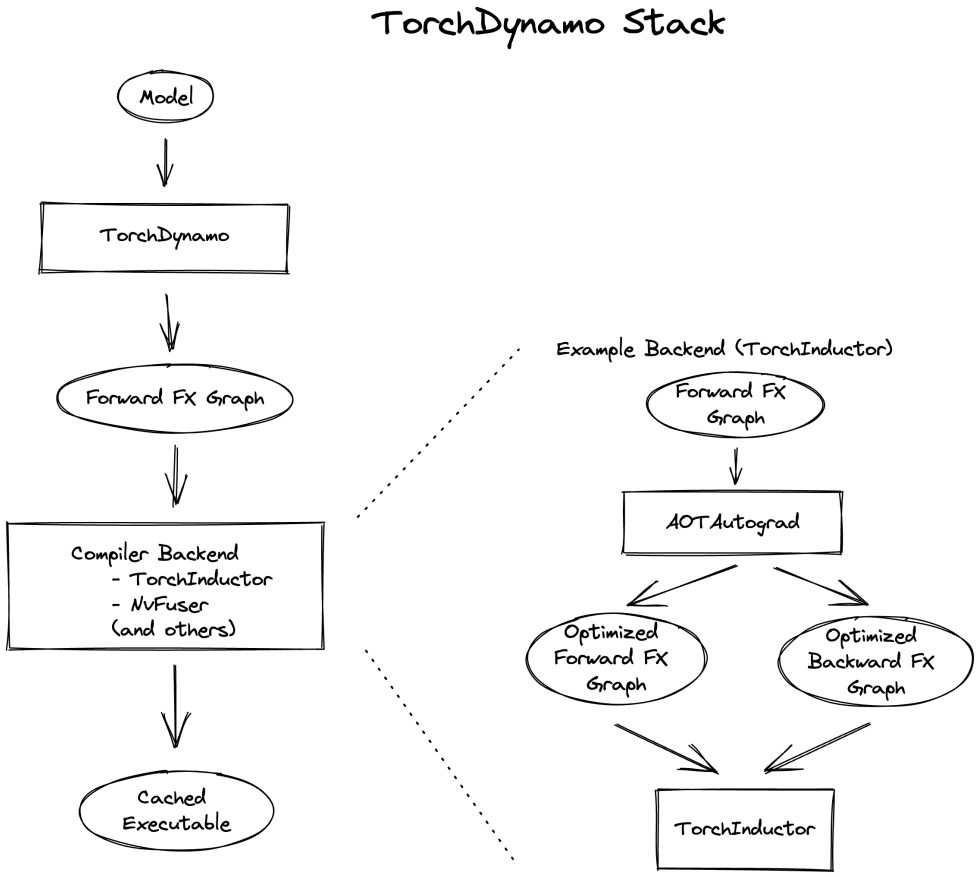
- Graph breaks for completeness
  - Revert to eager mode when needed
- Guards for soundness
  - Ensure all the assumptions are valid
- **Key Benefits:**
  - **Zero Code Change:** Users can optimize their models without modifying their Python code.
  - **Support for Dynamic Models:** TorchDynamo can handle models with dynamic behavior (e.g., loops, if-statements) because it captures code as it runs, making it more flexible than previous static tracing methods.
  - **Performance Improvements:** By compiling code at runtime, it can optimize hot code paths (frequently executed parts of the model) for faster execution, particularly during training.
  - Fallback to Eager Mode:
- When TorchDynamo encounters code it cannot compile or optimize, it can automatically fall back to PyTorch's eager execution mode, ensuring robustness and flexibility.

TorchDynamo

- Dynamo has two tasks:
  - Symbolically tracing the Python code given an example input
  - Caching and guarding the traced code to ensure a sound execution for all inputs
- The inputs of Dynamo are a Python function and its inputs
- The outputs of Dynamo are:
  - FX graphs: For each Dynamo input, a new graph is generated
  - Guards: For each Dynamo input, a set of conditions that make execution sound
  - Cache: A dictionary structure that matches FX graphs to inputs and execution environment through the guards for each graph

Dynamo Stack

- **TorchDynamo**: Intercepts Python model execution and extracts a **Forward FX Graph**.
- **FX Graph**: Intermediate representation of the model suitable for compiler optimizations.
- **Compiler Backends**: FX Graph is sent to backends (e.g., **TorchInductor, NVFuser**) to generate optimized code.
- **Cached Executable**: Compiled output is cached for efficient reuse.
- **AOTAutograd (Ahead-of-Time Autograd)**: Splits the model into optimized **forward** and **backward FX graphs** for training workloads.
- **TorchInductor Example**: Takes both forward & backward graphs, applies optimizations, and produces high-performance executables.



 Source: https://docs.pytorch.org/docs/stable/torch.compiler_troubleshooting.html

Dynamo

1.Original Function (my_func)

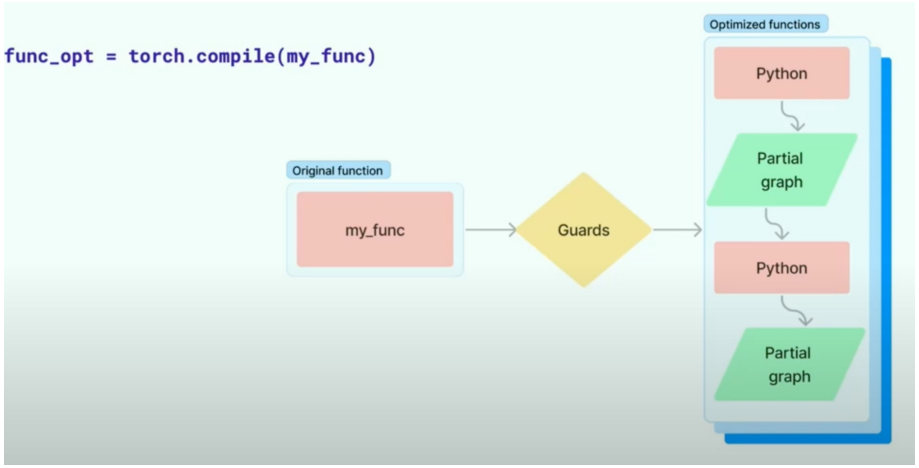
- This is the original Python function containing PyTorch tensor operations.
- It is **wrapped with torch.compile** to enable compilation-based optimizations.

2.Guards

- **Guards analyze input patterns** and execution paths.
- If a function's **control flow remains stable (e.g., loops, conditions, tensor shapes don't change drastically)**, it **compiles into a full optimized graph**.
- If unexpected behavior is detected, **PyTorch falls back to Python execution**.

3.Optimized Execution Path

- The **optimized function consists of alternating segments of compiled graph execution and Python execution**:
  - **Some parts run as compiled graphs** (fast execution).
  - **Other parts remain in Python** when they cannot be statically compiled.



Source: Christian Keller, Meta

The Role of Guards in torch.compile

- **Guards are a key mechanism for caching and reusing compiled code.** They are a set of checks that verify if a function's inputs have changed.
- **When a function is first compiled, a "guard function" is created.** This guard checks aspects like the **shape, data type, and device** of the input tensors.
- **If the guards pass on a subsequent run**, it means the inputs are compatible. The function can then **reuse the cached, highly optimized compiled graph**, leading to significant performance gains.
- **If a guard fails**, it triggers a **"recompilation."** This happens when an input characteristic changes (e.g., a tensor's shape changes), and a new graph is needed to handle the new inputs.

Why Does PyTorch Sometimes Fall Back to Python (Eager mode)?

- While **TorchDynamo** (inside torch.compile) tries to capture computations as a graph, some operations cannot be fully compiled:
  - **Dynamic shapes or control flow** (e.g., loops dependent on tensor values).
  - **Unsupported operations** that cannot be optimized into a compiled kernel.
  - **Python-specific logic** (like string manipulations, printing, or other non-tensor computations).
  - In these cases, **PyTorch automatically switches between compiled and Python execution**, as shown in the figure.
- These can cause what we call **Graph Breaks in certain cases**

When Does torch.compile Break the Graph?

Case	Graph Break?	Why?
Control flow depending on tensor values (if x.sum() > 0:)	Yes	Runtime-dependent condition prevents static tracing.
Python print statements (print(x))	Yes	Cannot be compiled into a tensor operation.
Converting tensor values to Python (int(x.item()))	Yes	Moves computation out of the tensor graph.
Static control flow (if flag: where flag is a function argument)	No	Known at compile time, can be traced.
Using torch.where(x > 0, x * 2, x - 2) instead of if	No	Fully tensor-based, remains in compiled graph.

What are dynamic shapes

- In PyTorch, **dynamic shapes** refer to tensors whose dimensions can vary at runtime. This contrasts with **static shapes**, where tensor dimensions remain fixed throughout execution.
- By default, PyTorch operates with dynamic shapes, meaning tensors can be resized, reshaped, or modified without requiring recompilation, unlike frameworks like TensorFlow (pre-TF2) that rely on static computation graphs.

Examples of Dynamic Shapes in PyTorch

- Example 1: Dynamic Batch Size

```python
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(10, 5)  # Fully connected layer

    def forward(self, x):
        return self.fc(x)

model = MyModel()

# Inputs with different batch sizes
x1 = torch.randn(4, 10)  # Batch size 4
x2 = torch.randn(16, 10)  # Batch size 16

output1 = model(x1)  # Works fine
output2 = model(x2)  # Works fine, PyTorch allows dynamic batch sizes
```

The **batch dimension (first dimension)** is **dynamic**, meaning we can change it between runs.

Examples of Dynamic Shapes in PyTorch

- Example 2: Dynamic Sequence Length in NLP

```
import torch.nn.functional as F

# Example: Variable-length sequences
x1 = torch.randn(1, 5)  # Sequence length = 5
x2 = torch.randn(1, 10) # Sequence length = 10

fc = nn.Linear(10, 3)  # Fully connected layer expecting input of size 10

# Need to pad x1 to match x2
x1_padded = F.pad(x1, (0, 5))  # Pads with zeros to make it (1, 10)

output1 = fc(x1_padded)  # Now it works
output2 = fc(x2)  # Already the right shape
```

The **sequence length (second dimension)** is dynamic.

Why Dynamic Shapes Matter

- **Pros:**
  - **Flexibility** – Models can handle different input sizes without retraining.
  - **Memory Efficiency** – No need to allocate memory for the largest possible shape.
  - **Adaptability** – Useful for real-world applications where input data varies.
- **Cons:**
  - **Performance Overhead** – Optimization is harder because compilers like torch.compile prefer fixed shapes.
  - **Graph Compilation Complexity** – When using torch.compile, dynamic shapes require extra **guards** to track changes.

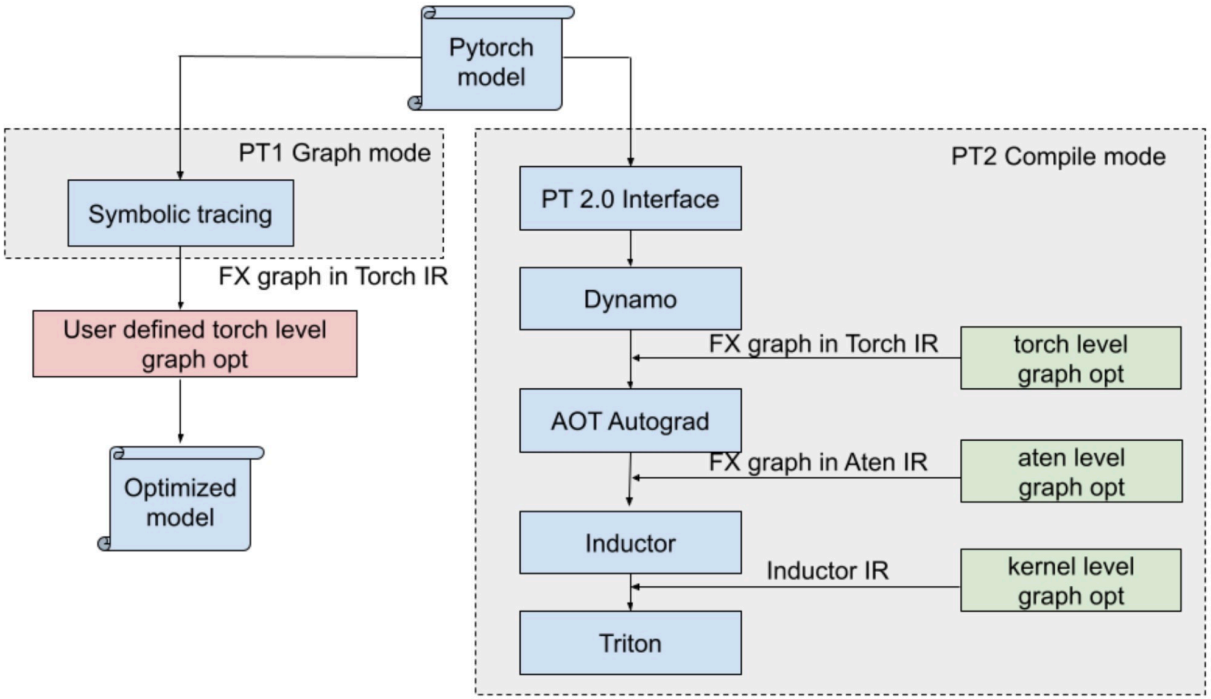
Handling Dynamic Shapes in torch.compile

- By default, torch.compile assumes **static shapes**, meaning it optimizes for a single input shape. However, you can enable dynamic shape support:

```
compiled_model = torch.compile(model, dynamic=True)
```

- This allows compilation to handle variable shapes, but it may introduce performance trade-offs.

PyTorch1 Graph mode vs PyTorch2 Compile mode.



Guard Failures vs. Graph Breaks

- **A guard failure is a "soft break."** The function recompiles, but the code still runs efficiently on the GPU.
- **A graph break is a "hard break."** It stops the compilation process entirely, and a portion of the code is executed in slower **eager mode**, which can cause performance bottlenecks.

AOTAutograd

AOTAutograd (Ahead-of-Time Autograd) is a key component in PyTorch 2.0's compilation stack, designed to optimize automatic differentiation (autograd) computations by separating forward and backward passes and lowering them to an optimized form for compilation.

Traditional Autograd in PyTorch

- In eager mode, PyTorch's autograd engine dynamically records operations performed on tensors at runtime and builds a computational graph for backpropagation.
- This dynamic nature allows for flexibility but introduces Python overhead, making it inefficient for compilation and kernel fusion.

What AOTAutograd Does

- Instead of dynamically tracing gradients at runtime, AOTAutograd captures the computation graph ahead of time.
- It symbolically decomposes the forward and backward computation into a structured Intermediate Representation (IR).
- The forward and backward graphs are separately compiled to prepare for inductor-based optimizations such as: kernel fusion, code optimizations, memory optimizations, etc.

Inductor

- Inductor optimizes code performance by fusing operators
- Further optimizations done at device-specific lowering
- Possible to register custom back ends from inductor
- Inductor is the reference backend implementation:
  - Receives FX graphs, returns compiled artifacts
  - For GPUs: fuses operations, reorders and optimizes memory operations, and runs other machine specific optimizations (there's 20+ optimization passes)
  - Returns a CompiledGraph object that references a python runtime and custom triton kernels (for GPU) or a compiled C++ executable (for CPU)

Who Performs the Optimizations: AOTAutograd vs. TorchInductor?

- Both **AOTAutograd** and **TorchInductor** play critical roles in optimizing PyTorch models, but they serve **different purposes** in the compilation process.

Component	Role in Optimization
AOTAutograd	Captures the forward and backward graphs and optimizes them for later compilation. Performs graph transformations like decomposing ops, removing redundant computations, and reordering execution for efficiency.
TorchInductor	Performs low-level optimizations and generates efficient kernel code. Uses backend optimizations like kernel fusion, loop unrolling, memory reuse, and tiling to improve performance.

TorchInductor's Role: Kernel Level Optimizations

- TorchInductor takes the optimized graph from AOTAutograd and compiles it into highly efficient machine code.
- TorchInductor optimizations include:
  - **Kernel Fusion**: Merges multiple operations into a single efficient CUDA kernel.
  - **Loop Unrolling**: Eliminates loop overhead for faster execution.
  - **Memory Optimization**: Reduces redundant tensor reads/writes by keeping data in registers.
  - **Vectorization & Parallelization**: Uses Triton for auto-tuned GPU kernels that maximize hardware utilization.

Example: Enabling Kernel Fusion Through Graph Transformations

- AOTAutograd enables kernel fusion **indirectly** by **simplifying and restructuring the computation graph**. Here's how:
- **Decomposing Complex Ops to Enable Fusion**
  - Some PyTorch operations are **too complex to be fused directly**.
  - AOTAutograd **rewrites these ops into smaller, more fusion-friendly operations**.

Example: BatchNorm Fusion in Vision Models

```
x = F.conv2d(input, weight, bias)
y = F.batch_norm(x, running_mean, running_var, weight_bn, bias_bn)
output = F.relu(y)
```

AOTAutograd **rewrites BatchNorm as a set of primitive ops**, making it easier for TorchInductor to fuse:

```
x = F.conv2d(input, weight, bias)
mean = x.mean(dim=[0, 2, 3], keepdim=True)
var = x.var(dim=[0, 2, 3], keepdim=True, unbiased=False)
y = (x - mean) / torch.sqrt(var + eps)
output = weight_bn * y + bias_bn
output = F.relu(output)
```

BatchNorm is just element-wise arithmetic, which is easy for **TorchInductor to fuse with ReLU and Conv2D** into one kernel. This avoids **multiple kernel launches** and **reduces memory access overhead**.

Example: Fusion Operations in **Vision Workloads** (Convolutional Models)

- Example: Fusing BatchNorm and ReLU into Convolution
  - Many CNN-based models (e.g., ResNet, EfficientNet) apply a sequence of operations like:
  - Convolution
  - Batch Normalization
  - ReLU activation
- These operations are typically implemented separately in PyTorch, requiring multiple kernel launches. AOTAutograd can fuse them into one optimized kernel.

Before Fusion (Standard PyTorch Execution)

```
y = F.conv2d(x, weight, bias)
z = F.batch_norm(y, running_mean, running_var, weight_bn, bias_bn)
output = F.relu(z)
```

Fused Computation in One Kernel

```
output = fused_conv_bn_relu(x, weight, bias, running_mean, running_var, weight_bn, bias_bn)
```

Example: Fusion Operations NLP Workloads (Transformer Models)

- **Example: Fusing LayerNorm, Linear, and GELU**
- Transformer-based models like **BERT, GPT, and T5** often perform:
  - **Layer Normalization** (stabilizes activations)
  - **Linear Transformation** (fully connected layer)
  - **GELU Activation** (smooth non-linearity)

Before Fusion (Standard PyTorch Execution)

```
x = F.layer_norm(input, normalized_shape, weight, bias)
y = F.linear(x, weight, bias)
output = F.gelu(y)
```

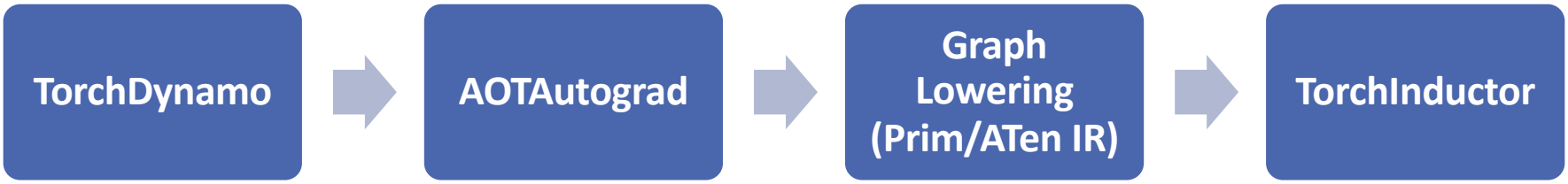
Fused Computation in One Kernel

```
output = fused_layernorm_linear_gelu(input, weight, bias, normalized_shape)
```

After AOTAutograd Fusion

- Fuses LayerNorm, Linear, and GELU into a single kernel.
- Minimizes memory reads/writes by keeping activations in registers.
- Optimized tensor contractions and vectorized math operations.

Summary



Captures Python-based PyTorch code dynamically and converts it into an FX graph.

Partitions the graph into **forward and backward** components and optimizes gradient computation.

Translates high-level PyTorch operations into lower-level tensor operations.

Generates highly optimized CUDA or CPU kernels using backend techniques like **kernel fusion, memory reuse, and loop optimizations.**

Torch.export

- With the introduction of PyTorch 2.0, the torch.export API was introduced as a mechanism for capturing and exporting models into a stable, structured graph representation. It works alongside torch.compile, but serves a different purpose:
  - torch.export captures the full computation graph in a portable format for deployment and interoperability.
  - torch.compile optimizes execution for performance by dynamically tracing and lowering the model to optimized machine code.

How torch.export Works

- torch.export freezes a PyTorch model into a structured, fully serialized computation graph that can be used across different runtimes. It enables long-term model compatibility by removing Python-specific behavior.
- Key Features of torch.export
  - **Captures a Static Graph:** Converts dynamic PyTorch models into an **explicit graph representation**.
  - **Backend Agnostic:** The exported model can be run on various runtimes (e.g., ONNX, XLA, TensorRT).
  - **No Python Dependency:** Removes Python-specific constructs like control flow (if, loops) based on tensor values.
  - **Better Deployment Support:** Ensures the model is portable across different systems.

Example

- The model is **traced and serialized** into a structured computation graph.
- All **dynamic control flow is removed**, making it backend-agnostic.
- The exported model **can be used in other runtimes**.

```python
import torch
import torch.export

class MyModel(torch.nn.Module):
    def forward(self, x):
        return torch.sin(x) + torch.exp(x)

model = MyModel()
example_input = torch.randn(10)

# Export the model to a structured graph
exported_model = torch.export.export(model, (example_input,))
print(exported_model)

# Load the model back
```

Torch.compile() Performance

- **Significant Speed-ups**
  - Improves inference time by 10% to 30% depending on the model and hardware
  - Example: ViT and ResNet50 show 15-20% speedups on A100 GPU
- **Kernel Fusion Optimization**
  - Fuses multiple operations into a single kernel to reduce overhead
  - Minimizes memory bandwidth usage and speeds up execution
- **Memory Optimization**
  - Optimizes memory layout to improve cache utilization and reduce cache misses
  - Leads to lower memory usage during execution
- **Enhanced Parallelism**
  - Leverages OpenMP (CPU) and Triton (GPU) to maximize parallel execution
  - Efficient distribution of tasks across hardware resources
- **Improved Inference on Specialized Hardware**
  - Demonstrates up to 2x improvement on models like GPT-2, BERT, and ResNet on AWS Graviton processors and A100 GPUs

PyTorch Performance – Profiling

A Different Mental Model Required

GPU PERFORMANCE TUNING

CPU

Optimized for single thread performance
- Majority of chip area is control logic & caches

Complex and deep out-of-order pipelines
- Extract instruction level parallelism

The brain
- Job is to keep the accelerator busy

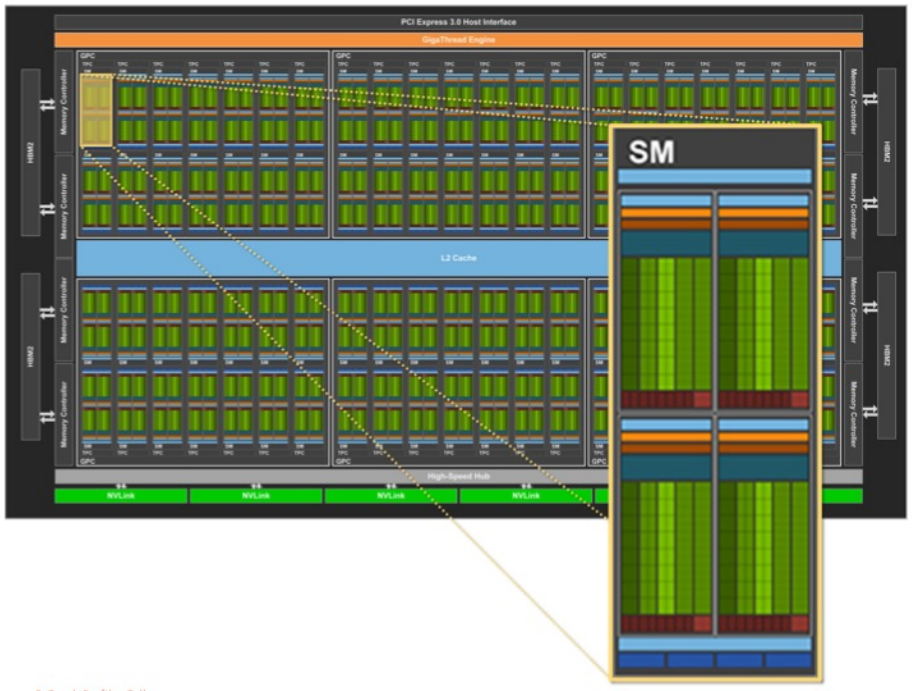
GPU

Optimized for throughput of data-parallel problems
- Majority of chip area is functional units

Simple, relatively slow in-order pipelines
- Achieves much higher total throughput

Accelerator attached via PCIe
- Order of magnitude faster but off to the side

NVIDIA Volta GPU Design



NVIDIA Volta V100 GPU

Composed of Streaming Multiprocessors (SMs)

Volta V100: 80x SMs
Ampere A100: 108 SMs

DGX A100 with 8 GPUs:
864 SMs vs 128 CPU cores

Streaming Multiprocessor Design

Streaming Multiprocessor

64x FP32 units

64x INT, 32x FP64, 32x LD/ST

8x Tensor Cores

5120 (6920 ON A100)
FP32 EXECUTION UNITS
PER GPU

Common Pitfalls

- Excessive CPU/GPU interactions – e.g., for loop launching GPU operations
  - Dominated by launch overheads
- Short GPU kernel durations – e.g., small inputs
  - Need enough data to feed 10s of thousands of threads
- CPU overheads and I/O bottlenecks are starving the GPU
  - Small operations on the CPU can quickly become dominant
- Framework inefficiencies
  - E.g., unnecessary copies and hidden CPU-side overheads

Visibility is key to understand what is going on and optimize your code

Profiling objectives

- **Measure** and identify critical sections and resource bottlenecks
  - CPU, GPU, Memory, Network, I/O (disk)
- Then**, Tune** the model based on the measured and profiled data

- **What can be profiled?** Almost everything with the right tools…
  - Hardware activity: performance counters
  - Operating system (perf)
  - Memory operations (perf, *valgrind*)
  - Libraries
  - Applications
  - Parallel Distributed Applications (MPI profilers…)

Profiling and Tracing Techniques Review

- **Counting** (Deterministic):
  - Count every time a hardware/software event happens (ex. memory load, function call)
  - Report a table of events count
- **Sampling** (Indeterministic: statistical effect):
  - Interrupt the application at **regular intervals** (sampling frequency) and increment a counter associated with the instruction that was interrupted
  - Compute a histogram associating samples to lines of code
  - Can be used to statistically **infer** the relative time in each part of the code
- **Tracing** (Deterministic):
  - Record every time a hardware/software event happens and also the time at which it happens (timestamp)
  - Report a table of relative time spent in each event

Profiling/Tracing techniques Overhead comparison

Counting:

- Mem. footprint: a counter for each (software/hardware) event
  - **low**

Sampling:

- Mem. footprint: state of the program (instruction counter minimum) at each interval
  - **medium** (depends on sampling frequency and state size)

Tracing:

- Mem. footprint: event type + timestamp at each (software/hardware) event
  - **high**

Identifying Bottlenecks

- You need to identify bottlenecks in the system before optimizing it.
  - Helps focus the optimization efforts on areas with the biggest impact on performance
- Bottlenecks can vary depending on several factors:
  - Size of the dataset
  - Complexity of the model
  - Hardware being used

- Examples:
  - Large dataset -> data loading step could be a bottleneck
  - Model is complex -> training steps could be a bottleneck
  - Code not using GPU -> CPU could be a bottleneck
  - Code using GPU -> bottleneck could be GPU memory or bandwidth between CPU and GPU

How to identify bottlenecks

- Traditional Command Line Tools
  - Helpful in monitoring PyTorch training and identifying bottlenecks
  - Easy to use
  - They can be accessed from any terminal
  - Can monitor various metrics: CPU usage, GPU usage, memory usage, and I/O traffic
- Visualization and tracing tools
  - Examples: Tensorboard, PyTorch Profiler Chrome tracing visualization, weights and biases, Flame Graph visualization, etc.
- Examples:
  - Large dataset -> data loading step could be a bottleneck
  - Model is complex -> training steps could be a bottleneck
  - Code not using GPU -> CPU could be a bottleneck
  - Code using GPU -> bottleneck could be GPU memory or bandwidth between CPU and GPU

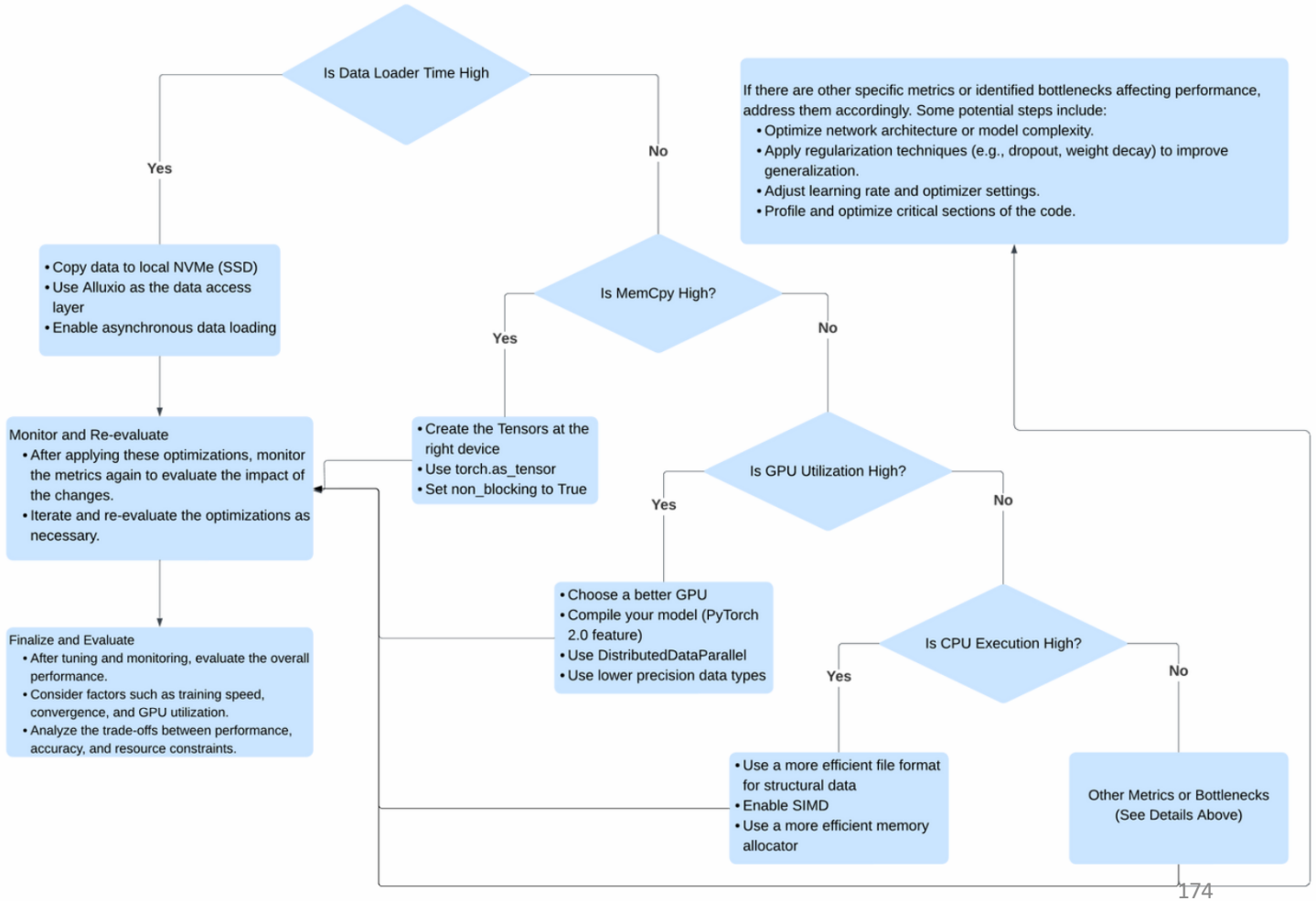
Common Profiling Command Line tools

- Most used command-line tools for monitoring resource usage:

Tool	Description
nvidia-smi	This tool provides information about GPU utilization, memory usage, and other metrics related to the NVIDIA GPU
htop	It is a command-line tool that hierarchically displays system processes and provides insights into CPU and memory usage
iotop	With this tool, you can monitor I/O usage by displaying I/O statistics of processes running on your system
gpustat	It is a Python-based user-friendly command-line tool for monitoring NVIDIA GPU status.
nvtop	Similar to nvidia-smi, nvtop displays real-time GPU usage and other metrics in a user-friendly interface.
py-spy	It is a sampling profiler for Python that helps identify performance bottlenecks in your code.
strace	This tool allows you to trace system calls made by a program, providing insights into its behavior and resource usage.

A General Performance Tuning Guide

A decision tree that shows a structured process for tuning PyTorch training based on key metrics such as GPU utilization, CPU execution, data loader time, and memory copy (memcpy).



Reference: PyTorch Model Training Performance Tuning Ebook by Alluxio

Python profiling tools

- **PyTorch Profiler**
- ***profile:*** pure python module: higher overhead
- ***cProfile*****:** CPython extension modules that traces the execution of Python programs, collecting information on the functions and primitives used:
  - Number of calls
  - Total time (time spent in the function/primitive, excluding nested calls)
  - Cumulative time (time including nested calls)
  - Call graph

  cProfile is the C implementation of the profile interface
- ***pstats***: a module that provides analysis methods for the data collected by the profilers

PyTorch Profiler

- Slides adopted from:

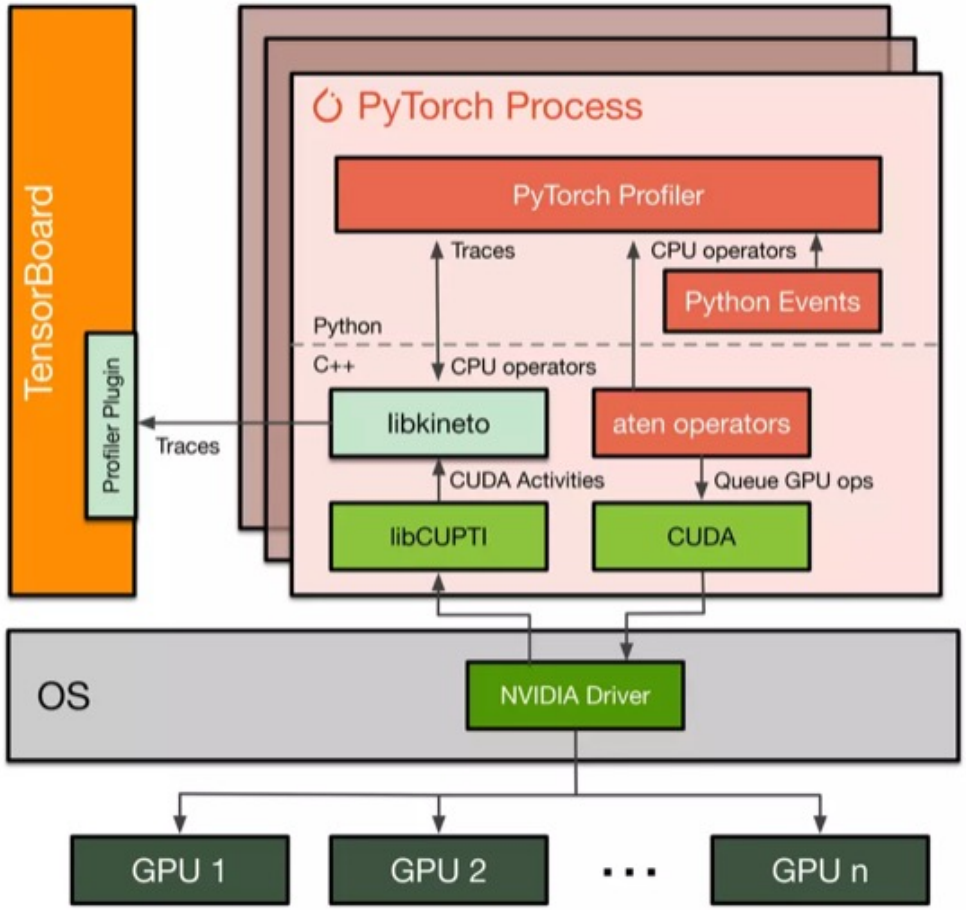
  GEETA CHAUHAN

  PYTORCH PARTNER ENGINEERING

  META AI

PyTorch Profiler

- Contributed by Microsoft & Facebook
  - PyTorch and GPU-level information
  - Automatic bottleneck detection
  - Actionable performance recommendations
  - Data scientist-friendly lifecycle and tools
  - TensorBoard Plugin – Chrome traces visualization
  - Kineto library- built on CUPTI (NVIDIA CUDA Profiling Tools Interface)
  - Easy-to-use Python API
  - VS Code integration



CUPTI: CUDA Profiling Tools Interfaces
ATEN: Short of "A Tensor Library", is a library on top all other Python and C++ interfaces in PyTorch are built
Libkineto: an in-process profiling library integrated with the PyTorch Profile

Profiling API: Basic Usage

```
import torch.profiler

with profiler.profile(..) as prof:
  <code to profile>

# Print results on console
print(prof.key_averages().table(..))
```

https://pytorch.org/tutorials/recipes/recipes/profiler.html

```
import torch
import torchvision.models as models
import torch.profiler as profiler

model = models.resnet18()
inputs = torch.randn(5, 3, 224, 224)

with profiler.profile(record_shapes=True) as prof:
   with profiler.record_function("model_inference"):
      model(inputs)

print(prof.key_averages().table(sort_by="cpu_time_total", row_limit=10))
```

```
# ------------------------  ------------  ----------  ------------  ---------
# Name                      Self CPU total   CPU total  CPU time avg  # Calls
# ------------------------  ------------  ----------  ------------  ---------
# model_inference           3.541ms         69.571ms    69.571ms      1
# conv2d                    69.122us        40.556ms    2.028ms       20
# convolution               79.100us        40.487ms    2.024ms       20
# _convolution              349.533us       40.408ms    2.020ms       20
# mkldnn_convolution        39.822ms        39.988ms    1.999ms       20
# batch_norm                105.559us       15.523ms    776.134us     20
# _batch_norm_impl_index    103.697us       15.417ms    770.856us     20
# native_batch_norm         9.387ms         15.249ms    762.471us     20
# max_pool2d                29.400us        7.200ms     7.200ms       1
# max_pool2d_with_indices   7.154ms         7.170ms     7.170ms       1
# ------------------------  ------------  ----------  ------------  ---------
```

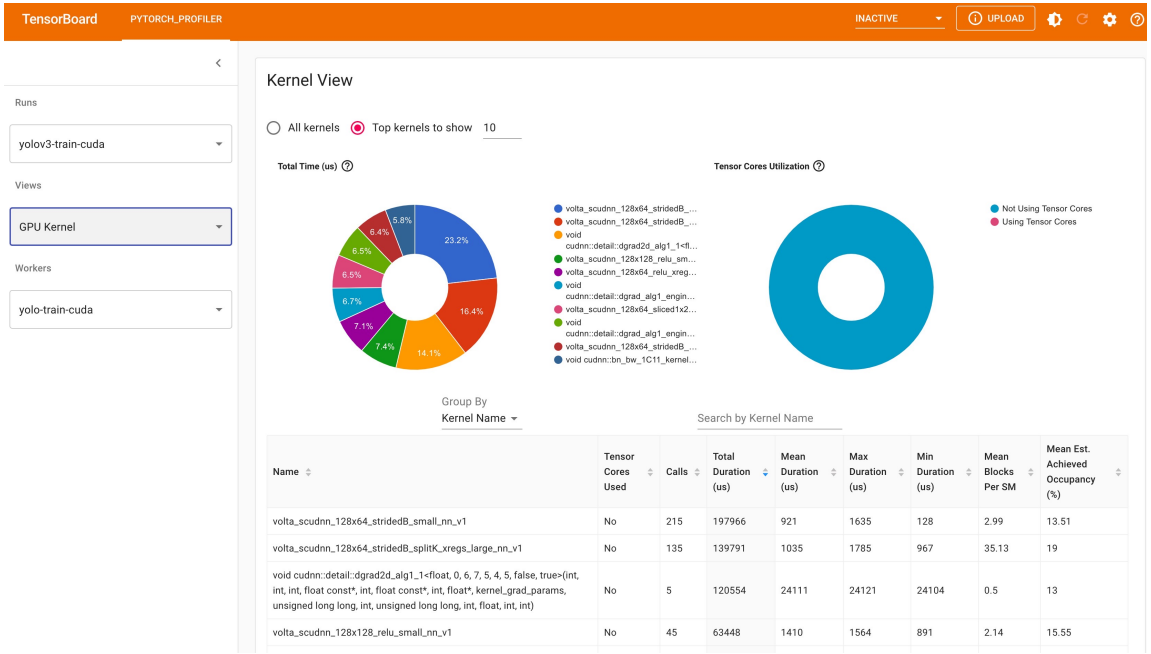
Profiling API: Tensorboard Plugin

```
$PIP INSTALL TORCH-TB-PROFIILER

import torch.profiler

# Export to tensorboard using
# on_trace_handler option
handler=
 tensorboard_trace_handler(..)
with profiler.profile(
 on_trace_ready=handler
) as prof:
 ...
```

```
import torch
import torchvision.models as models
import torch.profiler as profiler

model = models.resnet18()
inputs = torch.randn(5, 3, 224, 224)

with profiler.profile(
  record_shapes=True,
  on_trace_ready=torch.profiler.tensorboard_
trace_handler('results')
) as prof:
   model(inputs)

print(prof.key_averages().table(sort_by=
"cpu_time_total", row_limit=10))
```

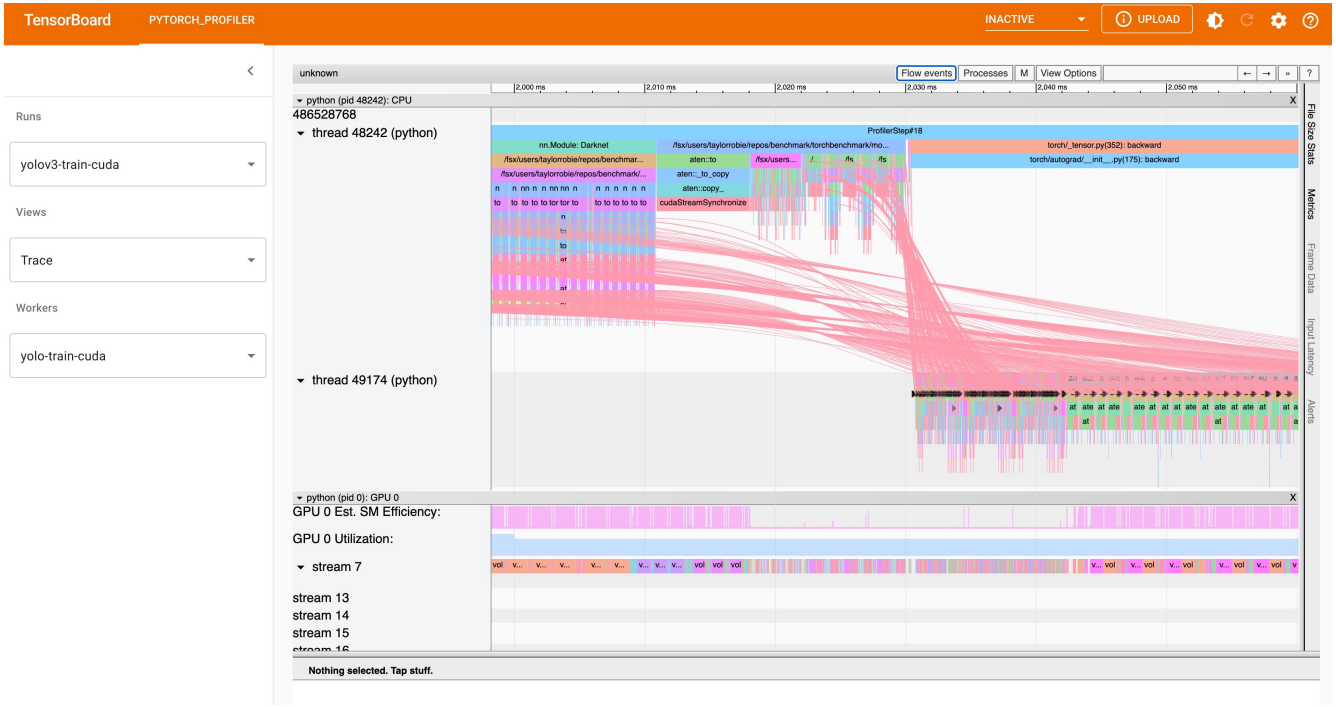
Profiling API: Tensorboard Plugin

```
$PIP INSTALL TORCH-TB-PROFIILER

import torch.profiler

# Export to tensorboard using
# on_trace_handler option
handler=
  tensorboard_trace_handler(..)
with profiler.profile(
  on_trace_ready=handler
) as prof:
  . . .
```



Profiling API: Tensorboard Plugin

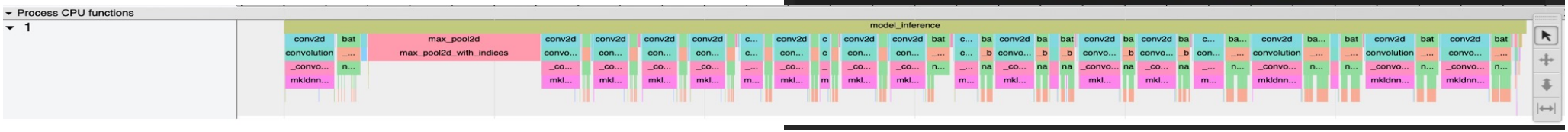
```
$PIP INSTALL TORCH-TB-PROFIILER

import torch.profiler

# Export to tensorboard using
# on_trace_handler option
handler=
  tensorboard_trace_handler(..)
with profiler.profile(
  on_trace_ready=handler
) as prof:
  . . .
```

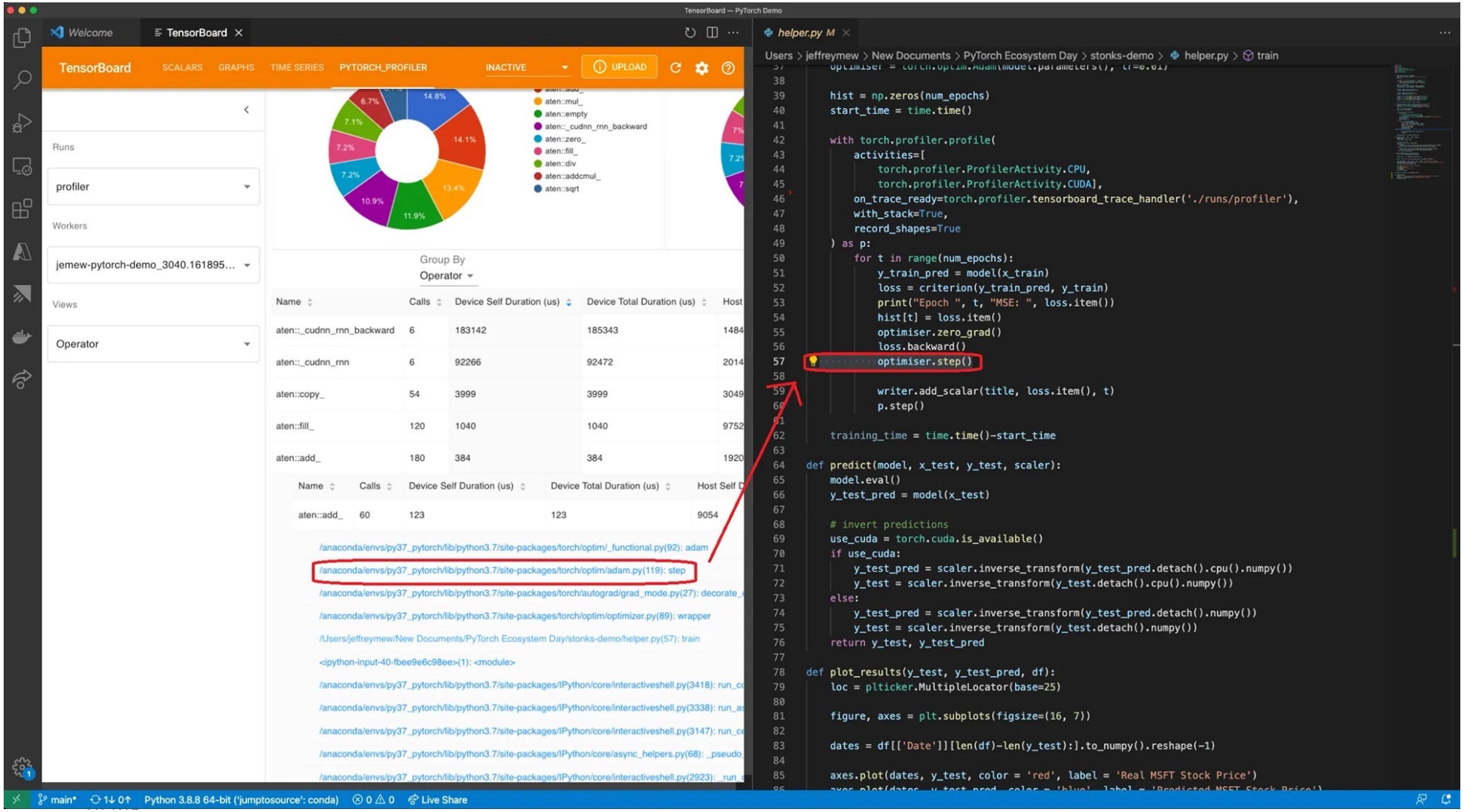


Advanced Options

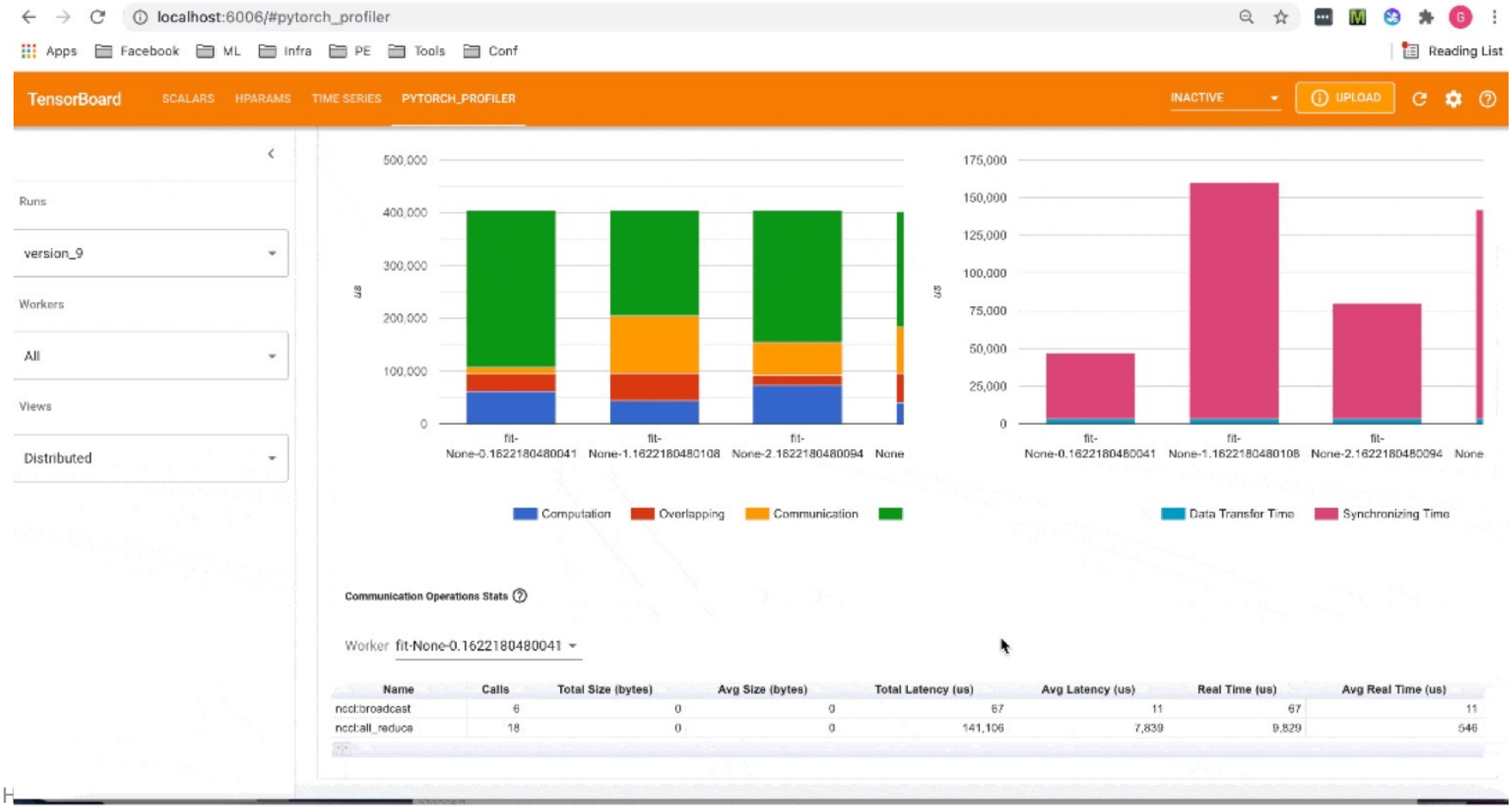
- When to trigger
- How many steps to profile
- Which activities to profile
- Callable handler to save the results
- Extra metadata, eg., shapes, stacks, memory
- Output options: eg.,, Chrome tracing, TensorBoard

```
import torch.profiler as profiler

with torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA],
    schedule=torch.profiler.schedule(
        wait=2,
        warmup=3,
        active=6),
    on_trace_ready=torch.profiler.tensorboard_trace_handler('./result'),
    record_shapes=True,
) as p:
    for step, data in enumerate(trainloader, 0):

        inputs, labels = data[0].to(device=device), data[1].to(device=device)
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if step + 1 >= 11:
            break
        p.step()

    p.export_chrome_trace(path)
    print(p.key_averages().table(
            sort_by="self_cuda_time_total", row_limit=-1))
```

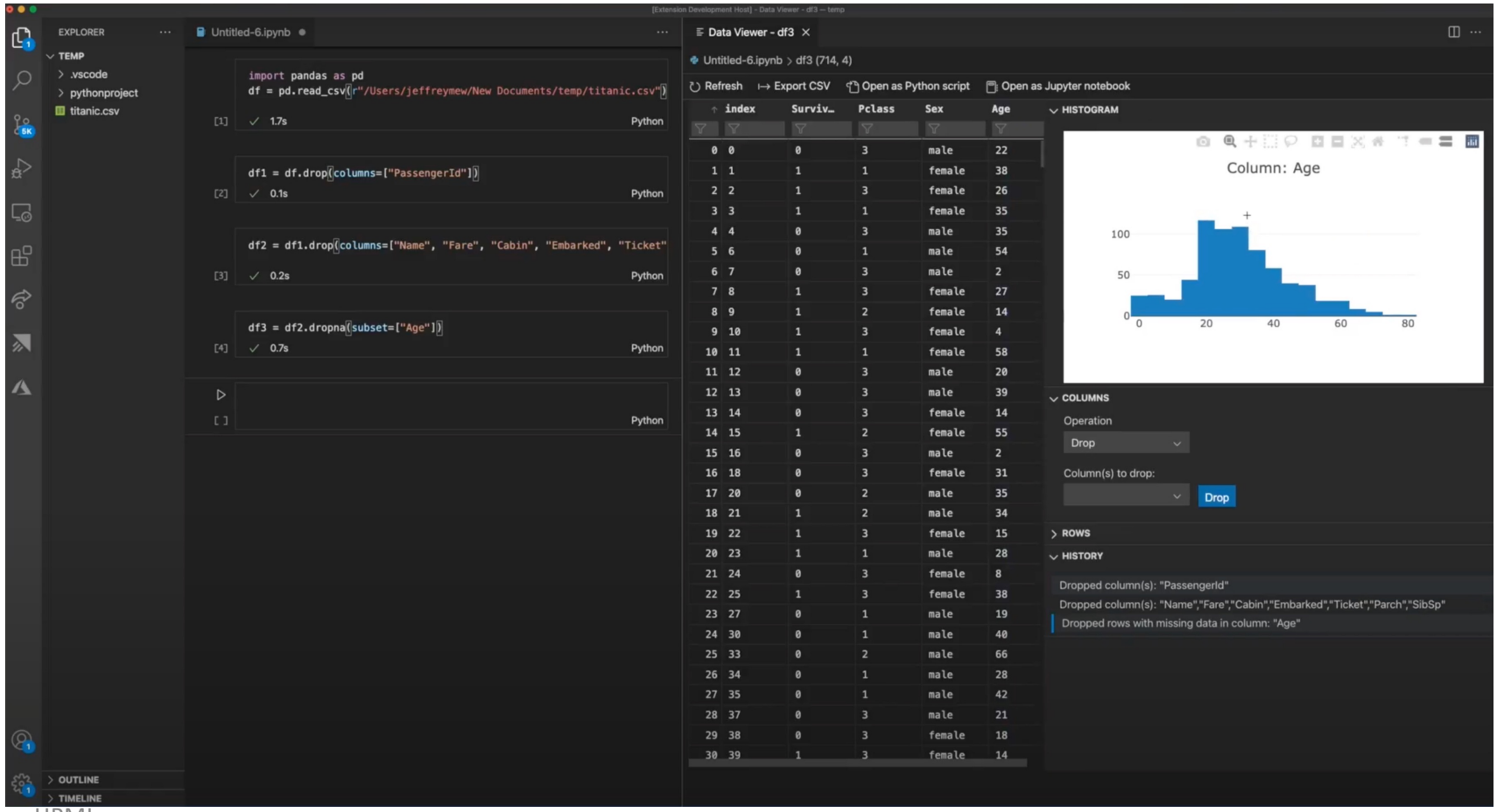



TensorBoard — PyTorch Demo
Welcome
TensorBoard
TensorBoard
SCALARS
GRAPHS
TIME SERIES
PYTORCH_PROFILER
INACTIVE
UPLOAD
Runs
profiler
Workers
jemew-pytorch-demo_3040.161895...
Views
Operator
14.8%
14.1%
13.4%
11.9%
10.9%
7.2%
7.2%
7.1%
6.7%
aten::mul_
aten::empty
aten::_cudnn_rnn_backward
aten::zero_
aten::fill_
aten::div
aten::addcmul_
aten::sqrt
Group By
Operator
Name
Calls
Device Self Duration (us)
Device Total Duration (us)
aten::_cudnn_rnn_backward 6 183142 185343
aten::_cudnn_rnn 6 92266 92472
aten::copy_ 54 3999 3999
aten::fill_ 120 1040 1040
aten::add_ 180 384 384
aten::add_ 60 123 123 9054
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/torch/optim/_functional.py(92): adam
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/torch/optim/adam.py(119): step
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/torch/autograd/grad_mode.py(27): decorate_
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/torch/optim/optimizer.py(89): wrapper
/Users/jeffreymew/New Documents/PyTorch Ecosystem Day/stonks-demo/helper.py(57): train
<ipython-input-40-fbee9e6c98ee>(1): <module>
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/IPython/core/interactiveshell.py(3418): run_co
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/IPython/core/interactiveshell.py(3338): run_as
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/IPython/core/interactiveshell.py(3147): run_co
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/IPython/core/async_helpers.py(68): _pseudo_
/anaconda/envs/py37_pytorch/lib/python3.7/site-packages/IPython/core/interactiveshell.py(2923): _run_
helper.py M
Users > jeffreymew > New Documents > PyTorch Ecosystem Day > stonks-demo > helper.py > train
hist = np.zeros(num_epochs)
start_time = time.time()
with torch.profiler.profile(
activities=[
torch.profiler.ProfilerActivity.CPU,
torch.profiler.ProfilerActivity.CUDA],
on_trace_ready=torch.profiler.tensorboard_trace_handler('./runs/profiler'),
with_stack=True,
record_shapes=True
) as p:
for t in range(num_epochs):
y_train_pred = model(x_train)
loss = criterion(y_train_pred, y_train)
print("Epoch ", t, "MSE: ", loss.item())
hist[t] = loss.item()
optimiser.zero_grad()
loss.backward()
optimiser.step()
writer.add_scalar(title, loss.item(), t)
p.step()
training_time = time.time()-start_time
def predict(model, x_test, y_test, scaler):
model.eval()
y_test_pred = model(x_test)
# invert predictions
use_cuda = torch.cuda.is_available()
if use_cuda:
y_test_pred = scaler.inverse_transform(y_test_pred.detach().cpu().numpy())
y_test = scaler.inverse_transform(y_test.detach().cpu().numpy())
else:
y_test_pred = scaler.inverse_transform(y_test_pred.detach().numpy())
y_test = scaler.inverse_transform(y_test.detach().numpy())
return y_test, y_test_pred
def plot_results(y_test, y_test_pred, df):
loc = plticker.MultipleLocator(base=25)
figure, axes = plt.subplots(figsize=(16, 7))
dates = df[['Date']][len(df)-len(y_test):].to_numpy().reshape(-1)
axes.plot(dates, y_test, color = 'red', label = 'Real MSFT Stock Price')
main*
Python 3.8.8 64-bit ('jumptosource': conda)
Live Share

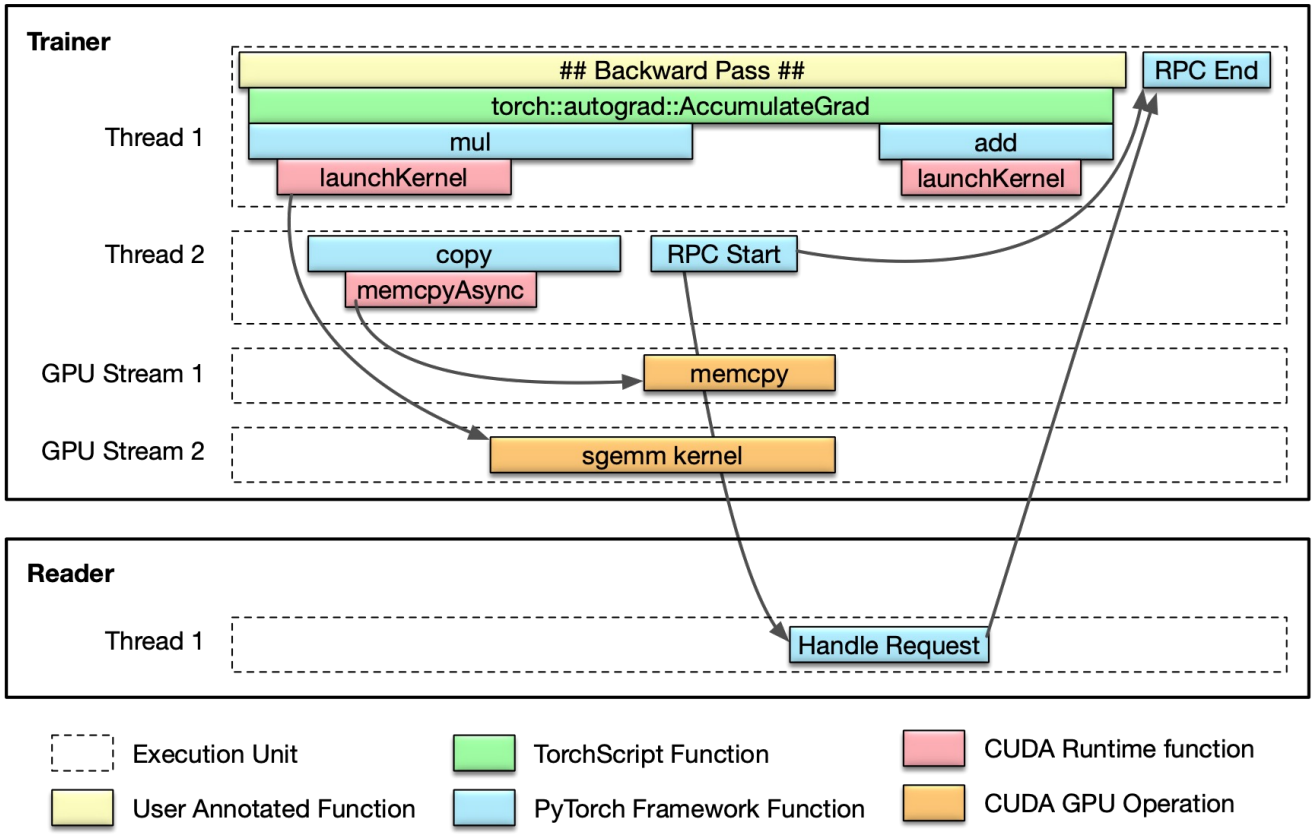
Distributed Training View



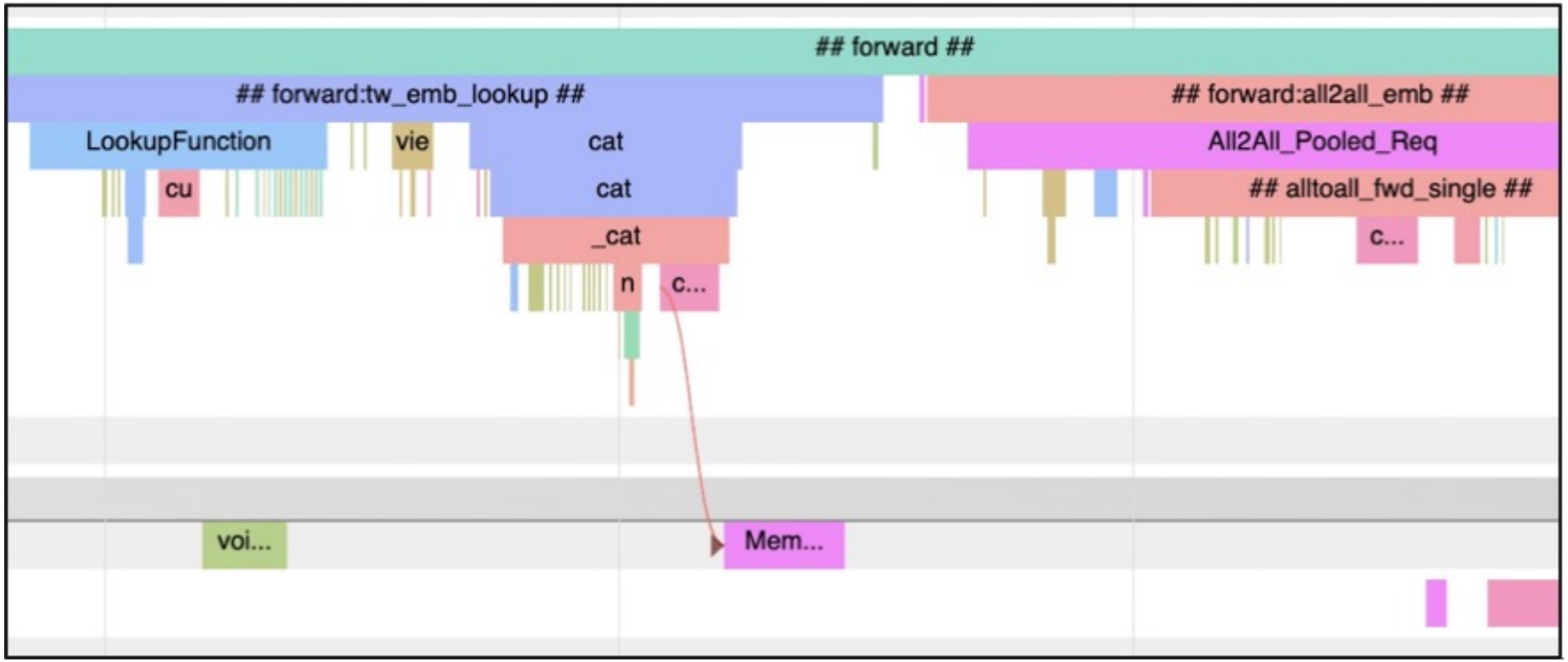
Vscode Integration: Data Wrangler



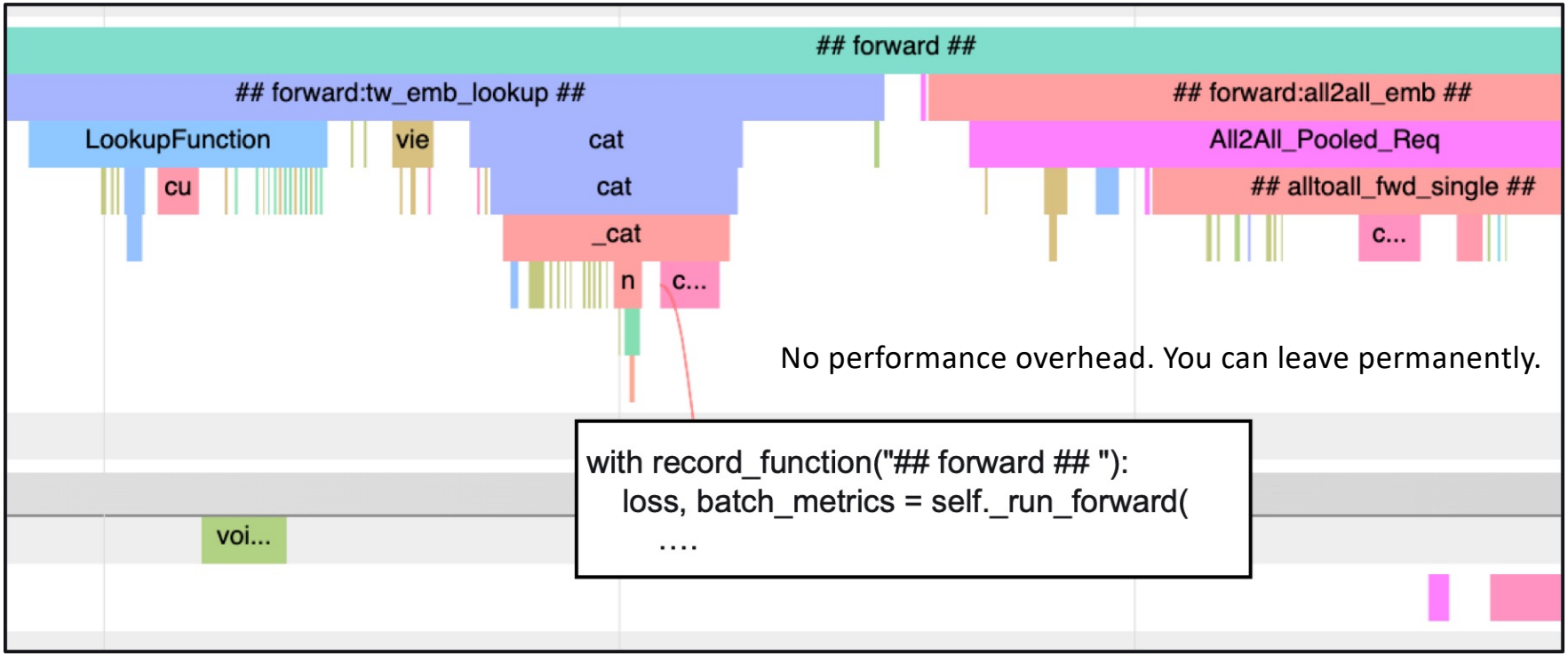
Timeline tracing: CPU and GPU Activities



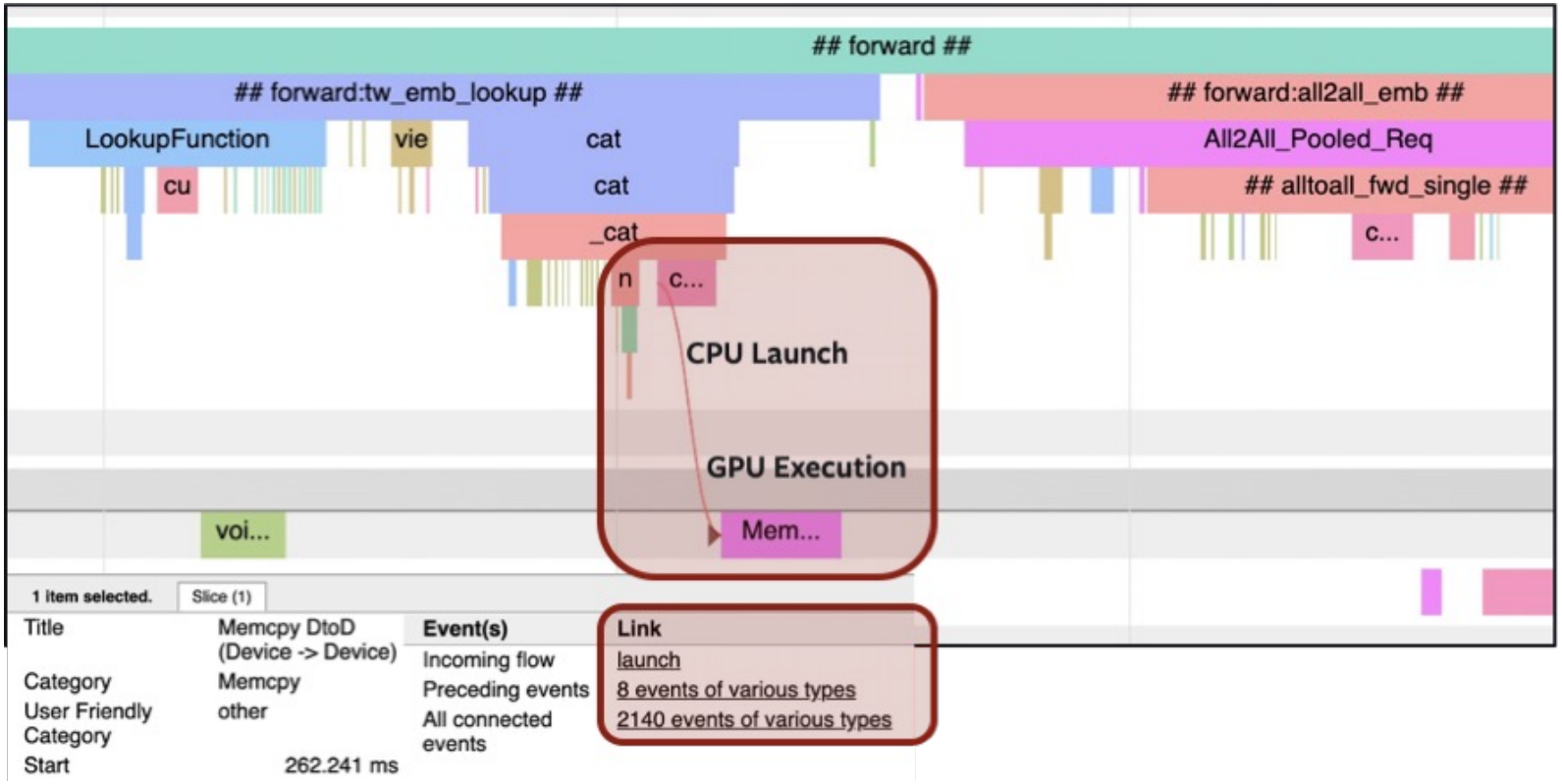
Timeline Tracing:
Chrome Trace Viewer- CPU and GPU timelines



Timeline Tracing:



You can see how CPU and GPU ops are connected

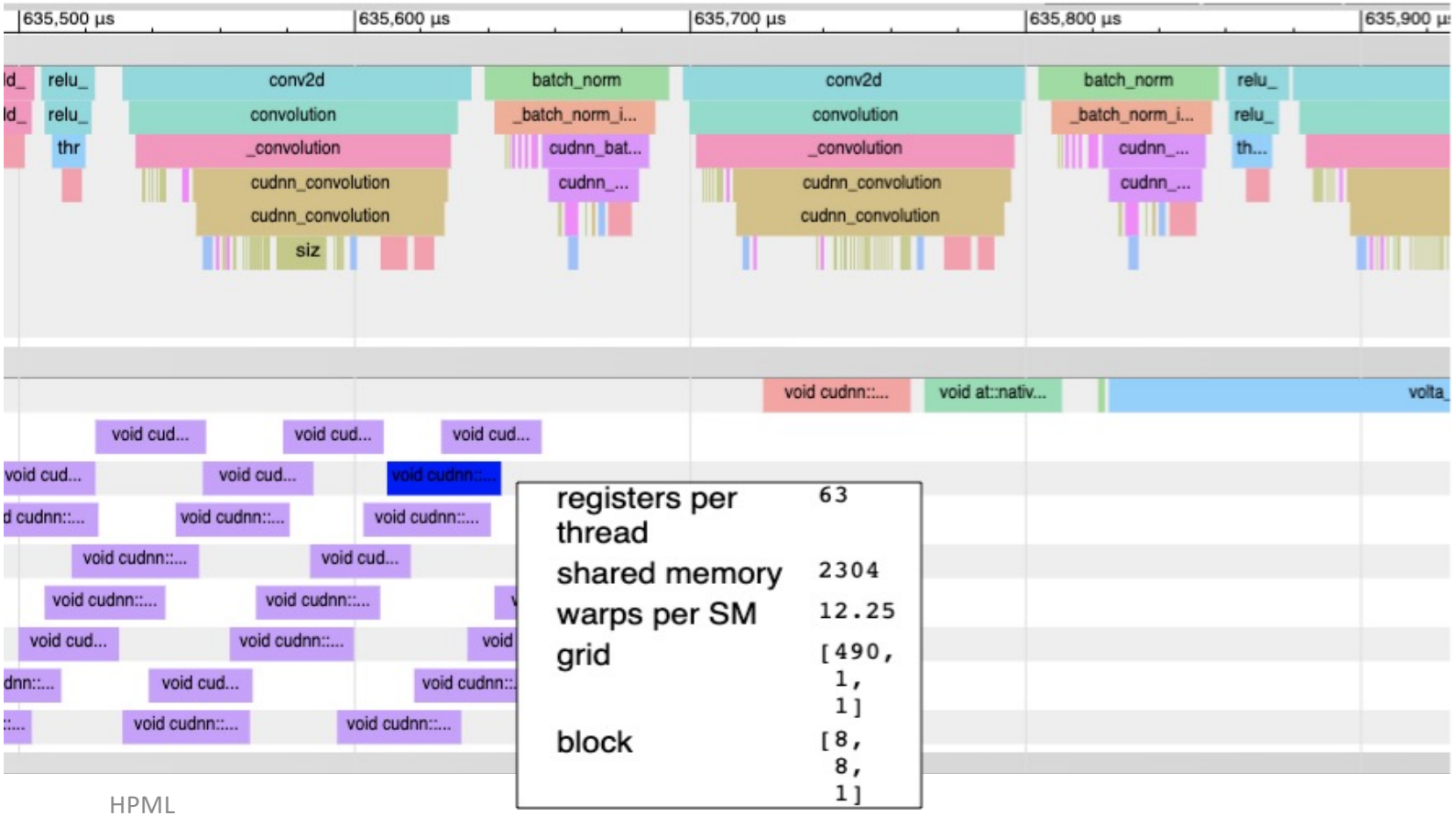


Timeline Tracing:
Inspect stats to individual activities

Nvidia-smi shows 86% utilization

But.. only a fraction of Streaming Multiprocessor are actually used by these kernels!

Timeline Tracing:
Inspect stats to individual activities



Much better utilization after increasing input sizes

Trace Analysis Examples with PyTorch Profiler

Thanks to Facebook Engineers for examples:
Lei Tian, Natalia Gimelshein, Lingyi Liu, Feng Shi & Zhicheng Yan

Anti-pattern: Long GPU idle time

Issue:

1. Large periods of GPU inactivity
2. Trace does not show why

Solution:

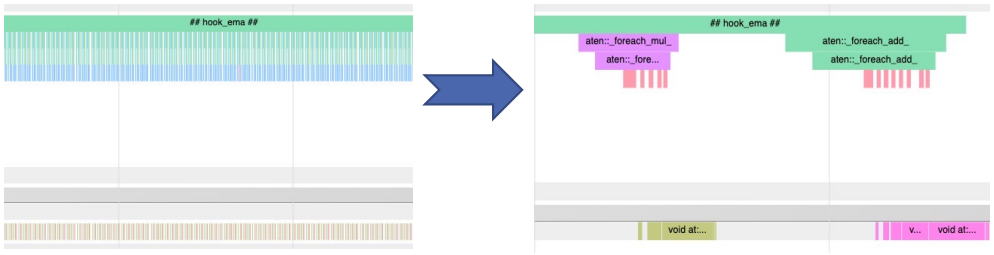
1. Use record_function to reveal bottlenecks on CPU
2. Parallelize CPU operations
3. Overlap CPU and GPU operations

```
temp = ""
num_substr = len(emb[k])

with record_function("## join_string {} ##".format(num_substr)):
    temp = ",".join(str(x) for x in emb[k]) # string concatenation

with record_function("## append_record_in_else ##"):
    records.append(f"{input_df.id[i + k]}\t{temp}\n") # list append
```

Anti-pattern: Excessive CPU/GPU Interactions

First issue:

- Exponential moving avg hook function has a loop – CPU bottleneck
- Can rewrite using torch._foreach ops – loop now on GPU




```
BEFORE
def on_step(self, task) -> None:
   ...
   with torch.no_grad():
      it = model_state_iterator(task.base_model)
      # iterate on every name & param
      for name, param in it:
         s = self.state.ema_model_state
         s[name] = self.decay * s[name] +
            (1 - self.decay) *
            param.to(device= self.device)


AFTER
def on_step(self, task) -> None:
   ...
   with torch.no_grad():
      torch._foreach_mul_(
         self.ema_model_state_list, self.decay)
      torch._foreach_add_(
         self.ema_model_state_list,
         self.param_list,
         alpha=(1 - self.decay))
```

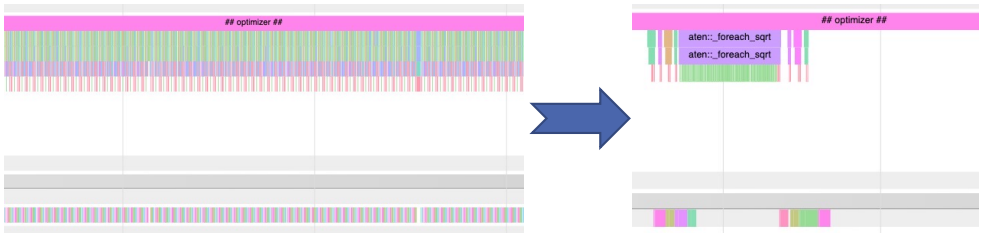
EMA HOOK 100X FASTER

ITERATION TIME: 860MS -> 770MS

Anti-pattern: Long GPU idle time

Second issue:

- Optimizer step uses a naïve implementation of RMSProp
- PyTorch provides an optimized multi-tensor version – using torch._foreach
- Switch to optimized version!




```
BEFORE
def prepare(self, param_groups):
   self.optimizer = RMSpropTFV2Optimizer(
      param_groups,
      …


AFTER
import torch.optim._multi_tensor as optim_mt
def prepare(self, param_groups):
   self.optimizer = optim_mt.RMSprop(
      param_groups,
      …
```

OPTIMIZER 12X FASTER

ITERATION TIME: 770MS -> 600MS

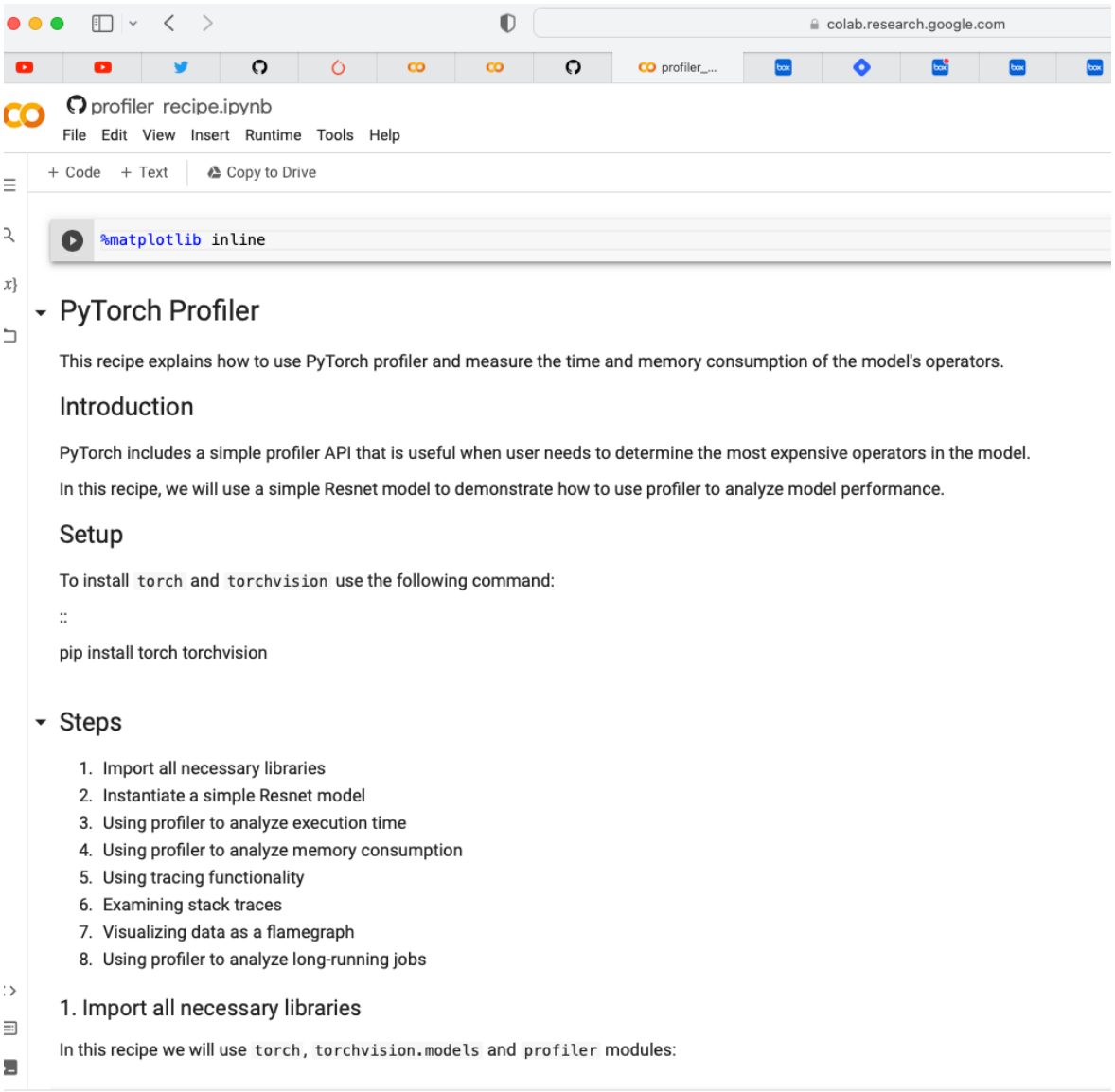
BERT Performance Optimization Case Study

- From 2.4 req/s to 1,400+ req/s
- CPU Inference
  - torch.set_num_threads(1)
  - Inel IPEX
  - Quantization
- GPU Inference on 1 T4 GPU
  - model.half()
  - DistiBERT
  - Increase batch size
  - Do not overpad
  - Faster Transformer

	Throughput	P99
BERT unoptimized bs=1	70.67 seq/s	20.44ms
BERT model.half() bs=8	359 seq/s	23.58ms
DistilBERT model.half() bs=16	689 seq/s	22.8ms
BERT Faster Transformer	885 seq/s	19.83ms
DistilBERT no padding model.half() bs=32	1423 seq/s	19.7ms

Example from PyTorch Documentation

https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html



Another Example from PyTorch Documentation with Tensorboard integration

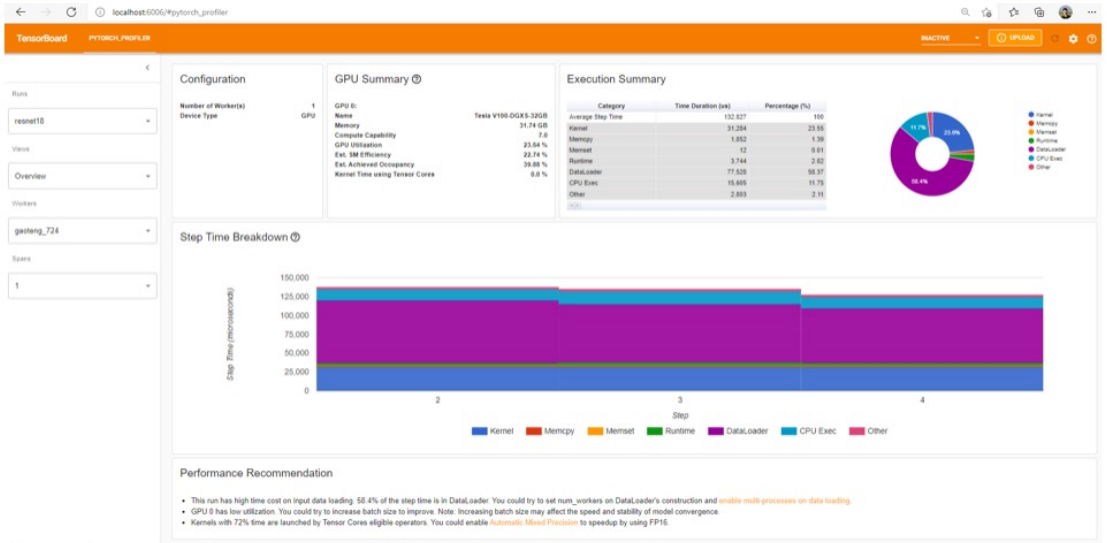
https://pytorch.org/tutorials/intermediate/tensorboard_profiler_tutorial.html

Tutorials > PyTorch Profiler With TensorBoard

```
http://localhost:6006/#pytorch_profiler
```

You could see Profiler plugin page as shown below.

- Overview



The overview shows a high-level summary of model performance.

The “GPU Summary” panel shows the GPU configuration, GPU usage and Tensor Cores usage. In this example, the GPU Utilization is low. The details of these metrics are here.

The “Step Time Breakdown” shows distribution of time spent in each step over different categories of execution. In this example, you can see the `DataLoader` overhead is significant.

The bottom “Performance Recommendation” uses the profiling data to automatically highlight likely bottlenecks, and gives you

Other Profiling Tools

Using *profile/cProfile*

- From your program:
  - Example profiling a regular expression

```
import cProfile
import re
cProfile.run('re.compile("foo|bar")')
```

- To profile a script:

```
python -m cProfile [-o output_file] [-s sort_order] myscript.py
```

- By default a summary is provided, using *pstats*
- By specifying an *output_file* the profile can be processed afterwards
- https://docs.python.org/3/library/profile.html

Profiling a PyTorch neural network

- Consider this NN example: a two-layers network with ReLU activation

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out),
)

loss_fn = torch.nn.MSELoss(size_average=False)
learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)

    loss = loss_fn(y_pred, y)
    print(t, loss.data[0])

    model.zero_grad()

    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
def main():
    x, y, model, loss_fn = setup(N, D_in, H, D_out)
    learn(x, y, model, loss_fn)

if __name__ == "__main__":
    main()
```

Profiling a PyTorch neural network

- Profile (partial) collected with:

```
python -m cProfile -s time nn.py
```

- Output:

```
326044 function calls (313968 primitive calls) in 1.073 seconds

   Ordered by: internal time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      500    0.249    0.000    0.249    0.000 {method 'run_backward' of 'torch._C._EngineBase' objects}
     1000    0.189    0.000    0.189    0.000 {built-in method torch._C.addmm}
    27/26    0.081    0.003    0.084    0.003 {built-in method _imp.create_dynamic}
      261    0.054    0.000    0.057    0.000 <frozen importlib._bootstrap_external>:830(get_data)
     1386    0.037    0.000    0.037    0.000 {built-in method posix.stat}
        3    0.026    0.009    0.063    0.021 utils.py:61(parse_header)
      261    0.025    0.000    0.025    0.000 {built-in method marshal.loads}
12000/5000   0.024    0.000    0.039    0.000 module.py:513(named_parameters)
     2000    0.023    0.000    0.023    0.000 {method 'mul' of 'torch._C.FloatTensorBase' objects}
  801/798    0.021    0.000    0.033    0.000 {built-in method builtins.__build_class__}
        1    0.021    0.021    1.073    1.073 nn.py:1(<module>)
```

Snakeviz

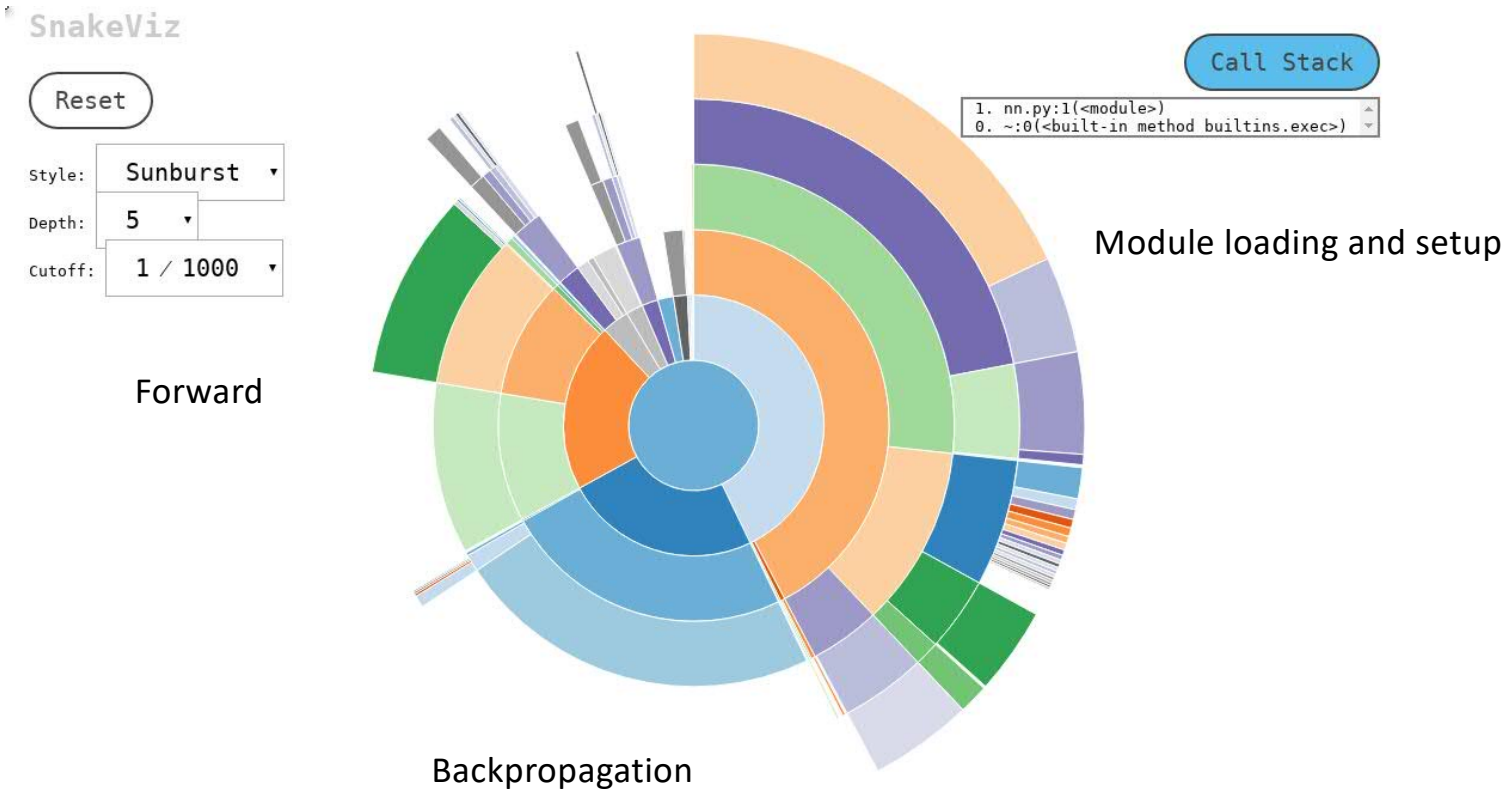
- Graphical tool to visualize content of profile created with cProfile
- Need to use the profile output file on your laptop
- Usage:

```
$ snakeviz nn.profile --server
snakeviz web server started on 127.0.0.1:8080; enter Ctrl-C to exit
http://127.0.0.1:8080/snakeviz/%2Fcygdrive%2Fc%2FUsers%2FAlessandroMORARI%2FBox+Sync%2Fwork%2Fcygwin%2FAlessandroMORARI%2Fnn.profile
```

- Then open the browser and connect to URL provided….

- https://jiffyclub.github.io/snakeviz/

SnakeViz and sunburst plots

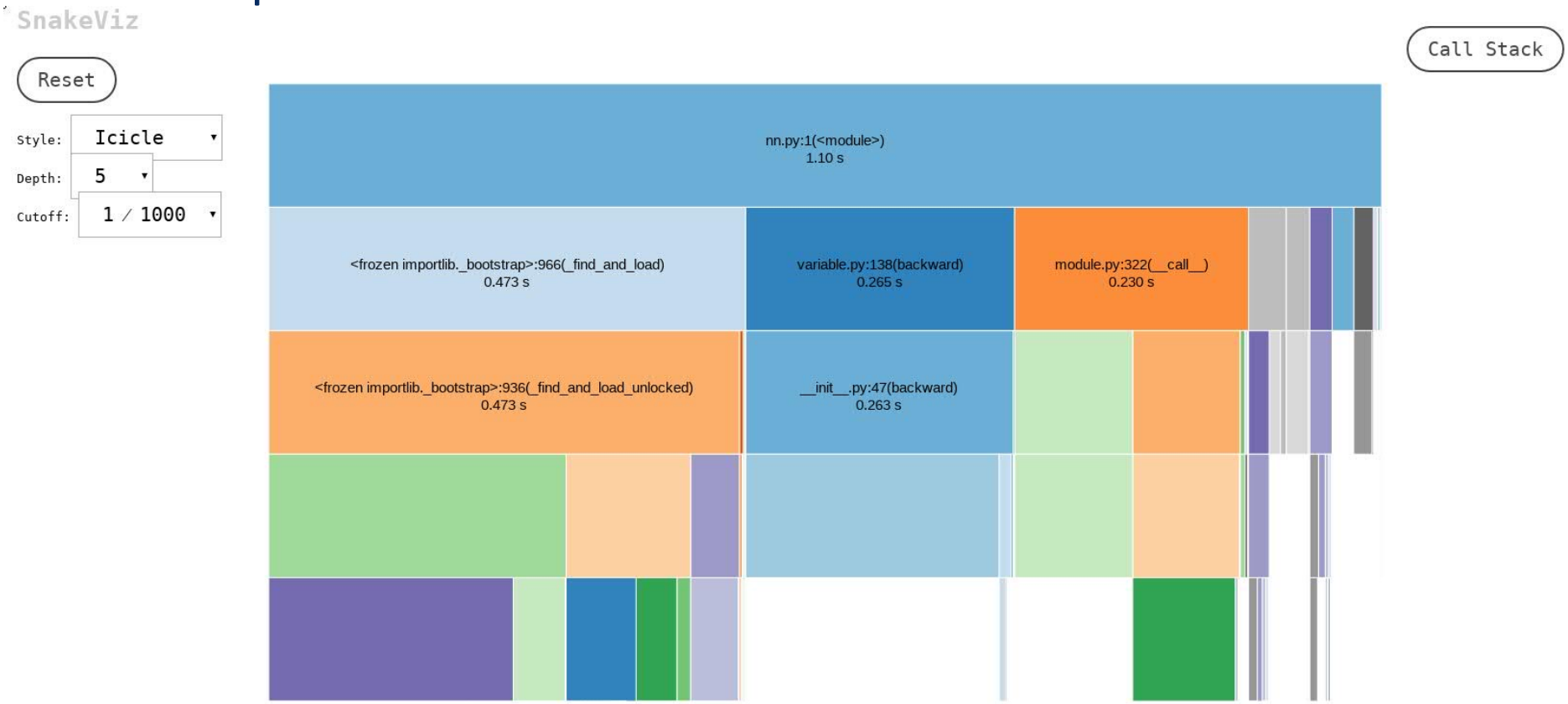


Forward

Module loading and setup

Backpropagation

Icicle plots



Profiling memory usage

- Important to determine whether:
  - Allocations can be avoided in the critical paths (e.g. by reusing allocated memory)
  - The overall memory usage is too high and limits the problem size that can be solved
- https://pypi.python.org/pypi/memory_profiler
- Usage:
  - `python –m memory_profiler nn.py`

Example

- In the following example, we create a simple function `my_func` that allocates lists `a`, `b` and then deletes `b`:

```
@profile
def my_func():
    a = [1] * (10 ** 6)
    b = [2] * (2 * 10 ** 7)
    del b
    return a

if __name__ == '__main__':
    my_func()
```

```
$ python -m memory_profiler example.py
```

Example

- Execute the code passing the option `-m memory_profiler` to the python interpreter to load the memory_profiler module and print to stdout the line-by-line analysis. If the file name was example.py

```
$ python -m memory_profiler example.py
```

- this would result in:

```
Line #    Mem usage    Increment  Occurrences   Line Contents
=============================================================
     3   38.816 MiB   38.816 MiB           1   @profile
     4                                         def my_func():
     5   46.492 MiB    7.676 MiB           1       a = [1] * (10 ** 6)
     6  199.117 MiB  152.625 MiB           1       b = [2] * (2 * 10 ** 7)
     7   46.629 MiB -152.488 MiB           1       del b
     8   46.629 MiB    0.000 MiB           1       return a
```

1st column: line number of the code that has been profiled

2nd column: memory usage of the Python interpreter after that line has been executed

3rd column: difference in memory of the current line with respect to the last one

Another Example: Output of *memory_profiler*

```
Filename: nn.py

Line #    Mem usage    Increment   Line Contents
================================================
    20   85.031 MiB   85.031 MiB   @profile
    21                             def learn(x, y, model, loss_fn):
    22   85.031 MiB    0.000 MiB       learning_rate = 1e-4
    23   91.242 MiB    0.000 MiB       for t in range(500):
    24   91.242 MiB    2.184 MiB           y_pred = model(x)
    25
    26   91.242 MiB    0.047 MiB           loss = loss_fn(y_pred, y)
    27   91.242 MiB    0.164 MiB           print(t, loss.data[0])
    28
    29   91.242 MiB    0.000 MiB           model.zero_grad()
    30
    31   91.242 MiB    3.242 MiB           loss.backward()
    32
    33   91.242 MiB    0.000 MiB           for param in model.parameters():
    34   91.242 MiB    0.574 MiB               param.data -= learning_rate * param.grad.data
```

PyTorch Performance - Benchmarking

Profiling vs. benchmarking

- In benchmarking we're interested in assessing the **absolute speed** of a piece of code
- Profiling results are not reliable for absolute values
  - Profiling introduces **overhead**
  - C++ and Python sections of the application are affected by profiling in a different way, depending on the profiling tools being used
- When benchmarking, variability has to be taken into account:
  - Dependencies on the input
  - Dependencies on temporary conditions
    - Always collect stats on multiple executions

Benchmarking: the *timeit* module

- The *timeit* module deals with many of the requirements of benchmarking
- Execute the code in a loop, and take the best of multiple runs
- Using from the command line
  - example (timing a matrix multiply in numpy, 5 runs of 20 iterations each):

```
% python3 -m timeit -v -n 20 -r 5 -s "import numpy; x=numpy.random.rand(1000, 1000)" "x=x.dot(x)"

raw times: 210 msec, 217 msec, 184 msec, 199 msec, 211 msec

20 loops, best of 5: 9.19 msec per loop
```

The *timeit* module

- From Python code:

```
import timeit

t = timeit.Timer("x = x.dot(x)",
                 setup = "import numpy; x = numpy.random.rand(1000, 1000)").repeat(number=20, repeat=10)

print("raw times: {0}\nbest of 5: {1:.0f} ms".format(" ".join([ "%.3f" % x for x in t ]), min(t)/20*1000))
```

  - Output:

```
% python3 test_timit.py
  raw times: 0.209 0.228 0.183 0.170 0.179 0.174 0.211 0.191 0.170 0.169
  best of 10: 8 ms
```

Lesson Key Points

- Python performance:
  - Interpreter inner workings
  - Memory Management
  - Typing
- PyTorch performance
  - Computation Graph Approach
  - Just In Time Compilation
  - Torch.compile
  - Profiling
  - Benchmarking

PyTorch Examples

Autograd Example (1)

```
Import torch

dtype = torch.float
device = torch.device("cpu") # Use the CPU as device
# Alternatively, use device = torch.device("cuda:0") to run on GPU

# N is batch size; D_in is input dimension;
# H is hidden dimension; D_out is output dimension.
N, D_in, H, D_out = 64, 1000, 100, 10

# Create random Tensors to hold input and outputs.
# Setting requires_grad=False indicates that we do not need to compute gradients
# with respect to these Tensors during the backward pass.
x = torch.randn(N, D_in, device=device, dtype=dtype)
y = torch.randn(N, D_out, device=device, dtype=dtype)

# Create random Tensors for weights.
# Setting requires_grad=True indicates that we want to compute gradients with
# respect to these Tensors during the backward pass.
w1 = torch.randn(D_in, H, device=device, dtype=dtype, requires_grad=True)
w2 = torch.randn(H, D_out, device=device, dtype=dtype, requires_grad=True)
```

From:
http://pytorch.org/tutorials/beginner/pytorch_with_examples.html

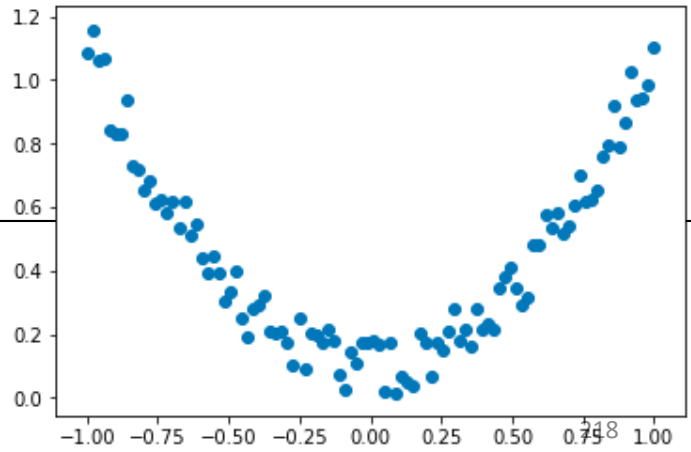
Autograd Example (2)

```
learning_rate = 1e-6
for t in range(500):
    # Forward pass: compute predicted y using operations on Variables;
    # mm: matrix multiply; clamp: clamp in [min;max]
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    # Compute and print loss. Now loss is a Tensor of shape (1,)
    # loss.item() is a scalar value holding the loss.
    loss = (y_pred - y).pow(2).sum()
    print(t, loss.item())
    # Use autograd to compute the backward pass.
    loss.backward()
    # Update weights using gradient descent; w1.data and w2.data are Tensors,
    # w1.grad and w2.grad are Variables and w1.grad.data and w2.grad.data are Tensors.
    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        w2 -= learning_rate * w2.grad
        # Manually zero the gradients after updating weights
        w1.grad.zero_()
        w2.grad.zero_()
```

From: http://pytorch.org/tutorials/beginner/pytorch_with_examples.html

Linear Regression Example (1)

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

x = torch.unsqueeze(torch.linspace(-1, 1, 100), dim=1)  # x data (tensor), shape=(100, 1)
#unsqueeze: reshape tensor
#linspace: return a one-dimensional vector of 100 points between -1 and 1
y = x.pow(2) + 0.2*torch.rand(x.size())                 # noisy y data (tensor), shape=(100, 1)

plt.scatter(x.numpy(), y.numpy())
plt.show()
```



https://github.com/MorvanZhou/PyTorch-Tutorial

Linear Regression Example (2)

```
class Net(torch.nn.Module):
    def __init__(self, n_feature, n_hidden, n_output):
        super(Net, self).__init__()
        self.hidden = torch.nn.Linear(n_feature, n_hidden)   # hidden layer
        self.predict = torch.nn.Linear(n_hidden, n_output)   # output layer

    # For the forward() method, we supply the input data x as the primary argument.
    # Then we pass through the two layers of our simple network
    def forward(self, x):
        x = F.relu(self.hidden(x))      # activation function for hidden layer
        x = self.predict(x)             # linear output
        return x
```

https://github.com/MorvanZhou/PyTorch-Tutorial

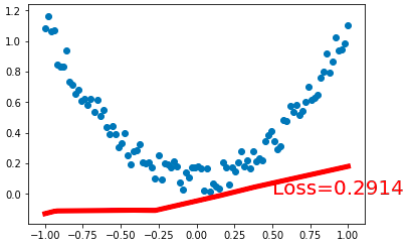
Linear Regression Example (3)

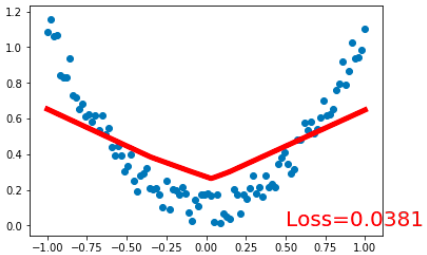
```
# create the network
net = Net(n_feature=1, n_hidden=10, n_output=1)     # define the network
print(net)  # net architecture

# create a stochastic gradient descent optimizer
# the method net.parameters() (from the base nn.Module class that we inherit in the Net class), contains all the parameters of our network
optimizer = torch.optim.SGD(net.parameters(), lr=0.2)
# define the loss, we use the regression mean squared loss
loss_func = torch.nn.MSELoss()

plt.ion()   # turn on interactive mode
for t in range(200):
    prediction = net(x)     # input x and predict based on x
    loss = loss_func(prediction, y)     # must be (1. nn output, 2. target)
    # With optimizer.zero_grad() we resets all the gradients in the model
    # so that it is ready to go for the next back propagation pass.
    optimizer.zero_grad()   # clear gradients for next train
    loss.backward()         # backpropagation, compute gradients
    # runs a back-propagation operation from the loss Tensor backwards
    # through the network and execute a gradient descent step based on
    # the gradients calculated during the .backward() operation.
    optimizer.step()        # apply gradients
```

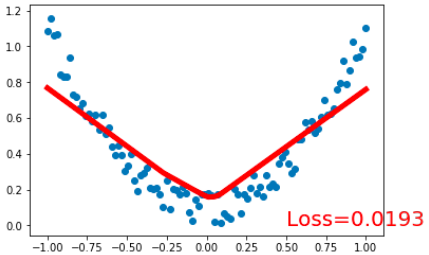
https://github.com/MorvanZhou/PyTorch-Tutorial

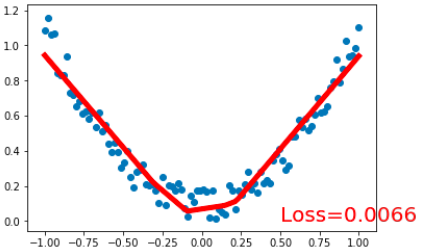
Linear Regression Example (4)

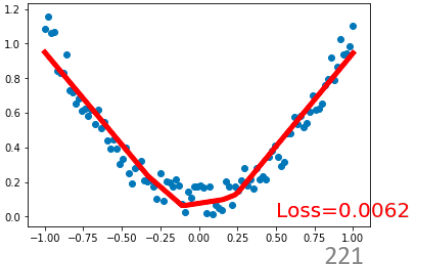
```
        # plot every 5 epochs the learning process
        if t % 5 == 0:
            # plot and show learning process
            plt.cla()
            plt.scatter(x.data.numpy(), y.data.numpy())
            plt.plot(x.data.numpy(), prediction.data.numpy(), 'r-', lw=5)
            plt.text(0.5, 0, 'Loss=%.4f' % loss.data.numpy(), fontdict={'size': 20, 'color':  'red'})
            plt.pause(0.1)
plt.ioff()
plt.show()
```











https://github.com/MorvanZhou/PyTorch-Tutorial

…….

NN Example (1)

```
import torch

# N is batch size; D_in is input dimension;
# H is hidden dimension; D_out is output dimension.
N, D_in, H, D_out = 64, 1000, 100, 10

# Create random Tensors to hold inputs and outputs
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)

# Use the nn package to define our model as a sequence of layers. nn.Sequential
# is a Module which contains other Modules, and applies them in sequence to
# produce its output. Each Linear Module computes output from input using a
# linear function, and holds internal Tensors for its weight and bias.
model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out),
)

# The nn package also contains definitions of popular loss functions; in this
# case we will use Mean Squared Error (MSE) as our loss function.
loss_fn = torch.nn.MSELoss(reduction='sum')
```

From: http://pytorch.org/tutorials/beginner/pytorch_with_examples.html

NN Example (2)

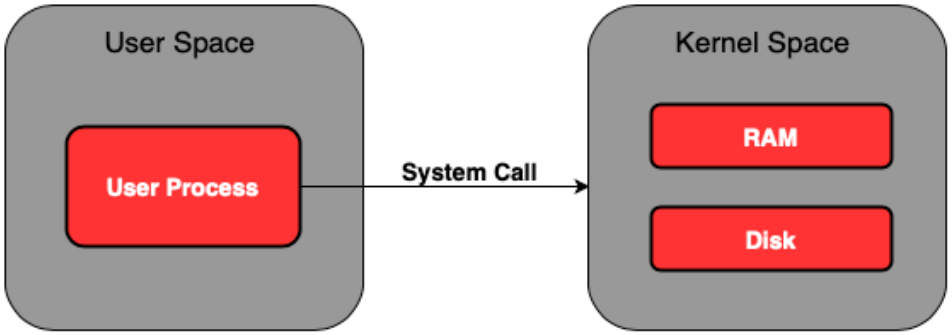
From: http://pytorch.org/tutorials/beginner/pytorch_with_examples.html

```
learning_rate = 1e-4
for t in range(500):
    # Forward pass: compute predicted y by passing x to the model. Module objects
    # override the __call__ operator so you can call them like functions. When
    # doing so you pass a Tensor of input data to the Module and it produces
    # a Tensor of output data.
    y_pred = model(x)
    # Compute and print loss. We pass Tensors containing the predicted and true
    # values of y, and the loss function returns a Tensor containing the loss.
    loss = loss_fn(y_pred, y)
    print(t, loss.item())
    # Zero the gradients before running the backward pass.
    model.zero_grad()
    # Backward pass: compute gradient of the loss with respect to all the learnable
    # parameters of the model. Internally, the parameters of each Module are stored
    # in Tensors with requires_grad=True, so this call will compute gradients for
    # all learnable parameters in the model.
    loss.backward()
    # Update the weights using gradient descent. Each parameter is a Tensor, so
    # we can access its gradients like we did before.
    with torch.no_grad():
        for param in model.parameters():
            param -= learning_rate * param.grad
```

Appendix

Memory Management Concepts

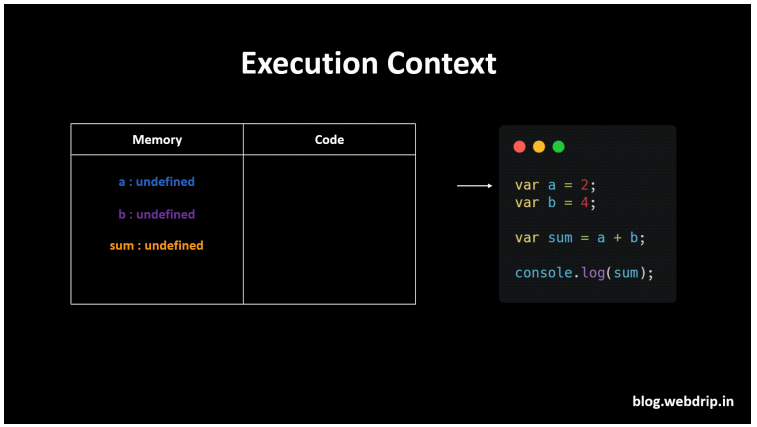
- **Kernel Space**
  - The kernel space can be accessed by user processes only using system calls that are requests in a Unix-like operating system such as input/output (I/O) or process creation.
- **User Space**
  - The user space is a computational resource allocated to a user, and it is a resource that the executing program can directly access. This space can be categorized into some segments.



What is the Program Execution Context?

The **execution context** of a program consists of:

1. **Process ID (PID)** – Unique identifier for the running process.
2. **Registers** – CPU stores values like program counter (PC), stack pointer (SP), and general-purpose registers.
3. **Memory Segments**:
   - **Code Segment** – Stores program instructions.
   - **Data Segment** – Stores global/static variables.
   - **Heap** – Dynamically allocated memory (e.g., malloc in C, new in C++).
   - **Stack** – Stores function calls, local variables, and return addresses.
4. **File Descriptors** – References to open files, sockets, and devices.
5. **Scheduling Information** – Determines when and how the OS executes the process.
6. **Privileges & Security Context** – Defines access rights (e.g., root vs. regular user).



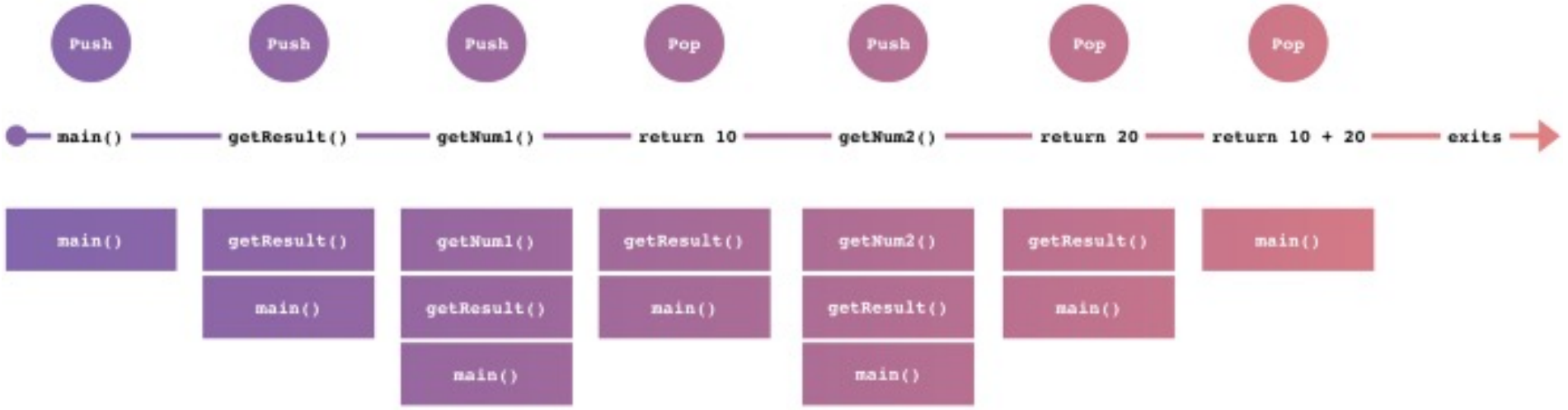
Example

The data pushed for a function call is named a **stack frame**, and it contains the following data.

- The arguments (parameter values) passed to the routine
- The return address back to the routine's caller
- Space for the local variables of the routine

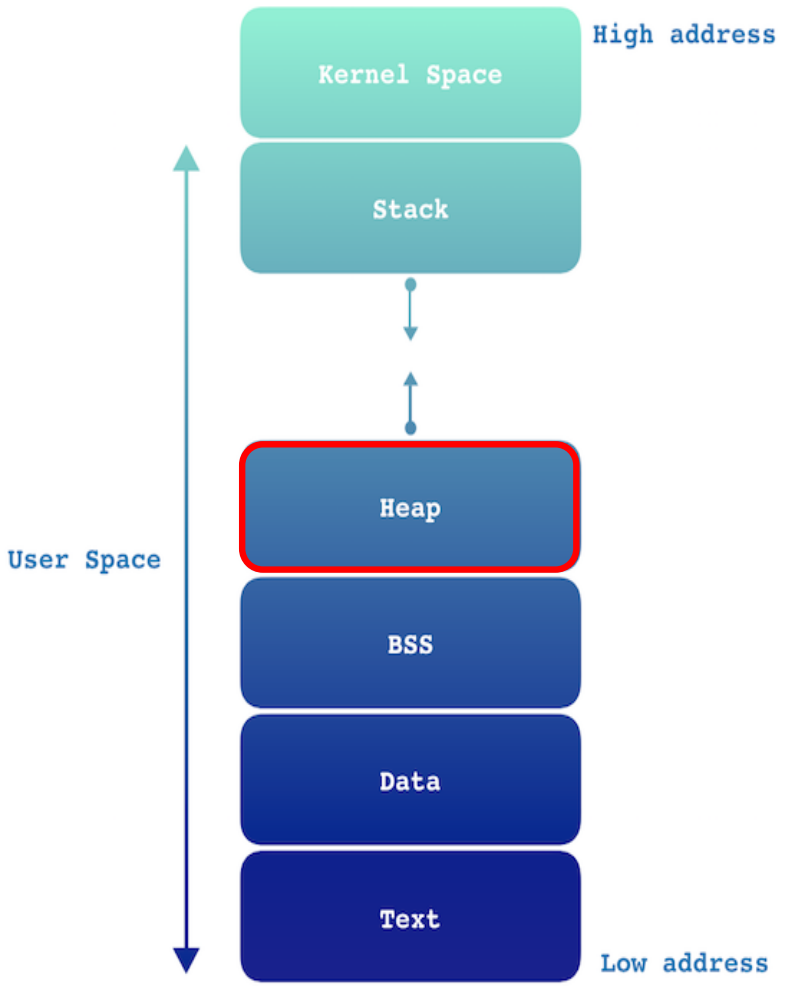
- When the function is called, the stack frame is pushed to the top of stack. Then the process is executed, and the function goes out of scope, the stack frame pops from the top.

```
int main() {
  int result = getResult();
}

int getResult() {
  int num1 = getNum1();
  int num2 = getNum2();
  return num1 + num2;
}

int getNum1() {
  return 10;
}

int getNum2() {
  return 20;
}
```



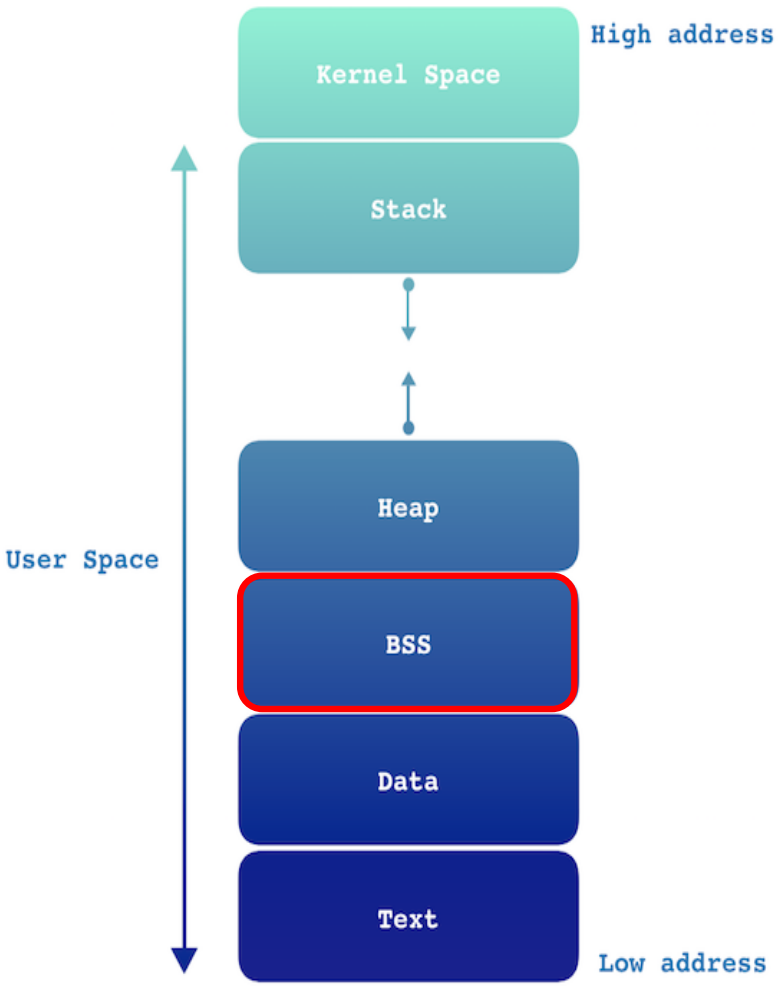
Memory Layout in a Program

- **Heap**
  - Segment where dynamic memory allocation usually takes place
  - In C, it's managed by malloc / new, free / delete
  - The allocation to the heap area occurs in the following cases:
    - Memory size is dynamically allocated at run-time
    - Scope is not limited. (i.e., variables referenced from several places)
  - Memory size is very large
  - The objects on the heap lead to memory leaks if they are not freed
  - In garbage-collected languages, the garbage collector frees memory on the heap and prevents memory leaks.
  - Application heap is not fixed
  - **Performance is relatively low**.



Memory Layout in a Program

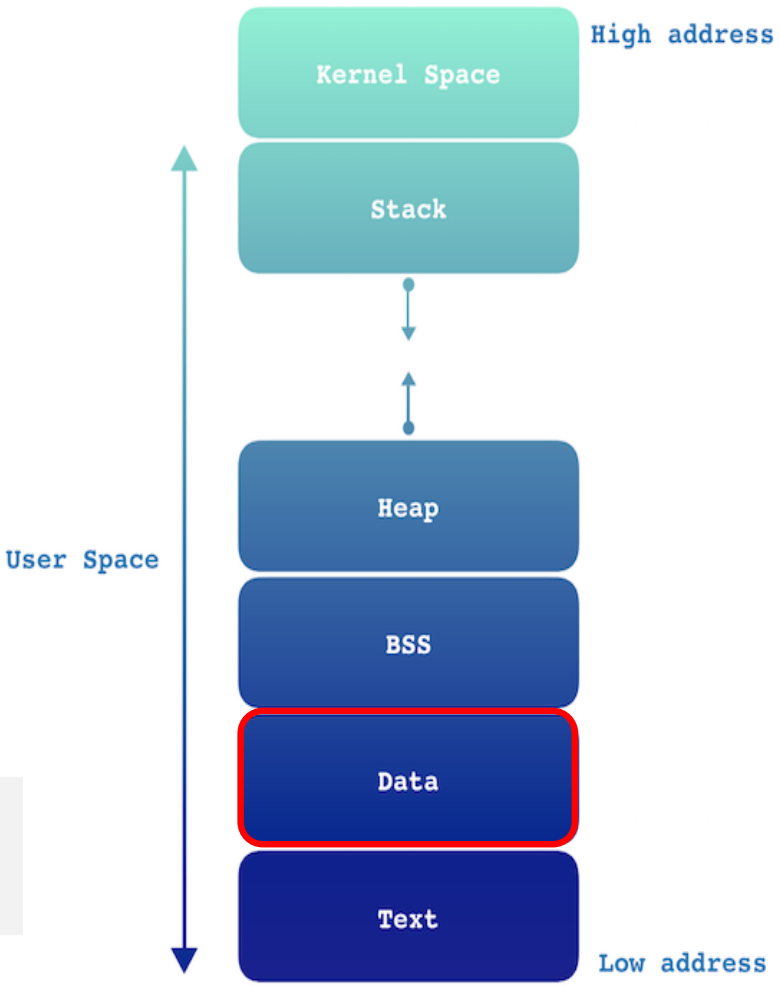
- **BSS (Block Started by Symbol)**
  - The uninitialized data segment is often called the BSS segment.
  - Data in this segment is initialized by the kernel to arithmetic 0 before the program starts executing.
  - Example: a variable declared as static int i; would be allocated to the BSS segment.
  - Both BSS and Data usually refer to RAM objects.



Memory Layout in a Program

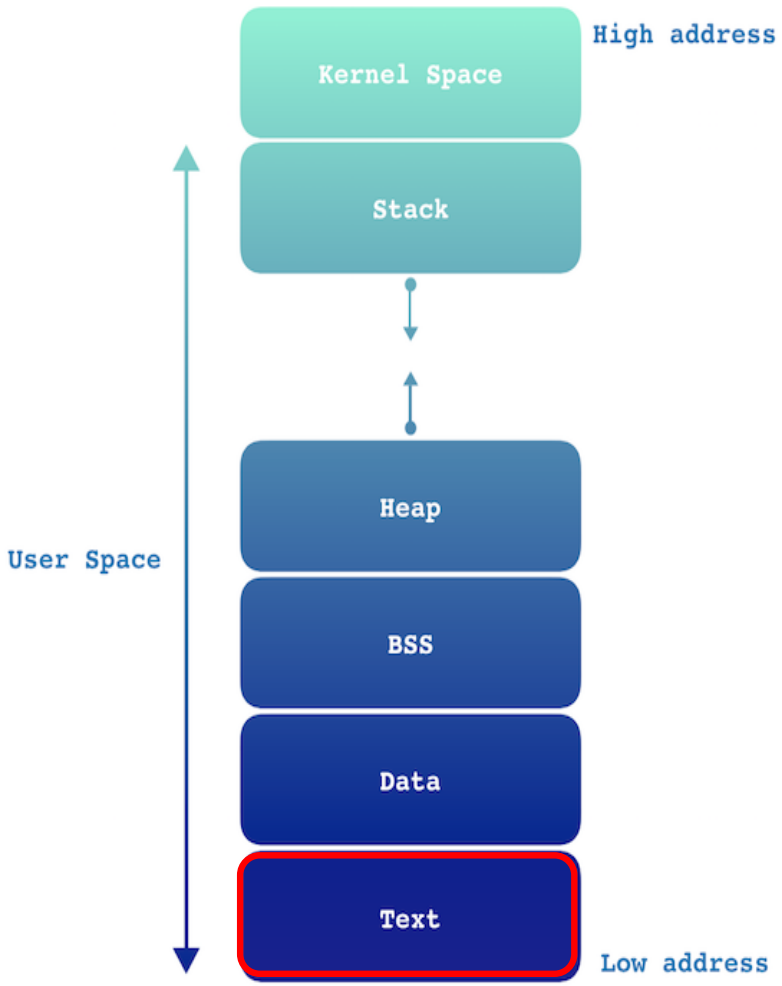
- **Data**
  - The data segment contains initialized global and static variables which have a pre-defined value and can be modified.
  - Divided into a read-only and a read-write space.
  - For example, the following C program outside the main

```
int val = 3;
char string[] = "Hello World";
```



Memory Layout in a Program

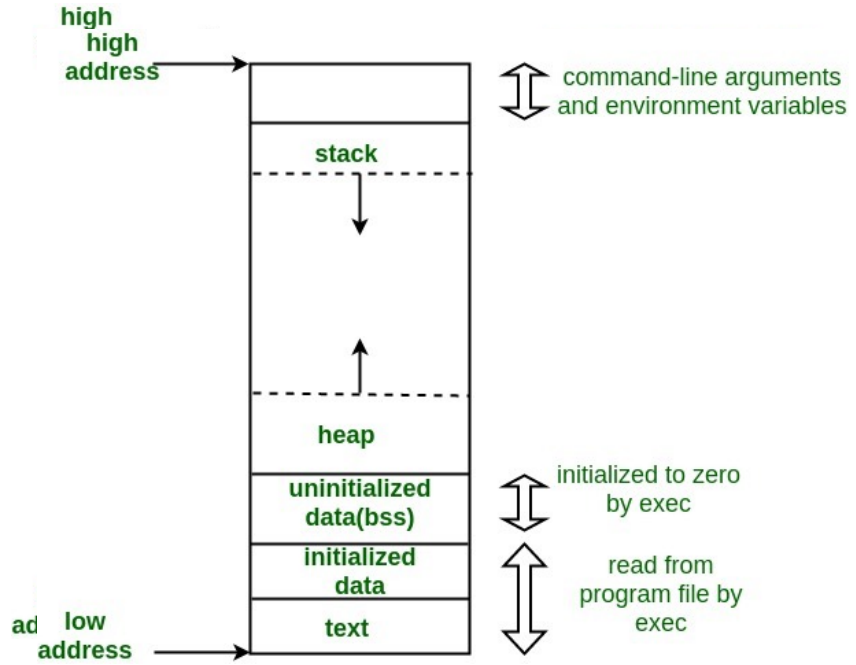
- **Text**
  - A segment in which a machine language instruction is stored.
  - This segment is a read-only space.



Memory Management

- Process' **stack**:
  - Automatic memory management by OS and Compiler
- Process' **heap**:
  - **Manual Memory management (ex. C)**
  - **Automatic Memory Management (ex. Python)**

- Thread's **stack:**
  - Resides in parent process' **heap**
  - Automatic mem. management by the thread library
- Thread's **heap**:
  - Manual Mem. Management (ex. C)
  - Automatic Mem. Management (ex. Python)



Process' memory layout

From: https://cdncontribute.geeksforgeeks.org/wp-content/uploads/memoryLayoutC.jpg

https://www.geeksforgeeks.org/memory-layout-of-c-program/

Manual Memory Management

- Programmer allocates and deallocates buffers in the **HEAP**
  - C language: *malloc() free()*
  - C++ language: *new* and *delete*

- Pros:
  - Higher **performance**
  - Deeper understanding of the program by the programmer
- Cons:
  - Code **complexity**
  - Higher **risk of bugs**
  - Lower programmer's **productivity**

- Languages:
  - Algol; **C; C++;** COBOL; Fortran; Pascal
- Tools for Memory Management Profiling: **Valgrind, GDB**

- http://www.memorymanagement.org/mmref/begin.html#manual-memory-management

Automatic Memory Management

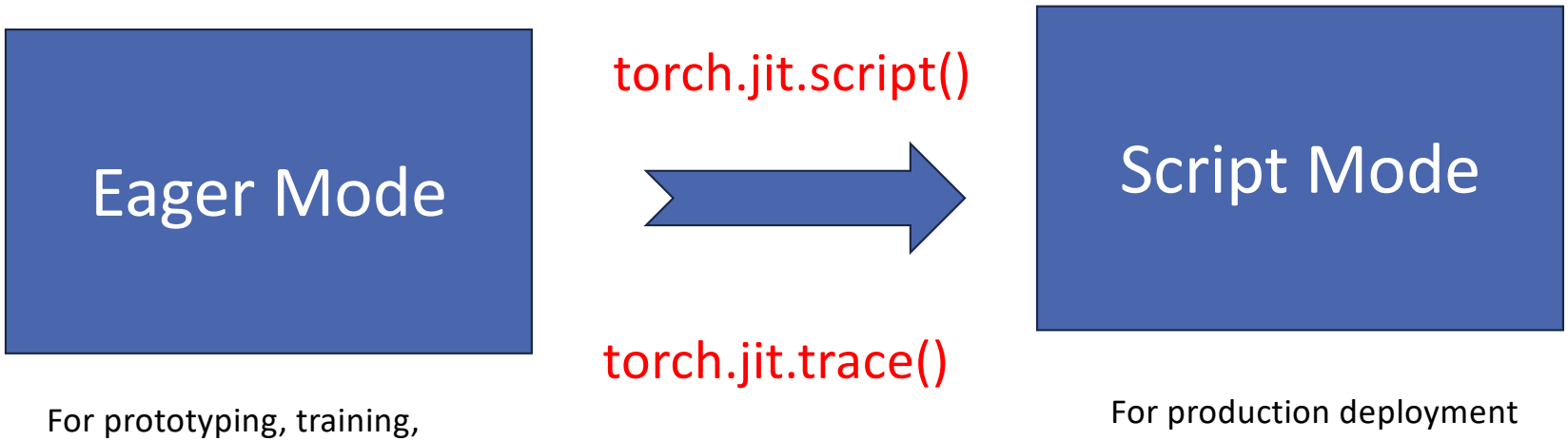
- Runtime system is in charge to allocate and deallocate buffers in the **HEAP:**
  - Allocation embedded in the language, ex. object creation
  - Recycling techniques – Garbage collection:
    - Keep track of all references to objects
    - Free objects that are not needed anymore
- Pros:
  - Higher programmer's **productivity**
  - Code **simplicity**
  - Lower risk of **bugs**
- Cons:
  - Lower **performance**
  - Lower memory management **time and space efficiency**
- Languages:
  - BASIC, Dylan, Erlang, Haskell, Java, JavaScript, Lisp, ML, Modula-3, Perl, PostScript, Prolog, **Python**, Scheme, Smalltalk, etc.

Python implementations

- Python Implementation
  - *a program or environment (runtime) which provides support for the execution of programs written in the Python language*
- **CPython** is the de-facto Python reference implementation written in C
- Alternative implementations
  - *Brython, CLPython, HotPy, IronPython, Jython, pyjs, PyMite, PyPy, pyvm, etc*.
- Compilers: compiling Python to C code
  - ***CPython**, 2c-python, GCC Python Front-end, etc*.
- Numerical Accelerators/Frameworks: offer accelerated numerical libraries (often c optimized libraries with Python APIs)
  - **PyTorch,** Numpy, Numba, Copperhead,

- https://wiki.python.org/moin/PythonImplementations
- https://medium.com/@elhayefrat/python-cpython-e88e975e80cd

PyTorch Approach to Static Graphs – Prior to PyTorch 2.0



Script Mode

- Script mode creates an intermediate representation (IR) of your PyTorch Eager module(through torch.jit.trace/torch.jit.script).
- The IR is internally optimized and utilizes PyTorch JIT compilation at runtime.
- PyTorch JIT compiler uses runtime information to optimize the IR.
- This IR is decoupled from the Python runtime.

TorchScript is legacy in PyTorch 2.0+, now superseded by torch.compile for most use cases

JIT Trace

- torch.jit.trace takes a data instance and your trained eager module as input.
- The tracer runs the supplied module and records the tensor operations performed.
- This recording is turned into a TorchScript module.

- Drawbacks:
  - It omits all control flow, data structures, and python constructs.
  - It can also create unfaithful representations without any warnings.

- Why it matters today (2025):
  - Tracing is still useful for exporting static parts of models to production.
  - But for most modern workflows, torch.compile and ONNX export have largely replaced tracing.

- Reading: https://towardsdatascience.com/pytorch-jit-and-torchscript-c2a77bac0fff

PyTorch Static Graphs

Examples adapted from PyTorch official documentation

EAGER TO SCRIPT MODE WITH `torch.jit.trace()`

Take an existing eager model, and provide example inputs.

The tracer runs the function, recording the tensor operations performed.

We turn the recording into a TorchScript module.

- Can reuse existing eager model code
- ⚠ Control-flow and data structures are ignored

```
import torch
import torchvision

def foo(x, y):
  return 2 * x + y

# trace a model by providing example inputs
traced_foo = torch.jit.trace(foo,
                             (torch.rand(3), torch.rand(3)))

traced_resnet = torch.jit.trace(torchvision.models.resnet18(),
                                torch.rand(1, 3, 224, 224))
```

EAGER TO SCRIPT MODE WITH `torch.jit.script()`

Write model directly in TorchScript, a high-performance subset of Python.

Pass instance of your model to `torch.jit.script()`

- Control-flow is preserved
- `print` statements can be used for debugging
- Remove the script() call to debug as a standard PyTorch module

```
class RNN(torch.nn.Module):
  def __init__(self, W_h, U_h, W_y, b_h, b_y):
    super(RNN, self).__init__()
    self.W_h = nn.Parameter(W_h)
    self.U_h = nn.Parameter(U_h)
    self.W_y = nn.Parameter(W_y)
    self.b_h = nn.Parameter(b_h)
    self.b_y = nn.Parameter(b_y)

  def forward(self, x, h):
    y = []
    for t in range(x.size(0)):
      h = torch.tanh(x[t] @ self.W_h + h @ self.U_h + self.b_h)
      y += [torch.tanh(h @ self.W_y + self.b_y)]
      if t % 10 == 0:
        print("stats: ", h.mean(), h.var())
    return torch.stack(y), h

rnn = RNN(torch.randn(3, 4), torch.randn(4, 4), torch.randn(4, 4),
torch.randn(4), torch.randn(4))
scripted = torch.jit.script(rnn)
```

EXPORTING A MODEL TO PRODUCTION

TorchScript models can be saved a model archive and loaded to run in PyTorch's just-in-time (JIT) compiler instead of the CPython interpreter.

C++ Tensor APIs support bindings to a wide range of languages and deployment environments.

The same TorchScript models can also be loaded in the PyTorch Mobile runtime.

```python
# Python: save model
traced_resnet = torch.jit.trace(torchvision.models.resnet18(),
                                torch.rand(1, 3, 224, 224))
traced_resnet.save("serialized_resnet.zip")
```

```cpp
// C++: load model
auto module = torch::jit::load("serialized_resnet.zip");
auto example = torch::rand({1, 3, 224, 224});

// Execute `forward()` using the PyTorch JIT
auto output = module->forward({example}).toTensor();
std::cout << output.slice(1, 0, 5) << '\n';
```

PYTORCH JIT INTERMEDIATE REPRESENTATION

```
graph(%0 : Float(3, 10), %1 : Float(3, 20), %2 : Float(3, 20), %3 : Float(80, 10)
      %4 : Float(80, 20), %5 : Float(80), %6 : Float(80)) {
  %7 : Float(10!, 80!) = aten::t(%3)
  %10 : int[] = prim::ListConstruct(3, 80)
  %12 : Float(3!, 80) = aten::expand(%5, %10, 1)
  %15 : Float(3, 80) = aten::addmm(%12, %0, %7, 1, 1)
  %16 : Float(20!, 80!) = aten::t(%4)
  %19 : int[] = prim::ListConstruct(3, 80)
  %21 : Float(3!, 80) = aten::expand(%6, %19, True)
  %24 : Float(3, 80) = aten::addmm(%21, %1, %16, 1, 1)
  %26 : Float(3, 80) = aten::add(%15, %24, 1)
  %29 : Dynamic[] = aten::chunk(%26, 4, 1)
  %30 : Float(3!, 20), %31 : Float(3!, 20), %32 : Float(3!, 20), %33 : Float(3!, 20) = prim::ListUnpack(%29)
  %34 : Float(3, 20) = aten::sigmoid(%30)
  %35 : Float(3, 20) = aten::sigmoid(%31)
  %36 : Float(3, 20) = aten::tanh(%32)
  %37 : Float(3, 20) = aten::sigmoid(%33)
  %38 : Float(3, 20) = aten::mul(%35, %2)
  %39 : Float(3, 20) = aten::mul(%34, %36)
  %41 : Float(3, 20) = aten::add(%38, %39, 1)
  %42 : Float(3, 20) = aten::tanh(%41)
  %43 : Float(3, 20) = aten::mul(%37, %42)
  return (%43, %41);
}
```

PYTORCH JIT INTERMEDIATE REPRESENTATION

```
graph(%0 : Float(3, 10), %1 : Float(3, 20), %2 : Float(3, 20), %3 : Float(80, 10)
      %4 : Float(80, 20), %5 : Float(80), %6 : Float(80)) {
  %7 : Float(10!, 80!) = aten::t(%3)
  %10 : int[] = prim::ListConstruct(3, 80)
  %12 : Float(3!, 80) = aten::expand(%5, %10, 1)
  %15 : Float(3, 80) = ate
  %16 : Float(20!, 80!) =
  %19 : int[] = prim::List
  %21 : Float(3!, 80) = at
  %24 : Float(3, 80) = ate
  %26 : Float(3, 80) = ate
  %29 : Dynamic[] = aten::
  %30 : Float(3!, 20), %31                                                    ListUnpack(%29)
  %34 : Float(3, 20) = ate
  %35 : Float(3, 20) = aten::sigmoid(%31)
  %36 : Float(3, 20) = aten::tanh(%32)
  %37 : Float(3, 20) = aten::sigmoid(%33)
  %38 : Float(3, 20) = aten::mul(%35, %2)
  %39 : Float(3, 20) = aten::mul(%34, %36)
  %41 : Float(3, 20) = aten::add(%38, %39, 1)
  %42 : Float(3, 20) = aten::tanh(%41)
  %43 : Float(3, 20) = aten::mul(%37, %42)
  return (%43, %41);
}
```

- Static typing
- Structured control flow (nested ifs and loops)
- Functional by default

Examples of compiler optimizations that can be applied

Operator Fusion: Combines multiple operations into one for efficiency.

Dead Code Elimination: Removes parts of the code that don't affect output, reducing memory and computation.

Common Subexpression Elimination: Identifies and reuses duplicate calculations.

Loop Unrolling: Improves performance by reducing loop overhead.

Constant Folding: Precomputes constant expressions at compile time.

Memory Layout Optimizations: Enhances data cache usage for faster access.

Tensor Memory Planning: Optimizes memory allocation for tensors.

Automatic Mixed Precision: Utilizes lower-precision computations where possible for speed and memory savings.

Static Shape Inference: Allows for optimizations in models with fixed input sizes.

Parallelism and Vectorization: Exploits opportunities to parallelize and vectorize operations for faster execution.

Target-dependent code generation: Taking parts of the program and compiling it for specific hw. Integration ongoing with several code generation frameworks: TVM, Glow, XLA

Tracing vs. Scripting

- Both tracing and scripting are methods used to convert PyTorch models to TorchScript, which is an intermediate representation of a PyTorch model that can be run in a high-performance environment independent of Python.

Tracing (torch.jit.trace)

Tracing records operations on example input.

- **Use Cases:**
  - Ideal for models with straightforward, static control flow.
  - Best for quick, easy conversions without dynamic behaviors.
- **Advantages:**
  - Simple, less modification required.
- **Limitations:**
  - Cannot capture dynamic behavior dependent on input data.
  - May miss complex control flows.

Scripting (torch.jit.script)

Scripting analyzes and converts Python code itself.

- **Use Cases:**
  - Suitable for models with dynamic control flow (if statements, loops).
  - Handles more complex model structures.
- **Advantages:**
  - Captures dynamic behaviors accurately.
  - Provides explicit error messages for easier debugging.
- **Limitations:**
  - Requires statically analyzable Python code.

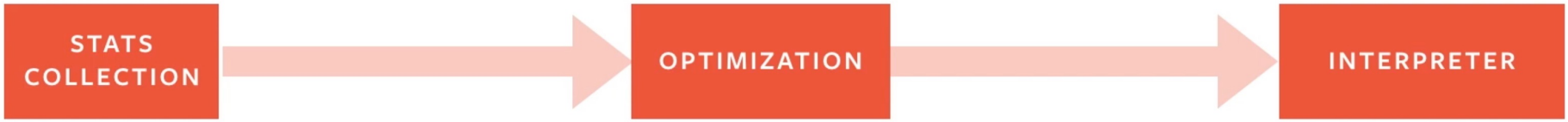
Hybrid Approach

In some cases, you might use both methods in different parts of your model:

- Use scripting for the parts of the model that have complex control flows or dynamic behaviors.
- Use tracing for simpler, more static parts of the model to speed up the conversion process.

PyTorch JIT Flow

User calls `forward()`



Specialization
- Shape
- Device
- requires_grad

Compiler optimizations
- Fusion (loop, op, etc)
- Algebraic rewriting
- Loop unrolling
- Code generation
- etc...

Execution
- Scheduling
- Parallelism

PyTorch: Static Graphs with JIT

Define model as a Python function

```
import torch

def model(x, y, w1, w2a, w2b, prev_loss):
  w2 = w2a if prev_loss < 5.0 else w2b
  y_pred = x.mm(w1).clamp(min=0).mm(w2)
  loss = (y_pred - y).pow(2).sum()
  return loss

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2a = torch.randn(H, D_out, requires_grad=True)
w2b = torch.randn(H, D_out, requires_grad=True)

graph = torch.jit.script(model)

prev_loss = 5.0
learning_rate = 1e-6
for t in range(500):
  loss = graph(x, y, w1, w2a, w2b, prev_loss)

  loss.backward()
  prev_loss = loss.item()
```

Slide courtesy of Justin Johnson

PyTorch: Static Graphs with JIT

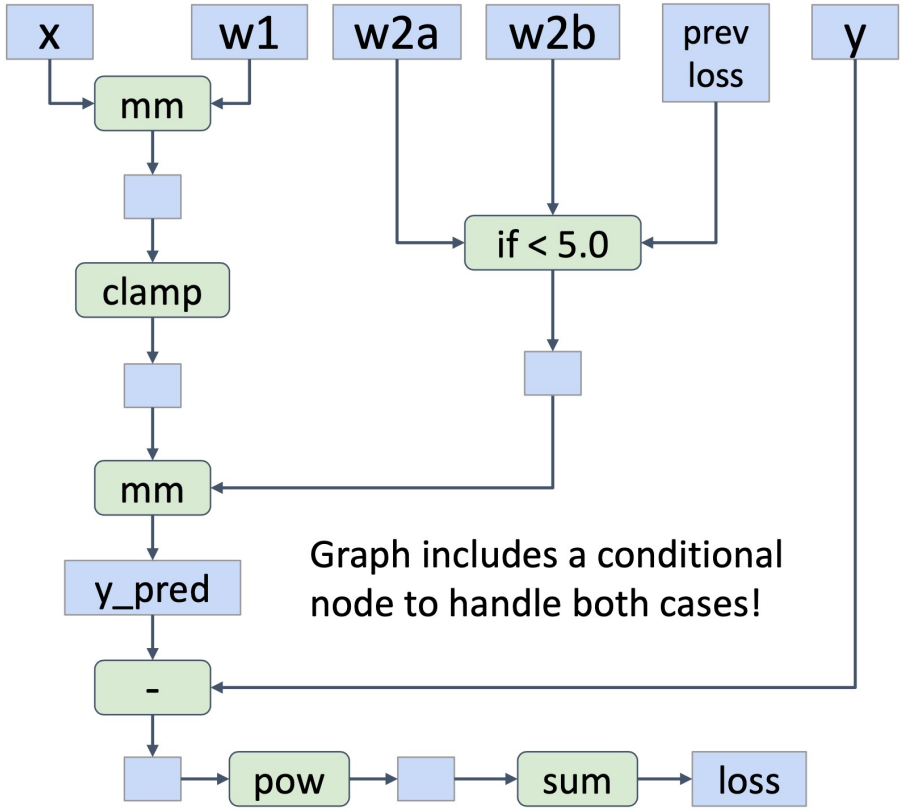
Just-In-Time compilation: Introspect the source code of the function, **compile** it into a graph object.

Lots of magic here!

```
import torch

def model(x, y, w1, w2a, w2b, prev_loss):
  w2 = w2a if prev_loss < 5.0 else w2b
  y_pred = x.mm(w1).clamp(min=0).mm(w2)
  loss = (y_pred - y).pow(2).sum()
  return loss

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=Tru
w2a = torch.randn(H, D_out, requires_grad=
w2b = torch.randn(H, D_out, requires_grad=

graph = torch.jit.script(model)

prev_loss = 5.0
learning_rate = 1e-6
for t in range(500):
  loss = graph(x, y, w1, w2a, w2b, prev_los

  loss.backward()
  prev_loss = loss.item()
```

Slide courtesy of Justin Johnson

PyTorch: Static Graphs with JIT




```
import torch

def model(x, y, w1, w2a, w2b, prev_loss):
  w2 = w2a if prev_loss < 5.0 else w2b
  y_pred = x.mm(w1).clamp(min=0).mm(w2)
  loss = (y_pred - y).pow(2).sum()
  return loss

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2a = torch.randn(H, D_out, requires_grad=True)
w2b = torch.randn(H, D_out, requires_grad=True)

graph = torch.jit.script(model)

prev_loss = 5.0
learning_rate = 1e-6
for t in range(500):
  loss = graph(x, y, w1, w2a, w2b, prev_loss)

  loss.backward()
  prev_loss = loss.item()
```


Slide courtesy of Justin Johnson

PyTorch: Static Graphs with JIT

```
import torch

def model(x, y, w1, w2a, w2b, prev_loss):
  w2 = w2a if prev_loss < 5.0 else w2b
  y_pred = x.mm(w1).clamp(min=0).mm(w2)
  loss = (y_pred - y).pow(2).sum()
  return loss

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2a = torch.randn(H, D_out, requires_grad=True)
w2b = torch.randn(H, D_out, requires_grad=True)

graph = torch.jit.script(model)

prev_loss = 5.0
learning_rate = 1e-6
for t in range(500):
  loss = graph(x, y, w1, w2a, w2b, prev_loss)

  loss.backward()
  prev_loss = loss.item()
```

Use our compiled graph object at each forward pass

Slide courtesy of Justin Johnson

PyTorch: Static Graphs with JIT

Even easier: add **annotation** to function, Python function compiled to a graph when it is defined

Calling function uses graph

```
import torch

@torch.jit.script
def model(x, y, w1, w2a, w2b, prev_loss):
  w2 = w2a if prev_loss < 5.0 else w2b
  y_pred = x.mm(w1).clamp(min=0).mm(w2)
  loss = (y_pred - y).pow(2).sum()
  return loss

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2a = torch.randn(H, D_out, requires_grad=True)
w2b = torch.randn(H, D_out, requires_grad=True)

prev_loss = 5.0
learning_rate = 1e-6
for t in range(500):
  loss = model(x, y, w1, w2a, w2b, prev_loss)

  loss.backward()
  prev_loss = loss.item()
```


Slide courtesy of Justin Johnson

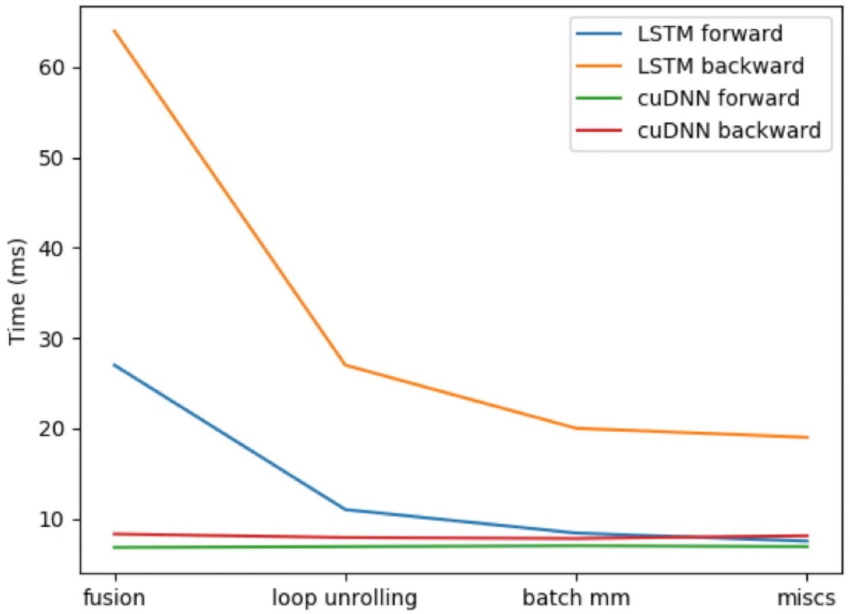
PyTorch JIT Summary: Key Functionalities

Tracing	Scripting	TorchScript	Serialization and Optimization
• Captures operations in functions/models. • Uses torch.jit.trace for graph representation. • Records tensor operations on example inputs.	• Converts Python code to TorchScript via torch.jit.script. • Useful for control flow elements (loops, if-statements). • TorchScript: Python subset, optimized for PyTorch.	• Result of tracing and scripting. • Statically-typed, optimized model representation. • Executable independently of Python runtime.	• JIT models can be saved and reloaded. • Suitable for different environments (e.g., servers). • JIT optimizations improve performance.

Impact of TorchScript on Performance

Read the blog post in this link:

https://pytorch.org/blog/optimizing-cuda-rnn-with-torchscript/



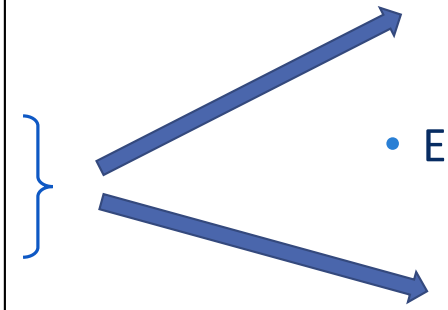
Optimizing CUDA Recurrent Neural Networks with TorchScript

Example JIT Compilation optimization: Fusion

- **Fusion** can significantly improve performance reducing the number of operations

- PyTorch code:

```
x = Variable(torch.randn(1, 10))
y = Variable(torch.randn(1,10))

xy = torch.mm(x.t(), y)
xy = xy * 100
xy = xy + 10

# Apply fusion then execute
print(xy)
```



- Example C implementation:

```
for (i = 0; i < x_rows; ++i)
  for (j = 0; j < y_cols; ++j)
    for (k = 0; k < y_rows; ++k)
      xy[i][j] += x[i][k] * y[k][j];

for (i = 0; i < x_rows; ++i)
  for (j = 0; j < y_cols; ++j)
    xy[i][j] = xy[i][j] * 100;

for (i = 0; i < x_rows; ++i)
  for (j = 0; j < y_cols; ++j)
    xy[i][j] = xy[i][j] + 10;
```

- Example C implementation with **Fusion:**

```
for (i = 0; i < x_rows; ++i)
  for (j = 0; j < y_cols; ++j) {
    for (k = 0; k < y_rows; ++k)
        xy[i][j] += x[i][k] * y[k][j];
    xy[i][j] = xy[i][j] * 100 + 10;
  }
```

Example JIT Compilation optimization: OOO and work scheduling

- **Out of order execution** and automatic **work scheduling**
- PyTorch code:

```
from torch.autograd import Variable

x = Variable(torch.randn(1000,1000))
y = Variable(torch.randn(1000,1000))
z = Variable(torch.randn(1000,1000))

xy = torch.mm(x, y)
xz = torch.mm(x, z)

xy = xy + 100
xz = xz + 1000

# Reorder, fuse, and execute on
different devices (GPUs or cores)
print(xy + xz)
```

- Reorder, fuse, and schedule on different devices

```
for (i = 0; i < x_rows; ++i)
  for (j = 0; j < y_cols; ++j) {
    for (k = 0; k < y_rows; ++k)
        xy[i][j] += x[i][k] * y[k][j];
    xy[i][j] = xy[i][j] + 100;
  }
```

```
for (i = 0; i < x_rows; ++i)
  for (j = 0; j < z_cols; ++j) {
    for (k = 0; k < z_rows; ++k)
        xz[i][j] += x[i][k] * z[k][j];
    xz[i][j] = xz[i][j] + 1000;
  }
```

For More Information

- For more information on TorchScript, visit https://pytorch.org/docs/stable/jit.html.
- Interactive Tutorial: https://pytorch.org/tutorials/beginner/Intro_to_TorchScript_tutorial.html

2025 PyTorch JIT Update

- **JIT is now mostly legacy.** PyTorch maintainers have shifted focus from torch.jit to **torch.compile** (via TorchDynamo and TorchInductor).
- The **flow is conceptually the same** (collect info → optimize → run), but now with more flexibility: torch.compile can target multiple backends (CPU, CUDA, XLA, Triton).
- PyTorch JIT was the "first generation" compiler, while torch.compile represents the new unified and more powerful compiler stack.